Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description

Portable Test Application

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	Thomas Marmeth Software Developer	2011-10-21
Verified :	Eric Malo, Eng. System Engineer	2011-10-24
Approved :	Martin Ouellet, Eng. Software Departement Supervisor	2011-10-24

Document No.	**201111295**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-20**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-10-20	Thomas Marmeth	Released version
0.1	---	2011-10-18	Thomas Marmeth	Implemented comments from inspection I-1856
0	---	2011-09-29	Thomas Marmeth	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998, IEEE 1558-2005.

Table of Contents

REVISION LIST **2**

PREFACE **3**

TABLE OF CONTENTS **4**

LIST OF FIGURES **6**

1 INTRODUCTION **9**

1.1 Purpose 9
1.2 Scope 9
1.3 Definitions and Acronyms 10
1.3.1 Definitions 10
1.3.2 Acronyms 10
1.4 Document Organization 11
1.4.1 Software Design Organization 11
1.4.2 Representations and Diagrams 12

2 REFERENCES **13**

3 SOFTWARE DESIGN **14**

3.1 Hardware 14
3.1.1 HW_COM Connection 14
3.1.2 USB or Parallel Port (LPT1) 15
3.1.3 Computer 15
3.1.4 DCU and Doors 15
3.2 Software Development Environment 15
3.3 Functional and Logical Decomposition 15
3.3.1 Data Models 15
3.3.2 Controllers Classes 16
3.3.3 Views and Graphical User Interfaces 16
3.3.4 Communicator 17
3.4 Modules 17
3.4.1 Main Module 17
3.4.2 Communicator Module 17
3.4.3 SW Download Module 18
3.4.4 Events & Log Module 18
3.4.5 Parameter & Snapshots Module 18
3.4.6 Self-Test Module 18
3.4.7 Memory Dump Module 18
3.4.8 Graph Module 18
3.4.9 DCU Configuration Module 18
3.4.10 DCU Time Module 18
3.4.11 SW Versions Module 18
3.4.12 System Counters Module 19
3.4.13 Project File Module 19
3.4.14 Reports Module 19
3.4.15 Debug Log Module 19
3.5 Dependencies 19
3.5.1 PTA Com Library 21
3.5.2 GUI 21
3.5.3 Controller 21
3.5.4 Data Model 21
3.5.5 Debug Error Log 21
3.5.6 Command Encrypter 21

3.5.7 Communicator ... 21
3.6 Graphical User Interface ... 23
3.6.1 Main ... 23
3.6.2 Control Menu ... 28
3.6.3 PTA Properties ... 29
3.6.4 Password Entry ... 29
3.6.5 SW Version ... 30
3.6.6 Transfer Progress ... 30
3.6.7 Send Flash ... 31
3.6.8 SW Download ... 33
3.6.9 Browser ... 34
3.6.10 Real-Time Data Monitoring ... 35
3.6.11 Event Description File View ... 36
3.6.12 Event Attributes ... 37
3.6.13 Operating Parameters Description File View ... 38
3.6.14 Graph View ... 39
3.6.15 Event Log Erasure ... 40
3.6.16 Event Log Table ... 41
3.6.17 Memory Dump ... 43
3.6.18 Snapshot Erasure ... 44
3.6.19 Snapshot Table ... 45
3.6.20 Snapshot View ... 46
3.6.21 System Counters ... 48
3.6.22 Self Test ... 49
3.6.23 Time ... 50
3.6.24 DCU Code Status ... 50
3.6.25 DCU Configuration ... 51
3.6.26 Event Log Erasure ... 52
3.6.27 About ... 52
3.7 Detailed Design ... 53
3.7.1 Main Module ... 53
3.7.2 Communicator Module ... 59
3.7.3 SW Download Module ... 79
3.7.4 Events & Log Module ... 83
3.7.5 Parameters & Snapshots Module ... 91
3.7.6 Self-Test Module ... 101
3.7.7 Memory Dump Module ... 105
3.7.8 Graph Module ... 108
3.7.9 DCU Configuration Module ... 111
3.7.10 DCU Time Module ... 115
3.7.11 SW Versions Module ... 120
3.7.12 System Counters Module ... 126
3.7.13 CGM Log Module ... 136
3.7.14 Project File Module ... 137
3.7.15 Reports Module ... 143
3.7.16 Debug Log Module ... 144

APPENDIX A: CONFIGURABLE PARAMETERS ... 148

List of Figures

Figure 3-1 Hardware Block Diagram 14
Figure 3-2 Module Decomposition 17
Figure 3-3 Architecture and Dependencies 20
Figure 3-4 Main GUI 23
Figure 3-5 Menu Bar 24
Figure 3-6 Tool Bar 25
Figure 3-7 Status Bar 26
Figure 3-8 Control Menu GUI 28
Figure 3-9 PTA Properties GUI 29
Figure 3-10 Password Entry GUI 29
Figure 3-11 SW Version GUI 30
Figure 3-12 Transfer Progress GUI 30
Figure 3-13 Send Flash File GUI 31
Figure 3-14 SW Download GUI 33
Figure 3-15 Browser GUI 34
Figure 3-16 Real-Time Data Monitoring GUI 35
Figure 3-17 EDF View GUI 36
Figure 3-18 Event Attribute GUI 37
Figure 3-19 OPDF View GUI 38
Figure 3-20 Graph View GUI 39
Figure 3-21 Log Erasure GUI 40
Figure 3-22 Log Table GUI 41
Figure 3-23 Memory Dump GUI 43
Figure 3-24 Snapshot Erasure GUI 44
Figure 3-25 Snapshot Table GUI 45
Figure 3-26 Snapshot View GUI 46
Figure 3-27 System Counters GUI 48
Figure 3-28 Self Test GUI 49
Figure 3-29 Time GUI 50
Figure 3-30 DCU Code Status GUI 50
Figure 3-31 DCU Config GUI 51
Figure 3-32 MDS Log Erasure GUI 52
Figure 3-33 About GUI 52
Figure 3-34 DCU Application Task Schedule Data Flow Diagram 53
Figure 3-35 Flow Chart - DCU Application Startup 54
Figure 3-36 Flow Chart - PTA Shutdown 55
Figure 3-37 Function Accessibility 56
Figure 3-38 Flow Chart – Password Security 57
Figure 3-39 Flow Chart – Login 58
Figure 3-40 Flow Chart - Logout 59
Figure 3-41 Communication Module Classes 59
Figure 3-42 Sequence Diagram - CheckComQueue 61
Figure 3-43 Sequence Diagram - OnReceive 64
Figure 3-44 Sequence Diagram - StartFileXfer() (to DCU) 66
Figure 3-45 Sequence Diagram - OnReceive_FileXfer 70
Figure 3-46 Data Communication – File Transfer Flow Chart (Receive from DCU) 72
Figure 3-47 Sequence Diagram - StartFileXfer (receive from DCU) 73
Figure 3-48 Flowchart - Load Flash File 74
Figure 3-49 FlowChart – Update Boot Loader 75
Figure 3-50 Sequence Diagram - OnReceive_BSL () 78
Figure 3-51 SW Download Module Classes 79
Figure 3-52 Flow Chart - DCU Application Upgrade 80
Figure 3-53 Communication Diagram - DCU Application Upgrade 81
Figure 3-54 Events & Log Module Classes 83
Figure 3-55 Sequence Diagram - UpdateEDF_OPDF() 83
Figure 3-56 Sequence Diagram - EDFChanged() 84

Figure 3-57 Flow Chart - EDF Read & Write 85
Figure 3-58 Sequence Diagram - ReadEDFFile() 87
Figure 3-59 Communication Diagram - EventSync 88
Figure 3-60 Communication Diagram - Log Download & Display 89
Figure 3-61 Flow Chart - Log Erasure 90
Figure 3-62 Communication Diagram - Log Erasure 91
Figure 3-63 Parameters & Snapshot Module Classes 92
Figure 3-64 Sequence Diagram - ReadOPDFFile() 94
Figure 3-65 Communication Diagram - Start Polling Data 95
Figure 3-66 Sequence Diagram - Poll Data 96
Figure 3-67 Sequence Diagram - Data Reply 97
Figure 3-68 Flowchart - Snapshot Download 98
Figure 3-69 Sequence Diagram - SnapshotDownloadReply 99
Figure 3-70 Communication Diagram - Snapshot Erasure 100
Figure 3-71 Flow Chart – Snapshot Erasure 101
Figure 3-72 Self-Test Module Classes 102
Figure 3-73 Flow Chart - Self-Test 103
Figure 3-74 Communication Diagram - Self-Test 104
Figure 3-75 Memory Dump Module Classes 105
Figure 3-76 Flow Chart - Memory Dump 106
Figure 3-77 Communication Diagram - Memory Dump 107
Figure 3-78 Graph Module Classes 108
Figure 3-79 Flow Chart - Graph Download 109
Figure 3-80 Communication Diagram - Graph Download 110
Figure 3-81 DCU Configuration Module Classes 111
Figure 3-82 Flow Chart - DCU Configuration Retrieval 112
Figure 3-83 Communication Diagram - Configuration Retrieval 113
Figure 3-84 Flow Chart – DCU Configuration Change 114
Figure 3-85 Communication Diagram – DCU Configuration Change 115
Figure 3-86 DCU Time Module Classes 115
Figure 3-87 Flow Chart - DCU Time Retrieval 116
Figure 3-88 Communication Diagram - DCU Time Retrieval 117
Figure 3-89 Flow Chart - DCU Time Set 118
Figure 3-90 Communication Diagram - DCU Time Set 119
Figure 3-91 Self-Test Module Classes 120
Figure 3-92 Flow Chart - SCI Version 121
Figure 3-93 Communication Diagram - SCI Version 122
Figure 3-94 Flow Chart - SW Version (EDF & OPDF) 123
Figure 3-95 Communication Diagram - SW Version (EDF & OPDF) 124
Figure 3-96 Flow Chart - DCU Code Status 125
Figure 3-97 Communication Diagram - DCU Code Status 126
Figure 3-98 System Counters Module Classes 126
Figure 3-99 Flow Chart - System Counters Download 128
Figure 3-100 Communication Diagram - System Counters 129
Figure 3-101 Communication Diagram - Reset Door Cycles 130
Figure 3-102 Communication Diagram - Reset Operating Hours 131
Figure 3-103 Communication Diagram - Reset Event Occurrences 132
Figure 3-104 Communication Diagram - Reset Last DCU Configuration Occurrence 133
Figure 3-105 Communication Diagram - Reset Last SW Download Occurrence 134
Figure 3-106 Communication Diagram - Reset Last Log Erasure Occurrence 135
Figure 3-107 CGM Log Module Classes 136
Figure 3-108 Project Module Classes 137
Figure 3-109 Sequence Diagram – SaveProjectFile() 140
Figure 3-110 Flow Chart - Download All 142
Figure 3-111 Reports Module Classes 143
Figure 3-112 Flow Chart - Print 144
Figure 3-113 Debug Log Module Classes 145
Figure 3-114 Sequence Diagram - AddCommLog() 146

Figure 3-115 Sequence Diagram - AddErrLog() 147

1 INTRODUCTION

1.1 Purpose

The purpose of this document is to present the Software Design Description (SDD) for Generic Software Documentation Project Portable Test Application (PTA). The Portable Test Application provides the capability to interact with the Door Control Unit (DCU) in an easy to use Graphical User Interface (GUI).

The intended audience is the Vapor Stone Rail Systems software development and maintenance team, Transit Authority and Car Builder.

1.2 Scope

The Portable Test Interface is a PC based data acquisition, database, and monitoring system. It is used to assist in the maintenance of the Door Control Unit (DCU). This software allows the user to interactively monitor the operations of the DCU and includes functions such as real-time monitoring of door activity and signals, retrieval and storage of DCU fault logs, getting result from DCU Self-Tests results, and status retrieval of the DCU system. All communication is done via a serial, USB or Ethernet port connection. In addition, the PTA allows the user to view graphs representing door positions, speed, current, PWM, and position error versus time. In essence, the PTA shows the user a graphical representation of the data the DCU received.

1.3 Definitions and Acronyms

1.3.1 Definitions

Boot Loader	Software starting the Door Control Module application.
Car Network	Data bus connecting the Door Control Module with the rest of the train
Communication Gateway Module	A specific processor based electronic module running a Communication Gateway Module Application. It acts as gateway between the Door Control Module Application and the Car Network/Portable Test Application.
Door Control Module	A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.
Door Control Unit	The final unit that is used to control one or more door panels. This Unit is normally made of one or two door control modules running the Door Control Module Application and possibly one Communication Gateway Module (CGM) Application.
Door Module System	A term that is used for generically identifying a processor based electronic module running any software within any DCU.
HW_COM	Communication interface used for communication between Portable Test Application and Door Control Module application, that can be USB, serial or Ethernet.

1.3.1.1 Event Level Definitions

Event:	Term used to globally identify an Info, an Exception or a Failure.
Info:	Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.
Exception:	An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).
Failure:	A failure is an occurrence of a condition that is outside of the specified parameters and affecting this LRU. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.3.2 Acronyms

ADO	ActiveX Data Objects
API	Application Program Interface
ASCII	American Standard Code for Information Interchange
BIN	Binary, often in reference to binary files
BL	Boot Loader
CDRL	Contract Data Requirements List
CGM	Communication Gateway Module
COMM	Communication
CRC	Cyclic Redundancy Check
CRC32	32-bit Cyclic Redundancy Check
CSV	Comma Separated Value
DCM	Door Control Module
DCU	Door Control Unit
DLL	Dynamic Link Library
DM	Data Model

DWL	Download
ECD	Engineering Change Document
EDF	Event Description File
GUI	Graphical User Interface
ID	Identification
IEEE	Institute of Electrical and Electronics Engineers
LDTS	Local Diagnostic and Test System
LED	Light Emitting Diode
LPT	Line Printer
LRU	Line Replaceable Unit
MDI	Multiple-Document Interface
MDS	Monitoring and Diagnostic System
MU	Multiple Units
MVC	Model View Controller
NVRAM	Non-Volatile Random Access Memory
OCX	OLE Custom Control
OLE	Object Linking and Embedding
OPDF	Operating Parameters Description File
PC	Personal Computer
PSI	Pounds per Square Inch
PTA	Portable Test Application
PWM	Pulse Width Modulation
RAM	Random Access Memory
RTDM	Real Time Data Monitoring
RTP	Real-Time Transport Protocol
SCI	Software Configuration Item
SDD	Software Design Description
SOS	Secondary Obstruction Sensing
SRAM	Static Random Access Memory
SRS	Software Requirements Specifications
SW	Software
TBD	To Be Determined
TL	Train Line
TM	TradeMark
TS	Technical Specification
UML	Unified Modeling Language
USB	Universal Serial Bus
UTC	Universal Time, Coordinated
VDC	Volt Direct Current
VSRS	Vapor Stone Rail Systems
XML	Extensible Markup Language
FMP	Fault Management Plan
SACS	Software Applications Configuration

1.4 Document Organization

1.4.1 Software Design Organization

This document presents the software design decisions and architecture in section 3. There are several views, decompositions and subsections to the software design. They are :

- Section 3.1 Hardware: presents the hardware elements, their purpose and their relation to each other. This section gives an idea of how and where the PTA is connected to the other components of the entire Door System.
- Section 3.2 Software Development Environment: This section presents the software tools used for development.

- Section 3.3 Functional and Logical Decomposition: Gives an explanation of the architectural elements of the Software, grouped by common functionalities or logical purpose.
- Section 3.4 Modules: Gives an explanation of the architectural elements of the Software divided into Modules.
- Section 3.5 Dependencies: This section lists the dependencies of the PTA software.
- Section 3.6 Graphical User Interface: This section presents the software interface.
- Section 3.7 Detailed Design: presents the inner workings of functionalities by using diagrams describing the behavior of the functions.

1.4.2 Representations and Diagrams

The presented diagrams are compliant or inspired by UML notation. Section 3 uses package diagrams, communication diagrams, flow charts and sequence diagrams to illustrate architecture and behavior of the software. It is highly recommended to have some knowledge of UML notation to interpret these diagrams.

The Sequence diagrams were generated by Altova UModel. In the sequence diagrams, the following terminology is used:

- Loop: Indicates a loop on the following condition. In C#, this can translate into a For loop.
- Alt: Indicates a condition to be filled. In C# this can translate into If clauses.
- Opt: Indicates options, similar to Alt. In C# this can translates into Switch Cases.

The sequence diagrams are used to show calls between functions, but not code or operations. With some exceptions, this may result in seemingly empty loops or portions of the diagram. These areas are not empty but have precise operations that are too detailed or not pertinent to the overall software architecture.

Consequently, the Flowcharts and other diagram have a similar level of abstractions. Some activities will encompass others and not every operation will be detailed.

This level of abstraction was chosen to help the readability of the diagrams and contribute to the maintainability of this document.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, 23 September 1998, IEEE, www.ieee.org
IEEE Std 1558-2005	IEEE Standard for Software Documentation for Rail Equipment and Systems, 23 March 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

CMM-01	Software Engineering Process, Rev. 3, 2001 03 31
CMMI-02	Software Engineering Policy Manual, Rev. 1, 2008-07-18
201111489	Generic Software Documentation Project – Portable Test Interface Protocol, Interface Control Document (ICD), Rev. 01
201111274	Portable Test Application Software Requirements Specification Document, VSRS, Rev. 01
S17-07	Software Design Document Template, Rev.1, 2003-02-19
SACS	Software Applications Configuration Specifications, Project's specific
FMP	Fault Management Plan, Project's specific

3 SOFTWARE DESIGN

3.1 Hardware

The hardware block diagram shown below consists of the DCU host computer system and its link to outside equipment.

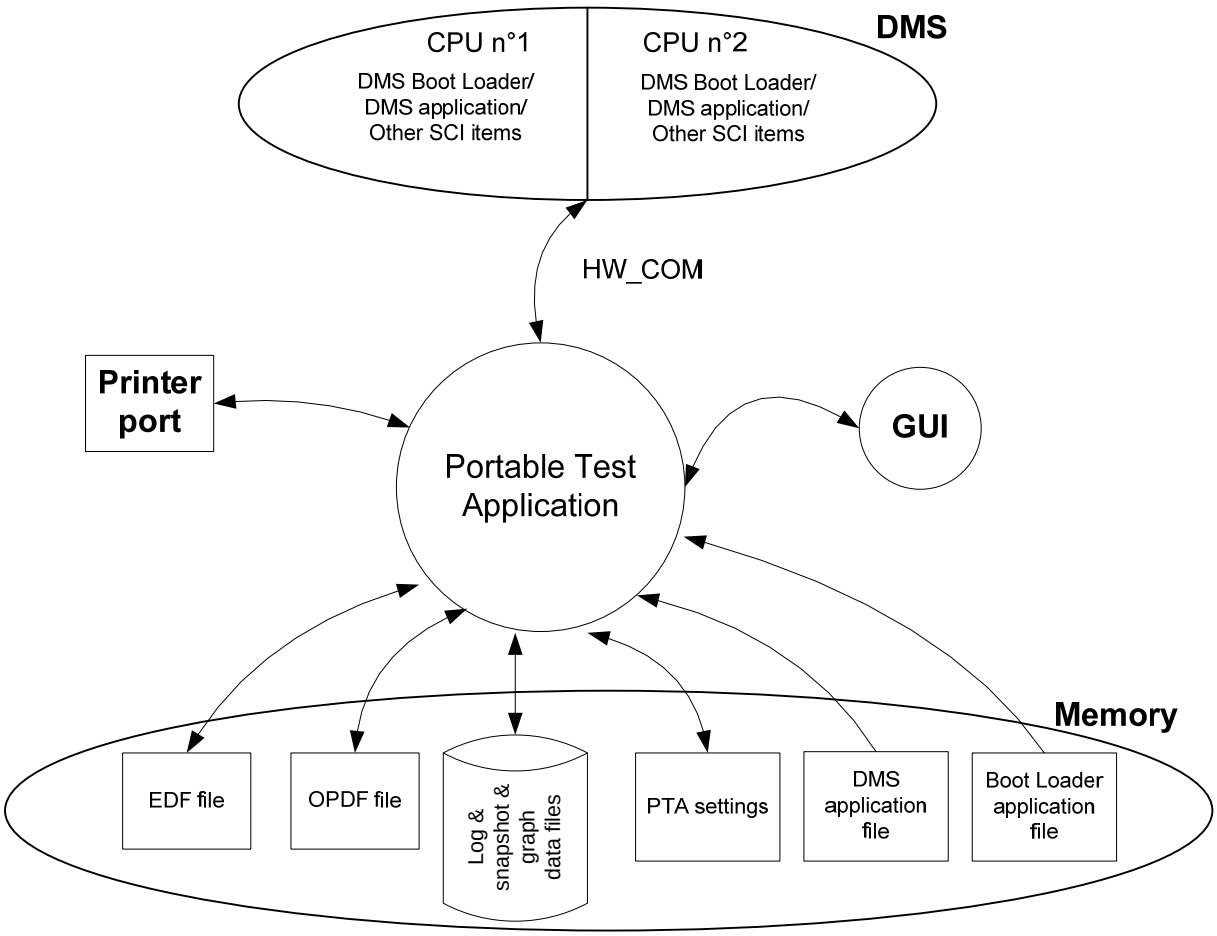


Figure 3-1 Hardware Block Diagram

The Portable Test Application software is installed in the PTA host computer, and is used to:

- Monitor, test and troubleshoot the Door Control System
- Retrieve and view the DCU Faults Log and Snapshots for system analysis.
- Performs the upgrade of the DCU application.

The following sections describe the hardware components used by the system.

3.1.1 HW_COM Connection

The HW_COM connection requires a serial, USB or Ethernet port on the laptop and a cable to link to the DCU.

3.1.2 USB or Parallel Port (LPT1)

The USB or Parallel Port is used to connect to a printer. The DCU information is sent to the printer via this port.

3.1.3 Computer

Any computer can be used to run the PTA.

3.1.4 DCU and Doors

The DCU is the component the PTA communicates with, in charge of controlling and monitoring the doors.

The DCU operates the door sides (left and right) independently. While both sides are controlled by a same DCU, there are two Door Control Modules (DCM) on the DCU itself. When the PTA communicates with a targeted door side, it is going through the same serial connection. In essence, the PTA is communicating with the same entity, but the message sent will have a target defined (left DCM or right DCM). The DCU itself will make sure that only the targeted side will respond to the message.

In the following functions, there is not always a distinction regarding the left or right door. That is because from the PTA's point of view, there is no difference in functions, features or commands in regards to the target side. The PTA can login to one or two DCMs. The logged in doors become the targets for all subsequent communications and commands.

Note: If communicating with both doors, then the PTA duplicates its action, always starting from the lower module ID.

3.2 Software Development Environment

The PTA is developed on a Windows Operating System computer. The development environment is Visual Studio 2008 Professional and the Microsoft .NET 3.5 Framework. The PTA is compiled using Visual Studio 2008. The installer is built within the Visual Studio 2008 solution and uses Windows installer 3.1.

The following external tools or libraries are needed:

- NPlot.dll: A .NET library used to draw plots and graphics.
- DevExpress Xtra Reports 2009 (v9.1): A Visual Studio plug-in used to make reports.

3.3 Functional and Logical Decomposition

3.3.1 Data Models

Data Model classes are responsible for storing all of the data and defining its format. The Data Models can be serialized to be saved onto a file (project file) and reloaded to their original state in a later execution of the PTA. The Data Models help control the access and centralize the data.

The Data Models are:

- ClientConstants
- DebugErrorLogDM
- DCUCodeStatDM
- DCUConfigDM
- EventDM
- GraphDM
- MainProjectDM
- MemoryDumpDM
- ParametersDM
- ProjectFileDM

- SelfTestDM
- SnapshotDM
- SWDwlDM
- SWVersionDM
- SysCountersDM
- TimeDM

3.3.2 Controllers Classes

The Controller classes are responsible for interpreting, manipulating and analyzing data or requests. They have a series of inputs and outputs that trigger certain actions.

The Controllers are:

- DCUConfigController
- EventController
- GraphController
- MainProjectController
- MemoryDumpController
- ParametersController
- ProjectFileController
- SelfTestController
- SWDwlController
- SWVersionController
- SysCountersController
- TimeController

3.3.3 Views and Graphical User Interfaces

The Views are responsible for taking inputs from the user and displaying the data to the user. The Views are mostly in the form of GUIs, Reports or other. The Views do not manipulate or store data; they are merely the interface between the external user and the PTA.

The Views are:

- AboutGUI
- BrowserGUI
- CarSelectionGUI
- ControlMenuGUI
- DataMonitoringGUI
- DCUCodeStatusGUI
- DCUConfigGUI
- DoorCmdGUI
- EDFGUI
- EventAttributesGUI
- GraphViewGUI
- LogErasureGUI
- LogTableGUI
- MainGUI
- MemoryDumpGUI
- OPDFGUI
- PasswordGUI
- PTAPropertiesGUI
- SendFlashGUI
- SnapshotErasureGUI
- SnapshotTableGUI
- SnapshotViewGUI
- SWDwlGUI
- SWVersionGUI

- o SysCountersGUI
- o TestGUI
- o TimeGUI
- o TransferProgressGUI

3.3.4 Communicator

The Communicator is in charge of everything related to communication with the DCU. There is a single communicator for this project: *DCUCommunicator*. The Communicator monitors the inputs and outputs of the device it is associated to.

The Communicator is the only entity aware of the communication protocol. It treats requests to send command from the PTA and implements the communication protocol. See Section 3.7.2 for more details.

3.4 Modules

Figure 3-2 presents the modules composing the Portable Test Application.

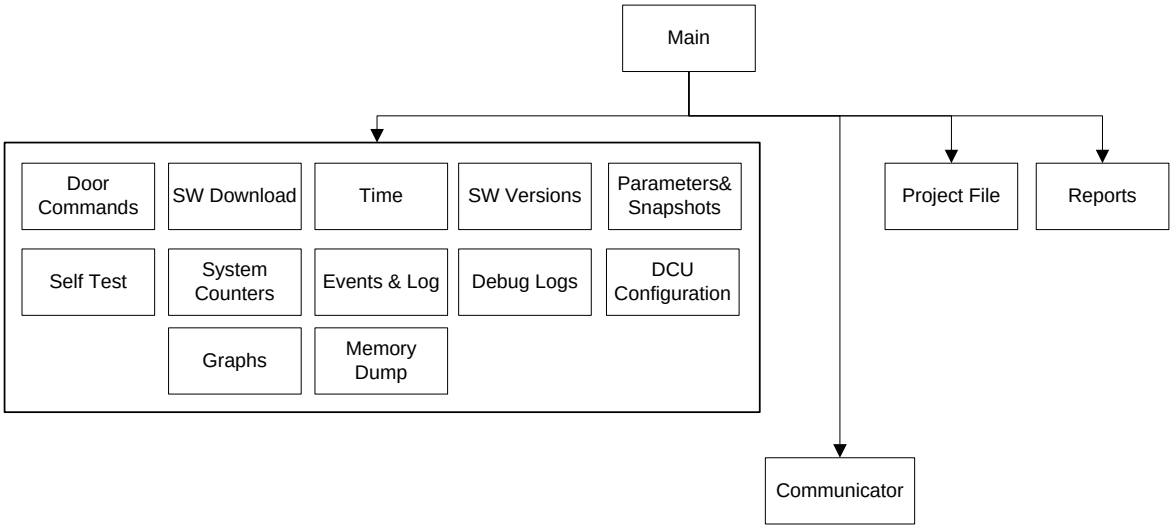


Figure 3-2 Module Decomposition

3.4.1 Main Module

The Main module includes a GUI, controller and data model set. It is in charge of coordinating the other modules, the communication and the high level PTA functions such as login. It is the starting point and the core that ties the different modules of the software together. In addition to being the basis for the other modules, it has the following responsibilities:

- Request Logins and Logouts of the DCU Applications or Boot Loaders.
- Verify and store the status of the security requirements.
- Save and Load the user configuration preferences.
- Manages which door side is the target: left, right or both.

3.4.2 Communicator Module

The Communicator has the following responsibilities:

- Establish a connection to the DCU via serial port connection.
- Encrypting and decrypting messages that are transferred between the DCU Application and the PTA.
- Manage the inputs and outputs of the serial port.
- Manage timers to maintain the login with the DCU Application.
- Manage the sequence for file transfers and Boot Loader upgrade.

3.4.3 SW Download Module

The SW Download module includes a GUI, controller and data model set. It is used to upload a DCM application file and verify its integrity. Once a file is uploaded, the module is used to force a restart of the newly programmed application.

3.4.4 Events & Log Module

The Events and Log module includes many GUIs, a controller and a data model. It is responsible for three global features: the event log, the event synchronization and the Event Description File. More specifically, it allows the user to:

- Download event log from the DCU
- Delete the event log data in the DCU
- View the events
- View the EDF file
- Poll and display events in real time (event synchronization)

3.4.5 Parameter & Snapshots Module

The Parameter and Snapshots module includes many GUIs, a controller and a data model. It is responsible for the DCU snapshot manipulation, the real time data monitoring and the Operating Parameters Description File (OPDF). More specifically, it allows the user to:

- Download the Snapshot from the DCU.
- Remove Snapshot from the DCU.
- View the snapshot list and details
- View the OPDF file
- Poll and display the parameter values in real time

3.4.6 Self-Test Module

The Self-Test module includes a GUI, controller and data model set. It is used to retrieve and display DCU's Self-Test results.

3.4.7 Memory Dump Module

The Memory Dump includes a GUI, controller and data model set. It is used to download the DCU SRAM memory and display it.

3.4.8 Graph Module

The Graph module includes a GUI, controller and data model set. It allows the user to view the DCU's last door open and close operation motor profile, a graphical representation of prepared data from the DCU Application.

3.4.9 DCU Configuration Module

The DCU Configuration module includes a GUI, controller and data model set. It provides the PTA user an interface to view and modify the DCU configuration parameters.

3.4.10 DCU Time Module

The DCU Time module includes a GUI, controller and data model set. It allows the user to retrieve, view and set the current time on the DCU board.

3.4.11 SW Versions Module

The SW Version module includes a GUI, controller and data model set. It takes care of retrieving the SCI, EDF and OPDF software versions on the DCU. The GUI displays the SCI versions to the user. The EDF and OPDF versions contained in this module reflect the versions in the DCU and are not displayed; they are used to determine if the EDF and OPDF files loaded on the PTA are out of synchronization with the ones on the DCU.

3.4.12 System Counters Module

The System Counters Module includes a GUI, controller and data model set. It is responsible for the downloading, displaying and resetting of the various counters maintained by the DCU. The following represents the functionality included in the System Counters Module:

- Download: Download all the System Counters from the DCU.
- Reset Door Cycles: Reset the door cycles counter in the DCU to zero
- Reset Operating Hours: Reset the operating hours counter in the DCU to zero
- Reset Event Occurrences: Reset all the counters for the number of times the various events in the DCU occurred.
- Reset Last DCU Configuration Occurrence
- Reset Last SW Download Occurrence
- Reset Last log Erasure Occurrence.

3.4.13 Project File Module

The Project File Module includes a controller and a data model set. This Module is responsible for two major features:

- Project file open and save
- Download all.

3.4.14 Reports Module

The Reports module includes a GUI and several reports. It allows the user to generate and print a report containing the previously downloaded information from the DCU, such as EDF, OPDF, summary information, event log and snapshots.

3.4.15 Debug Log Module

The Debug Log module includes two GUIs and a data model. It has two logs:

- A debug log that stores all the incoming and outgoing bytes, along with other messages;
- An error log that registers all the errors and failures caught by the application.

3.5 Dependencies

The Application design is inspired by the MVC (model view controller) design pattern. Each functionality mentioned in Section 3.4 follows the same functioning strategy. The reason behind this design is that it allows a common core of classes to be used for all PTA clients. Each client, however needs their own set of GUIs.

The Controllers are the middle-man between the GUI and the data model. They receive requests from the GUI and execute the requests, usually by forwarding a command to the Communicator. The controller also receives replies or change request. They interpret data from the Communicator and forward it to the DMs in the appropriate format. They have full control over the Data Model.

The Data Model is responsible for defining the data structure and storing it. When a model's data has been modified, the DM send an event notifying any anonymous listeners that a change has happened.

The listeners are the GUIs. The GUIs are the layer between the user and the rest of the application. When a change occurs in a data model, the GUIs react accordingly. The GUIs are receiving user inputs and display data. GUIs are the higher level of the application, meaning they are the owner or the starting point of the program declarations.

The following diagrams do not name the functionalities mentioned in the Decomposition description. That is because each functionality module has the same dependency pattern. For example, in the Graph Module, the GUI would be the GraphViewGUI, the Controller would be the GraphController and the DM would be the GraphDM.

Figure 3-3 shows the dependencies of the Portable Test Application.

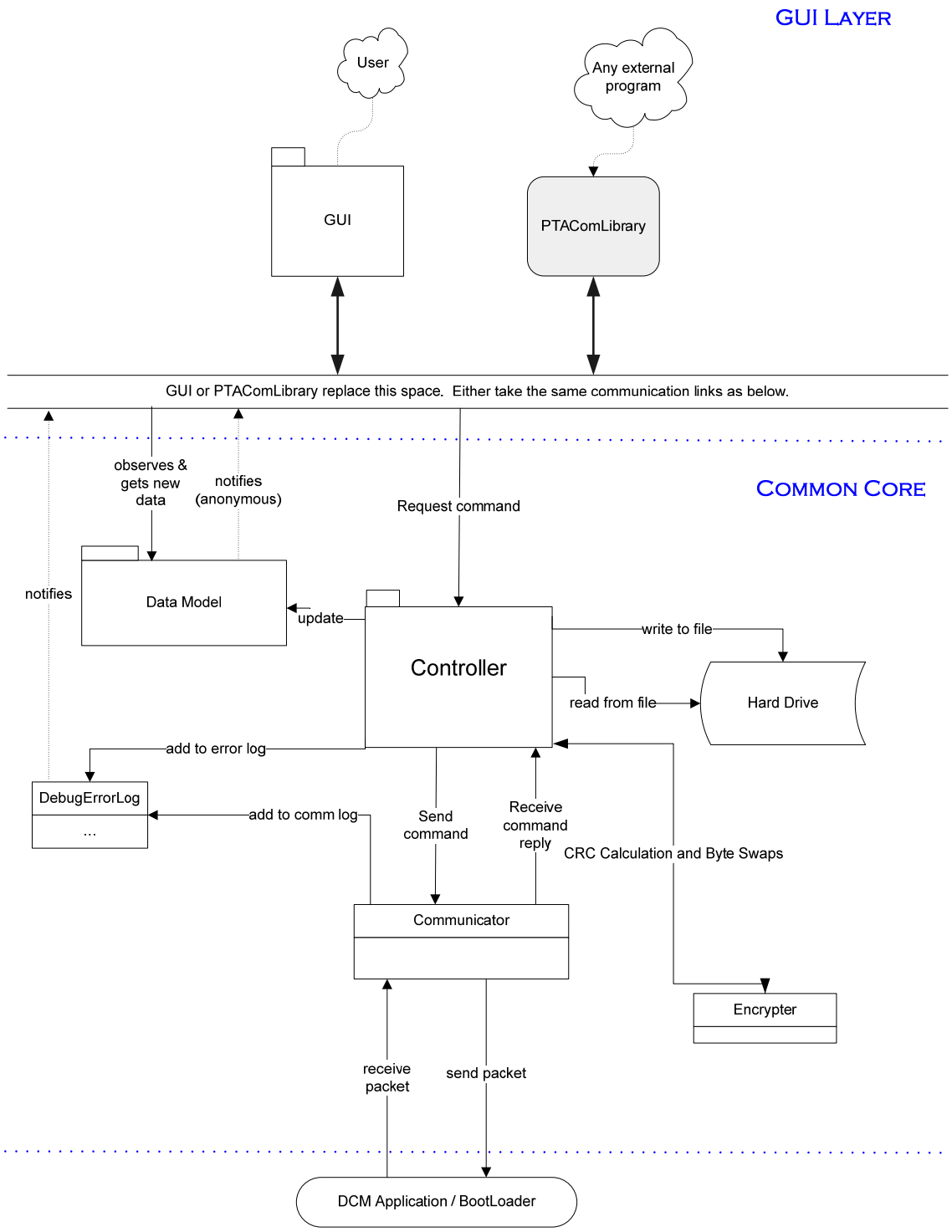


Figure 3-3 Architecture and Dependencies

3.5.1 PTA Com Library

The *PTA Com Library* sends commands to the *Controllers*. The *PTA Com Library* observes the *Data Models* and receives updated data from them.

3.5.2 GUI

The GUI package in the diagram is actually a set of entities with similar inputs and outputs. The *GUI* sends commands to the *Controllers*. The *GUI* observes the *Data Models* and receives updated data from them.

3.5.3 Controller

The Controller package in the diagram is actually a set of entities with similar inputs and outputs. The *Controller* receives commands from the *GUI* or the *PTA Com Library*. It sends the command to the *Communicator*. It receives a reply to the command from the *Communicator*. It updates the data in the *Data Models*. The *Controller* adds an error recording to the error log in *DebugErrorLog* whenever it encounters an error.

3.5.4 Data Model

The Data Model package in the diagram is actually a set of entities with similar inputs and outputs. The Data Models get updated by the Controllers. The Data Models notifies its observers (GUI or PTA Com Library) of any change. It also gets request from them for its data.

3.5.5 Debug Error Log

The *DebugErrorLog* behaves much in the same way as the *DataModels*. It notifies it's observers of any changes. Its error log gets updated by the *Controllers*. Its communication log gets updated by the *Communicator*.

3.5.6 Command Encrypter

The *Command Encrypter* gets CRC calculation requests. It also gets *Byte Swap* requests.

3.5.7 Communicator

The *Communicator* receives commands from the *Controllers*. It sends commands received to the *Controllers*. The *Communicator* sends packets to the *DCU* and receives packets from it as well. It requests that the *Command Encrypter* calculate the CRC and swap bytes. It will add to the communication log in the *DebugErrorLog*.

This page has been intentionnaly left blank

3.6 Graphical User Interface

The following graphical user interfaces describe the general layout of every screen and their purpose. These figures are an indication of what to find on the user interfaces and there might be some minor differences with the actual software.

3.6.1 Main

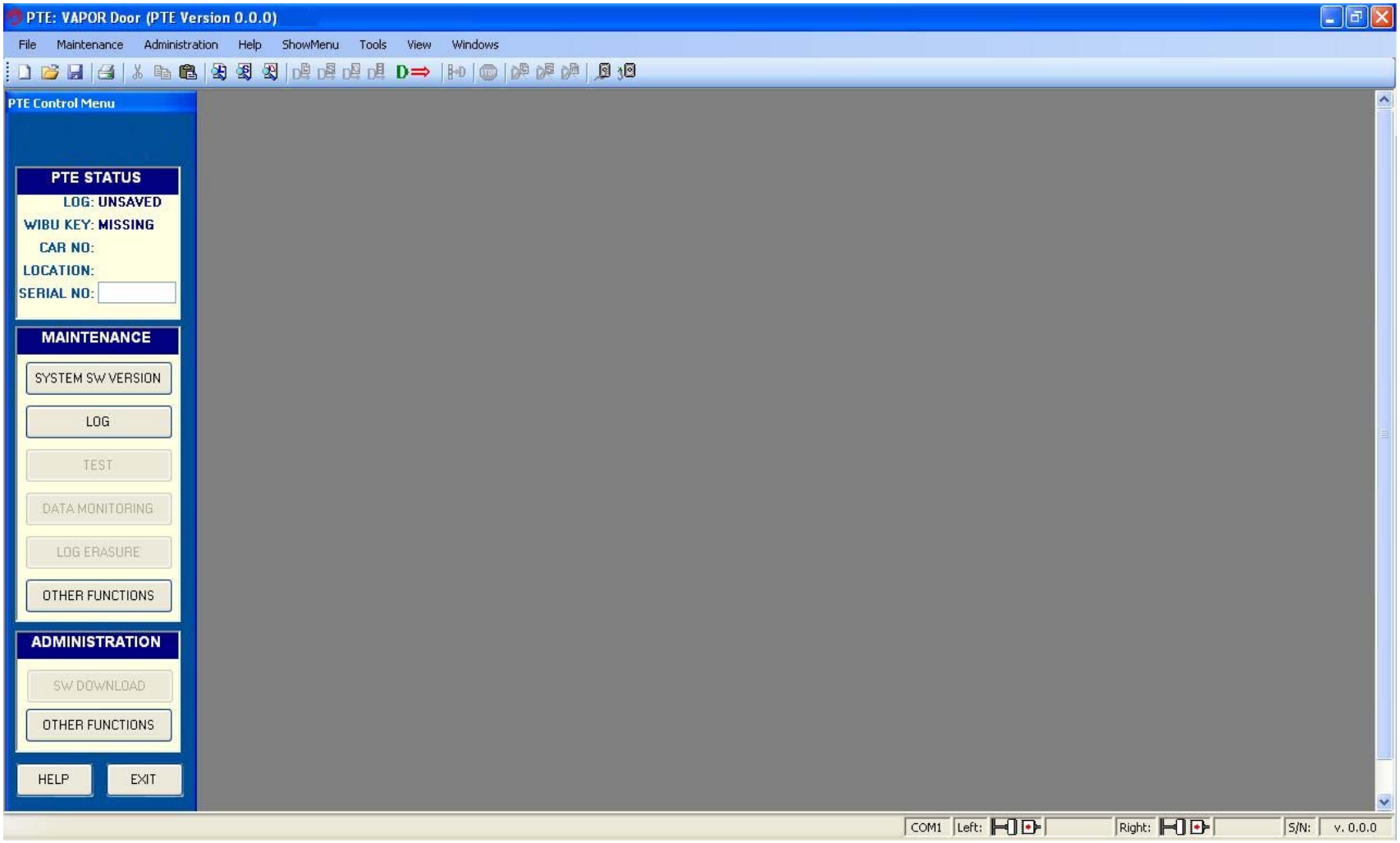


Figure 3-4 Main GUI

The PTA applies the *multiple-document interface* (*MDI*), with the main window maximized at the application start-up. The main window includes the menu bar, toolbar, and status bar.

The menu bar presents all operations of the PTA. The details are as follows:

Figure 3-5 Menu Bar

⇒ **File**
- New Project: Create a new PTA project.
- Open Project: Open a PTA data file.
- Save Project: Save the current PTA project.
- Save Project As: Save the current PTA project to user selected file.
- Save Log As: Save the current Log to a text file.
- Properties: Open the PTA properties window.
- Print: Print the current PTA project, including Log and Snapshots.
- Print OPDF: Print the Operating Parameters Description File.
- Print EDF: Print the Event Description File.
- Exit: Exit the PTA.

⇒ **Maintenance**
- System SW Version: Display the DCU software versions.
- Log View: Open the PTA project event log window.
- Test: Request and display the DCU Self-Test.
- Real-Time Data Monitoring: Monitor the DCU Status.
- Other Functions
  - Snapshot View: Open the PTA project snapshot list window.
  - Snapshot Download: Download the current DCU snapshot.
  - Log Download: Download the current DCU event log.
  - Graph View: Display the Door Graph.
  - Graph Download: Download the Door Graph.
  - Memory Dump: Download and display the DCU SRAM memory contents.
  - Browser: Perform real time monitoring to DCU operating parameters.
  - System Counters View: Display the DCU System Counters.
  - System Counters Download: Download the DCU System Counters.
  - Door Commands: Displays the Door Commands window.
  - Download All: Downloads the event log, snapshots, graphs and system counters consecutively.

⇒ **Administration**
- Log Erasure: Delete the DCU event log.

- SW Download: Open and verify the user selected DCU binary file. Then upgrade the current DCU with the user selected application file.
- Other Functions
  - LoadFlash
    - Open Flash File: Open DCU Boot Loader file.
    - Send Flash File: Send DCU Boot Loader file to DCU.
  - DCU Configure: Perform the configuration to the current DCU.
  - Snapshot Erasure: Delete all snapshots inside DCU.
  - Password: Allows the user to enter the administration password to unlock all administration functions

⇒ **Help**
- Help Content: Open the PTA help file.
- About: Open the PTA About window.
- Web Home: Open the Vapor Stone Rail Systems' web page with Internet explorer.

⇒ **ShowMenu**: Set the focus on the PTA Control Menu.

⇒ **Tools**
- Time: Retrieve, display, and set the DCU date and time.
- Operating Parameters List: Display the Operating Parameters List.
- Events Description: Display the Event Description list.
- Update EDF&OPDF: Update the EDF (Event Description File) and OPDF (Operating Parameters Description File) from the DCU.
- Code Status: Retrieve and display the DCU Boot Loader, DCU Application and Neuron Application's size, CRC information.
- Debug
  - Communication Debug: Open communication window to view messages sent to and received from DCU.
  - Error Log: Open PTA Error Log to view error messages.
- Login: Login to the connected DCU.
- Logout: Logout from the connected DCU.

⇒ **View**
- Tool Bar: Show or hide the Tool Bar in the PTA window
- Status Bar: Show or hide the Status Bar in the PTA window

⇒ **Windows**
- Cascade: Arrange the PTA open windows in Cascade mode
- Tile Horizontal: Arrange the PTA open windows in Tile Horizontal mode
- Tile Vertical: Arrange the PTA open windows in Tile Vertical mode
- Arrange Icon: Arrange the PTA open windows Icon
- Close All Windows: Closes all child windows, including the Control Menu.

The toolbar is shown below and links to the following functions:

Figure 3-6 Tool Bar

- New: Same as the File->New Project:
- Open: Same as the File->Open Project:
- Save: Same as the File->Save Project:
- Print: Same as the File->Print:
- Log View: Same as the Maintenance -> Log View:
- Snapshot View: Same as the Maintenance -> Other Functions -> Snapshot View:
- Graph View: Same as the Maintenance -> Other Functions -> Graph View:
- Log Download: Same as the Maintenance -> Other Functions -> Log Download:
- Snapshot Download: Same as the Maintenance -> Other Functions -> Snapshot Download:
- Graph Download: Same as the Maintenance -> Other Functions -> Graph Download:
- Memory Dump Download: Same as the Maintenance -> Other Functions -> Memory Dump:
- Download All: Same as the Maintenance -> Other Functions -> Download All:
- SW Application Download: Same as the Administration -> SW Download:
- Stop: Stop the current file download/upload (Upgrade):
- Log Erasure: Same as the Maintenance -> Log Erasure:
- Snapshot Erasure: Same as the Maintenance -> Other Functions -> Snapshot Erasure:
- Login: Log into DCU:
- Logout: Log out from DCU:

The Status Bar is shown below and presents the following information:

Figure 3-7 Status Bar

- Communication Port : Displays the selected communication port. Clicking it will open the PTA Properties window.

- Connected Icon: Shows the Login Status and allows the user to login and logout by clicking it. One per side.
- Login Target option: When connected, shows whether it is connected to the Boot Loader or the DCU Application.
- S/N: Only a label.
- Version: displays the version or the PTA.

3.6.2 Control Menu

Figure 3-8 Control Menu GUI

Every time you start a new PTA session, the PTA Control Menu is displayed. The PTA Control Menu contains three sections: PTA Status, Maintenance, Administration.The contents of the sections are described below:

PTA Status:

- LOG: Displays the saved status of the event log. It is not related to the project file.
- WIBU KEY: Displays the status of the password and Dongle, which allows access to administration functions.
- CAR NO: Displays the Car Number and the Position. Format will be “<Car No>, <Position”
- LOCATION: Displays the Location ID string of the connected DCU (or the value stored in the loaded project file).
- SERIAL NO: Used by technicians to enter an identifier. The serial number is not validated. It is displayed on the print and saved with the project file.

Maintenance:

- SYSTEM SW VERSION: Retrieve and Display the software versions.
- LOG: Open the PTA project event log window.
- TEST: Request and display the Self-Test.
- DATA MONITORING: Monitor the DCU Status by retrieving the Real-Time Data.
- LOG ERASURE: Delete the event log.
- OTHER FUNCTIONS: Show the following buttons:
    - SNAPSHOT VIEW: Open the PTA project snapshot list window.
    - SNAPSHOT DOWNLOAD: Download the current snapshot.
    - LOG DOWNLOAD: Download the current event log.
    - GRAPH VIEW: Display the Graph.
    - GRAPH DOWNLOAD: Download the Graph.
    - MEMORY DUMP: Download and display the DCU SRAM memory contents.
    - BROWSER: Perform real time monitoring to all operating parameters of DCU.
    - SYSTEM COUNTERS VIEW: Display the system Counters of DCU.
    - SYSTEM COUNTERS DOWNLOAD: Download the System Counters of DCU.

Administration:

- SW DOWNLOAD: Open and verify the user selected Application binary file, and then upgrade the current DCU with the user selected application file.
- OTHER FUNCTIONS: Show the following buttons.
    - OPEN FLASH FILE: Open DCU Boot Loader Application file.
    - SEND FLASH FILE: Send DCU Boot Loader Application file.
    - DCU CONFIGURE: Perform the configuration to the current DCU.
    - SNAPSHOT ERASURE: Delete all snapshots in DCU.

HELP : Open the PTA Help File
EXIT: Close the PTA Software.

3.6.3 PTA Properties

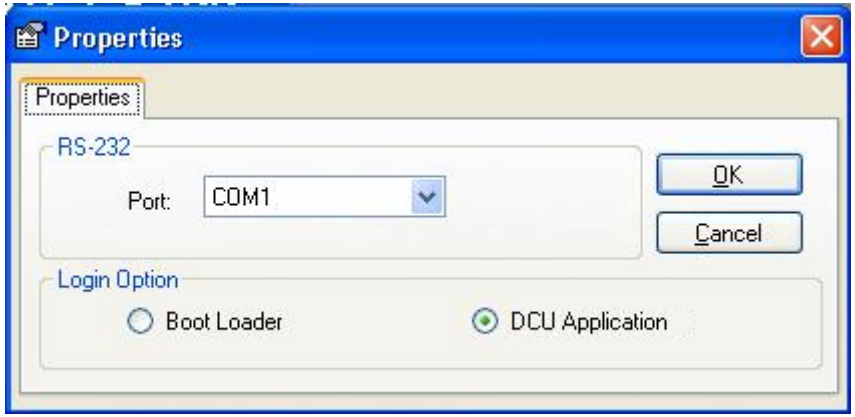


Figure 3-9 PTA Properties GUI

The Properties window allows the user to modify the communication device, and the PTA login option.

Two command buttons are used to cancel and change the user inputs:

- OK button: Apply current user settings and save to system registry.
- Cancel button: Exit properties without any modification to the PTA settings.

3.6.4 Password Entry

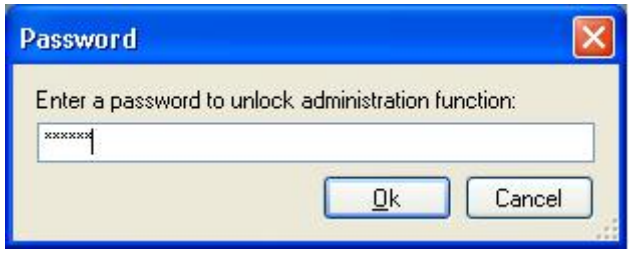


Figure 3-10 Password Entry GUI

The password entry screen is used to enter a hardcoded password. Correct entry of this password will grant access to all functions listed under the Administration menu or Administration section in the Control Menu.

3.6.5 SW Version

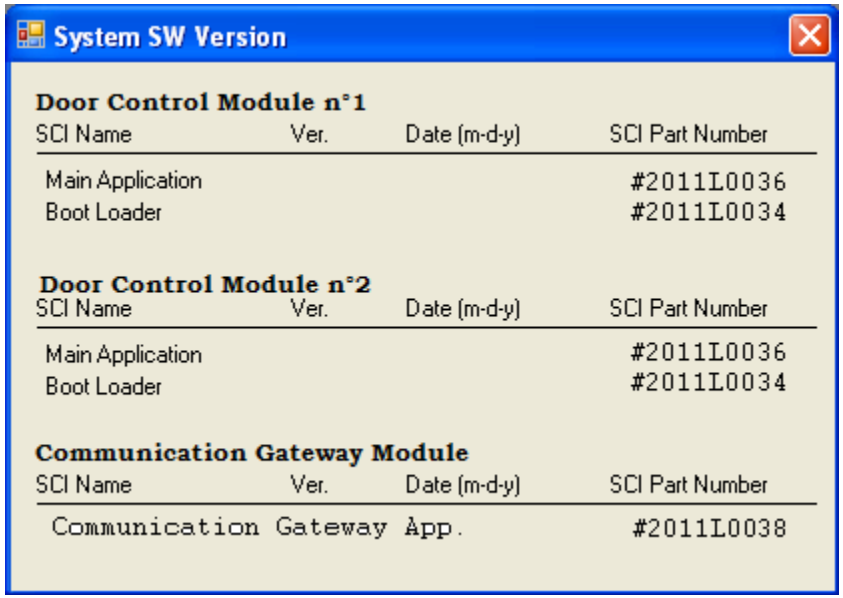


Figure 3-11 SW Version GUI

The SW Version displays the retrieved software versions from the DCU. Empty version fields means the application is not present or there have been no successful SW Version Requests made by the PTA.

3.6.6 Transfer Progress

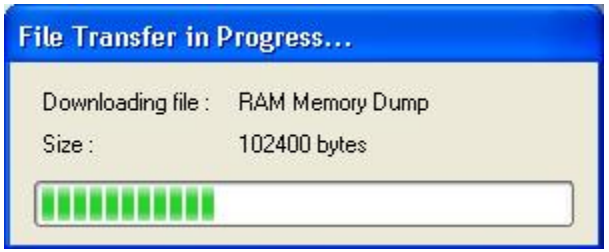


Figure 3-12 Transfer Progress GUI

The Transfer Progress is displayed during file transfers. It is an indicator of the progress in the file bytes received or sent.
The GUI will be open when a file transfer is started and close automatically when the transfer is finished or interrupted.

3.6.7 Send Flash

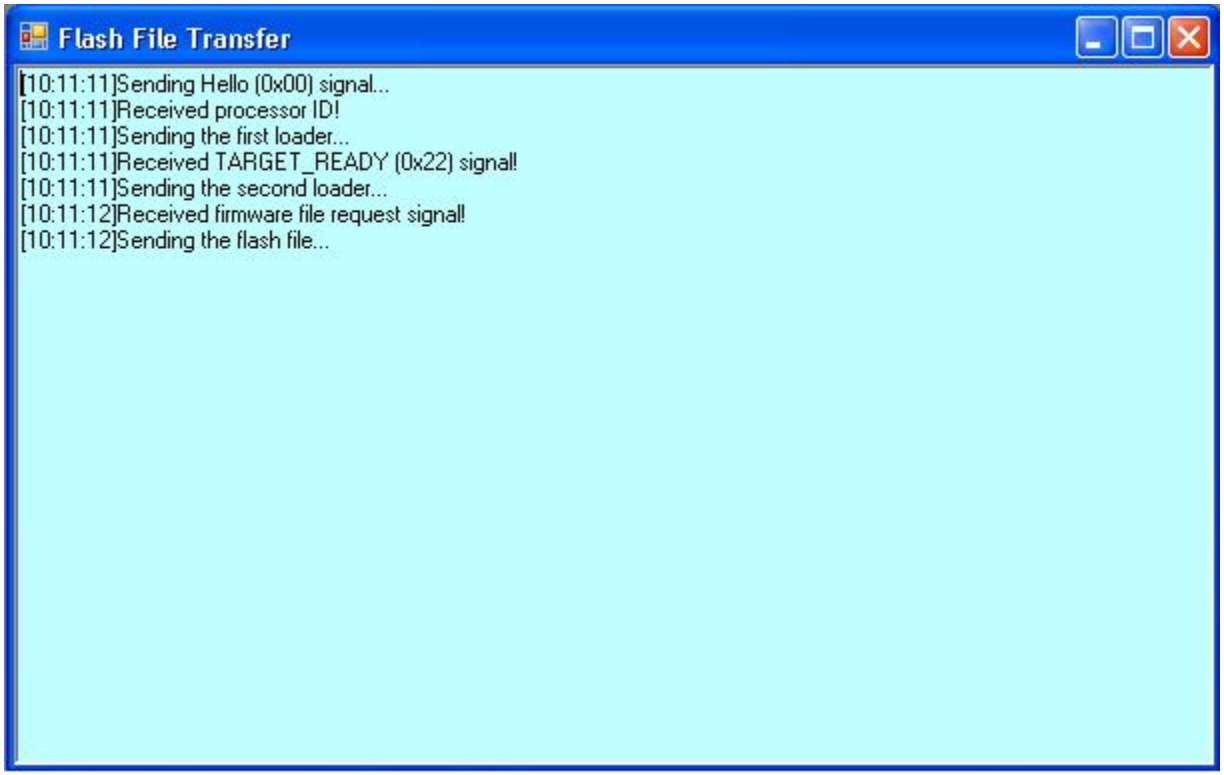


Figure 3-13 Send Flash File GUI

The Send Flash GUI appears when initiating a “Send Flash” action, used to upload the Boot Loader or RTP Tool into the DCU.

It displays the progress and steps of the transfer in real time. When the transfer is complete, the GUI will display a message notifying the user. The user must close the window manually after seeing this message.

This page has been intentionnaly left blank.

3.6.8 SW Download

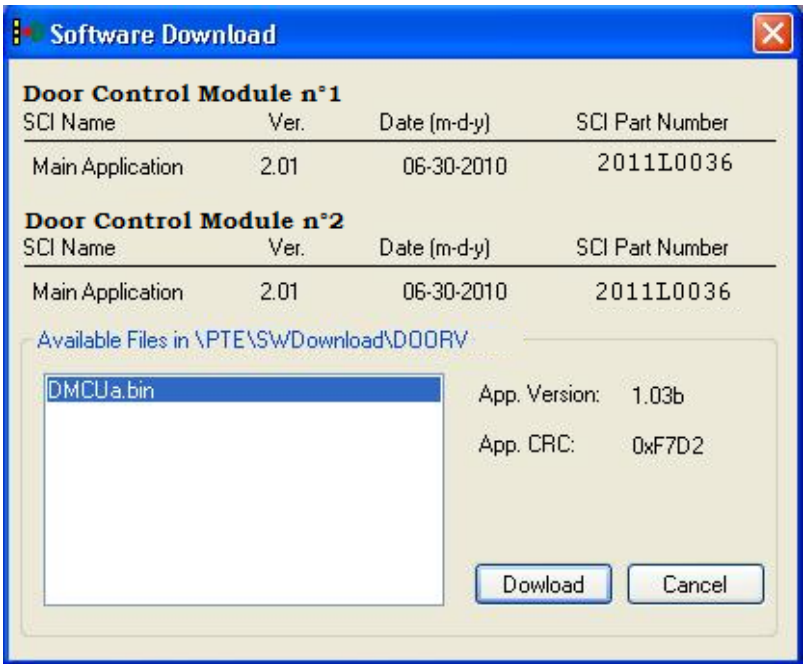


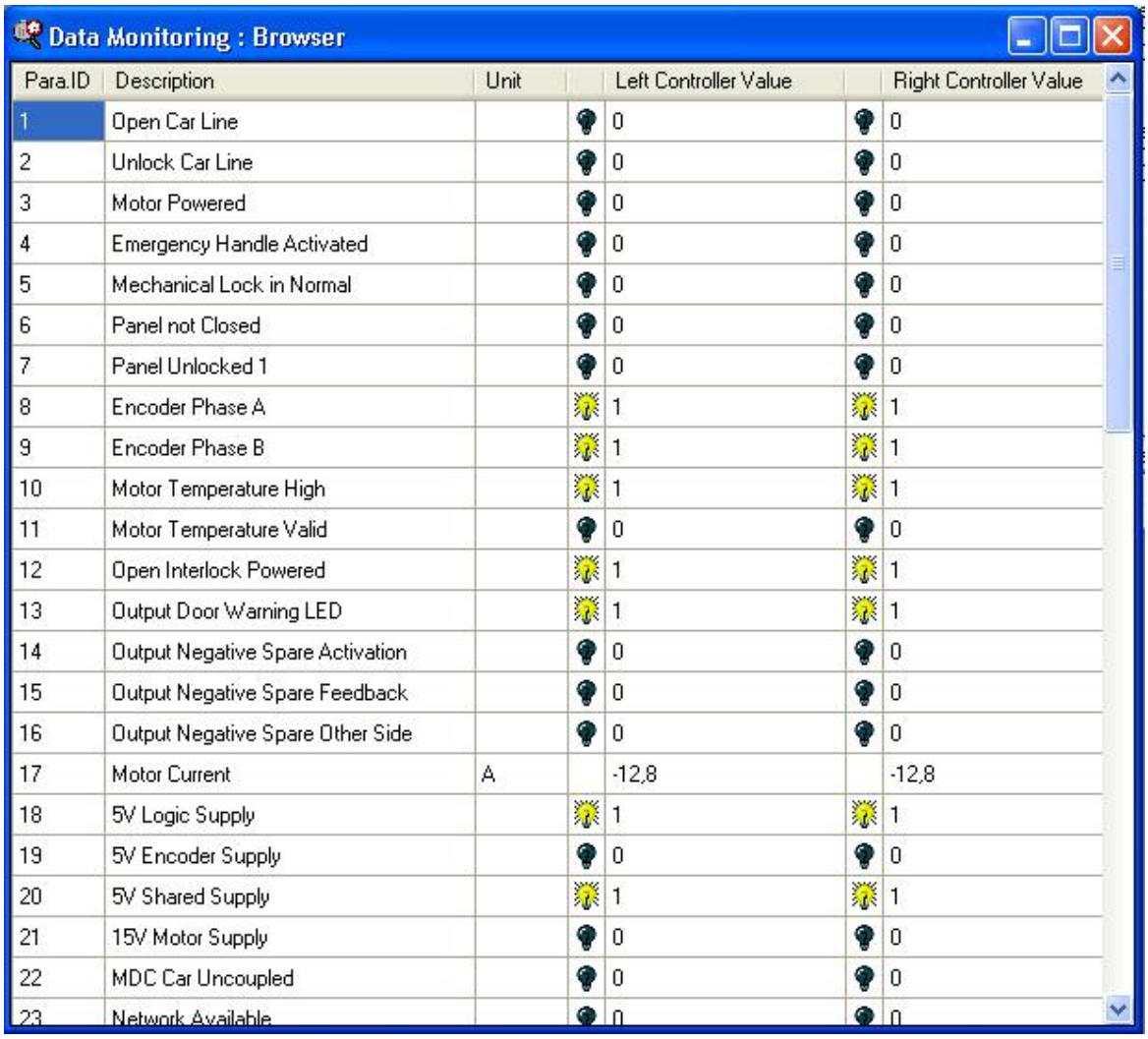
Figure 3-14 SW Download GUI

The Software Download GUI is related to the DCU Application upload. The upper section displays the current DCU Application loaded into the DCU (or shows the Boot Loader if no DCU Application). The version displayed is the same as the SW Version GUI.

The lower section allows the user to select which file will be sent to the DCU. When clicking a file, the PTA will validate the CRC and displays it, along with the SW version of the file.

Clicking on Download will ask for confirmation then initiate the transfer for the DCU Application. Clicking Cancel will close the GUI without starting a transfer.

3.6.9 Browser



Data Monitoring : Browser

Para.ID	Description	Unit		Left Controller Value		Right Controller Value
1	Open Car Line			0		0
2	Unlock Car Line			0		0
3	Motor Powered			0		0
4	Emergency Handle Activated			0		0
5	Mechanical Lock in Normal			0		0
6	Panel not Closed			0		0
7	Panel Unlocked 1			0		0
8	Encoder Phase A			1		1
9	Encoder Phase B			1		1
10	Motor Temperature High			1		1
11	Motor Temperature Valid			0		0
12	Open Interlock Powered			1		1
13	Output Door Warning LED			1		1
14	Output Negative Spare Activation			0		0
15	Output Negative Spare Feedback			0		0
16	Output Negative Spare Other Side			0		0
17	Motor Current	A		-12,8		-12,8
18	5V Logic Supply			1		1
19	5V Encoder Supply			0		0
20	5V Shared Supply			1		1
21	15V Motor Supply			0		0
22	MDC Car Uncoupled			0		0
23	Network Available			0		0

Figure 3-15 Browser GUI

The Browser GUI displays the real time monitoring of the status of all DCU signals. All signals contained in the PTA OPDF are listed in this GUI in the same order as in the OPDF.

When opened, the Browser GUI will constantly poll the DCU for the status of the signals. When closed, it will stop polling.

The signal status in this GUI is the same as the Data Monitoring GUI, but in a more detailed view.

3.6.10 Real-Time Data Monitoring

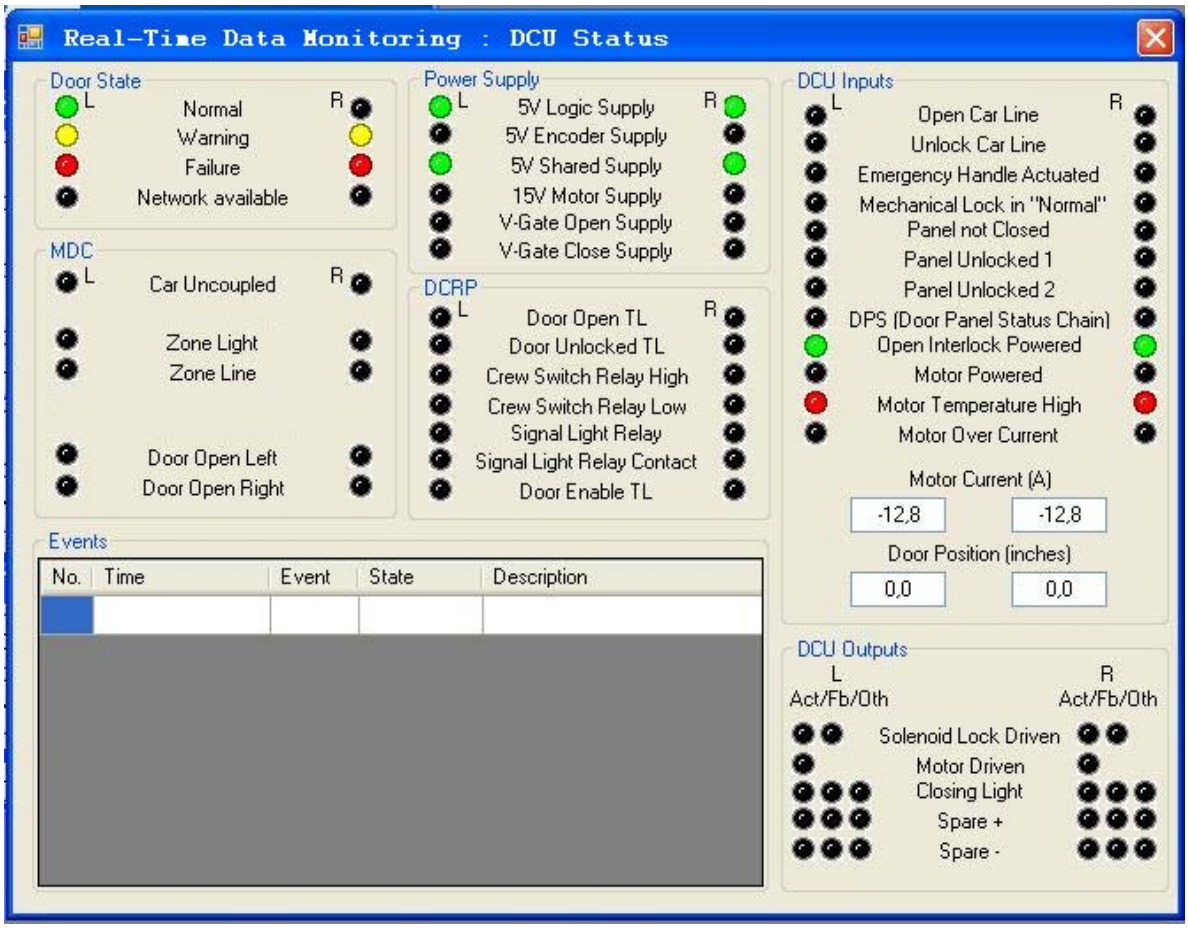


Figure 3-16 Real-Time Data Monitoring GUI

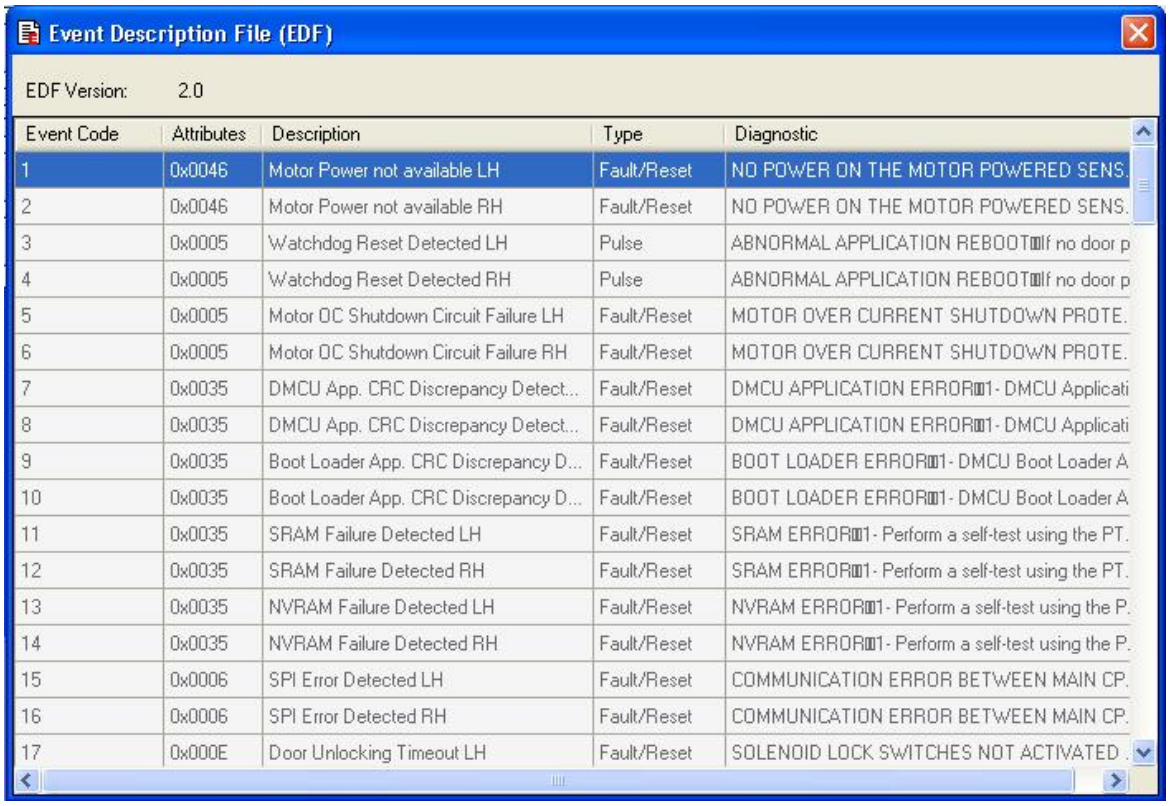
The Data Monitoring GUI displays the real time monitoring of the status of certain DCU signals. A selected set of signals contained in the PTA OPDF are displayed in this GUI. The Data Monitoring GUI will also display Events in real time as they occur.

None of the textboxes displayed are modifiable, they are used for display only.

When opened, the Data Monitoring GUI will constantly poll the DCU for the status of the signals and the event synchronization. When closed, it will stop polling.

The signal status in this GUI is the same as the Browser GUI, but in a less detailed view.

3.6.11 Event Description File View



Event Code	Attributes	Description	Type	Diagnostic
1	0x0046	Motor Power not available LH	Fault/Reset	NO POWER ON THE MOTOR POWERED SENS.
2	0x0046	Motor Power not available RH	Fault/Reset	NO POWER ON THE MOTOR POWERED SENS.
3	0x0005	Watchdog Reset Detected LH	Pulse	ABNORMAL APPLICATION REBOOT If no door p
4	0x0005	Watchdog Reset Detected RH	Pulse	ABNORMAL APPLICATION REBOOT If no door p
5	0x0005	Motor OC Shutdown Circuit Failure LH	Fault/Reset	MOTOR OVER CURRENT SHUTDOWN PROTE.
6	0x0005	Motor OC Shutdown Circuit Failure RH	Fault/Reset	MOTOR OVER CURRENT SHUTDOWN PROTE.
7	0x0035	DMCU App. CRC Discrepancy Detect...	Fault/Reset	DMCU APPLICATION ERROR 1- DMCU Applicati
8	0x0035	DMCU App. CRC Discrepancy Detect...	Fault/Reset	DMCU APPLICATION ERROR 1- DMCU Applicati
9	0x0035	Boot Loader App. CRC Discrepancy D...	Fault/Reset	BOOT LOADER ERROR 1- DMCU Boot Loader A
10	0x0035	Boot Loader App. CRC Discrepancy D...	Fault/Reset	BOOT LOADER ERROR 1- DMCU Boot Loader A
11	0x0035	SRAM Failure Detected LH	Fault/Reset	SRAM ERROR 1- Perform a self-test using the PT.
12	0x0035	SRAM Failure Detected RH	Fault/Reset	SRAM ERROR 1- Perform a self-test using the PT.
13	0x0035	NVRAM Failure Detected LH	Fault/Reset	NVRAM ERROR 1- Perform a self-test using the P.
14	0x0035	NVRAM Failure Detected RH	Fault/Reset	NVRAM ERROR 1- Perform a self-test using the P.
15	0x0006	SPI Error Detected LH	Fault/Reset	COMMUNICATION ERROR BETWEEN MAIN CP.
16	0x0006	SPI Error Detected RH	Fault/Reset	COMMUNICATION ERROR BETWEEN MAIN CP.
17	0x000E	Door Unlocking Timeout LH	Fault/Reset	SOLENOID LOCK SWITCHES NOT ACTIVATED .

Figure 3-17 EDF View GUI

The Event Description File View displays the content of the EDF loaded in the PTA. It is not necessarily the EDF file used by the DCU Application.

The GUI displays the following information:

- EDF version: Version of the EDF. At the top of the GUI.
- Event Code: Code of the Event
- Attributes: A numerical value of the Attributes. When parsed into binary, each bit is a Boolean value corresponding to an attribute.
- Description: A short description of the event.
- Type: Type of the event, either “Fault/Reset” or “Pulse”
- Diagnostic: A longer description of the event, when it occurs and why.

Double-clicking any event will open the Event Attributes GUI.

3.6.12 Event Attributes

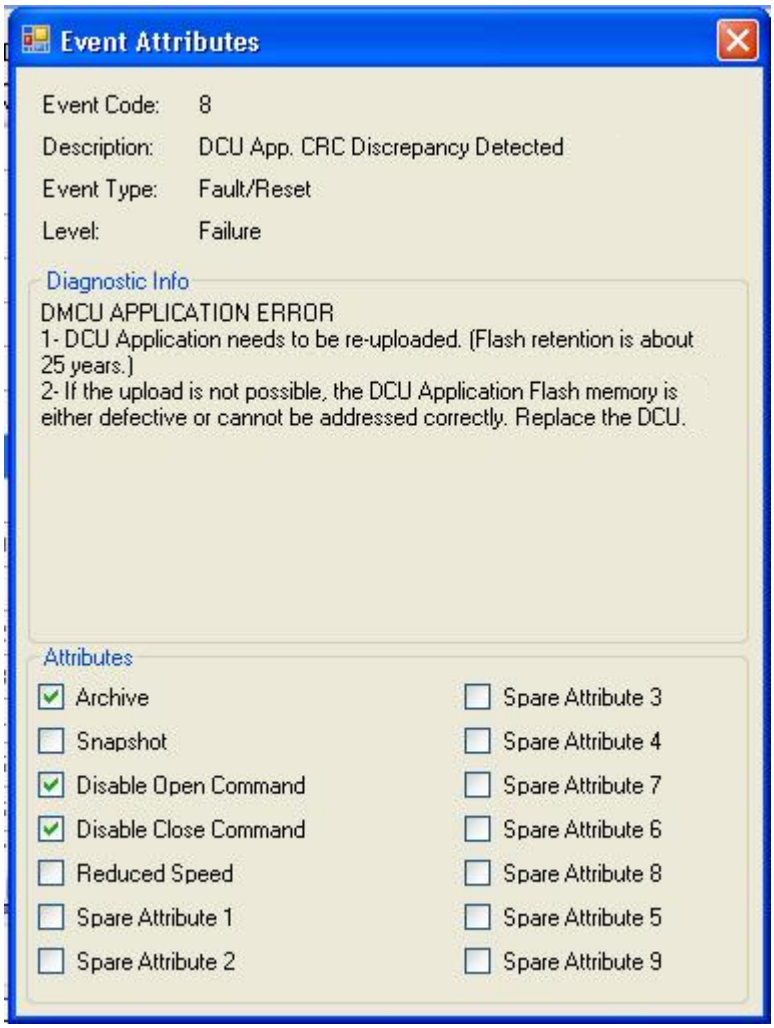


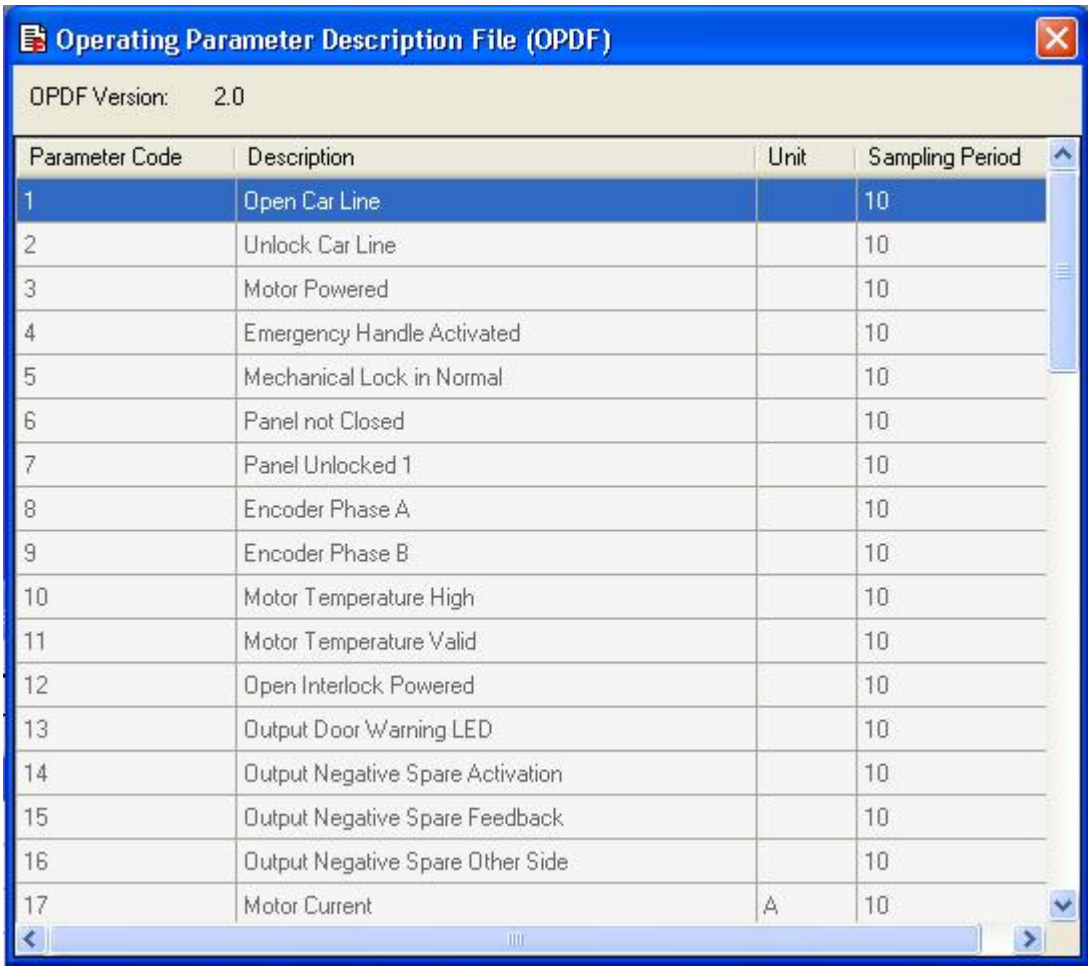
Figure 3-18 Event Attribute GUI

The Event Attributes GUI displays the event details. The details and the Attributes cannot be modified.

The GUI displays the following information:

- Event Code: Code of the Event
- Description: A short description of the event.
- Type: Type of the event, either “Fault/Reset” or “Pulse”
- Level: Can be “Event”, “Exception”, “Failure” or “Other”.
- Diagnostic: A longer description of the event, when it occurs and why.
- Attributes: A list of all possible attributes. Those applying to the event are checked. These are not modifiable through the PTA.

3.6.13 Operating Parameters Description File View



Parameter Code	Description	Unit	Sampling Period
1	Open Car Line		10
2	Unlock Car Line		10
3	Motor Powered		10
4	Emergency Handle Activated		10
5	Mechanical Lock in Normal		10
6	Panel not Closed		10
7	Panel Unlocked 1		10
8	Encoder Phase A		10
9	Encoder Phase B		10
10	Motor Temperature High		10
11	Motor Temperature Valid		10
12	Open Interlock Powered		10
13	Output Door Warning LED		10
14	Output Negative Spare Activation		10
15	Output Negative Spare Feedback		10
16	Output Negative Spare Other Side		10
17	Motor Current	A	10

Figure 3-19 OPDF View GUI

The Operating Parameter Description File View GUI displays the Operating Parameters details. It is not necessarily the OPDF used by the DCU Application.

The GUI displays the following information:

- OPDF Version: Version of the OPDF. At the top of the GUI.
- Parameter Code: Code of the parameter
- Description: A short description of the parameter.
- Unit: The unit in which the parameter value is displayed
- Sampling Period: The rate at which the parameter's value is checked by the DCU Application.

3.6.14 Graph View

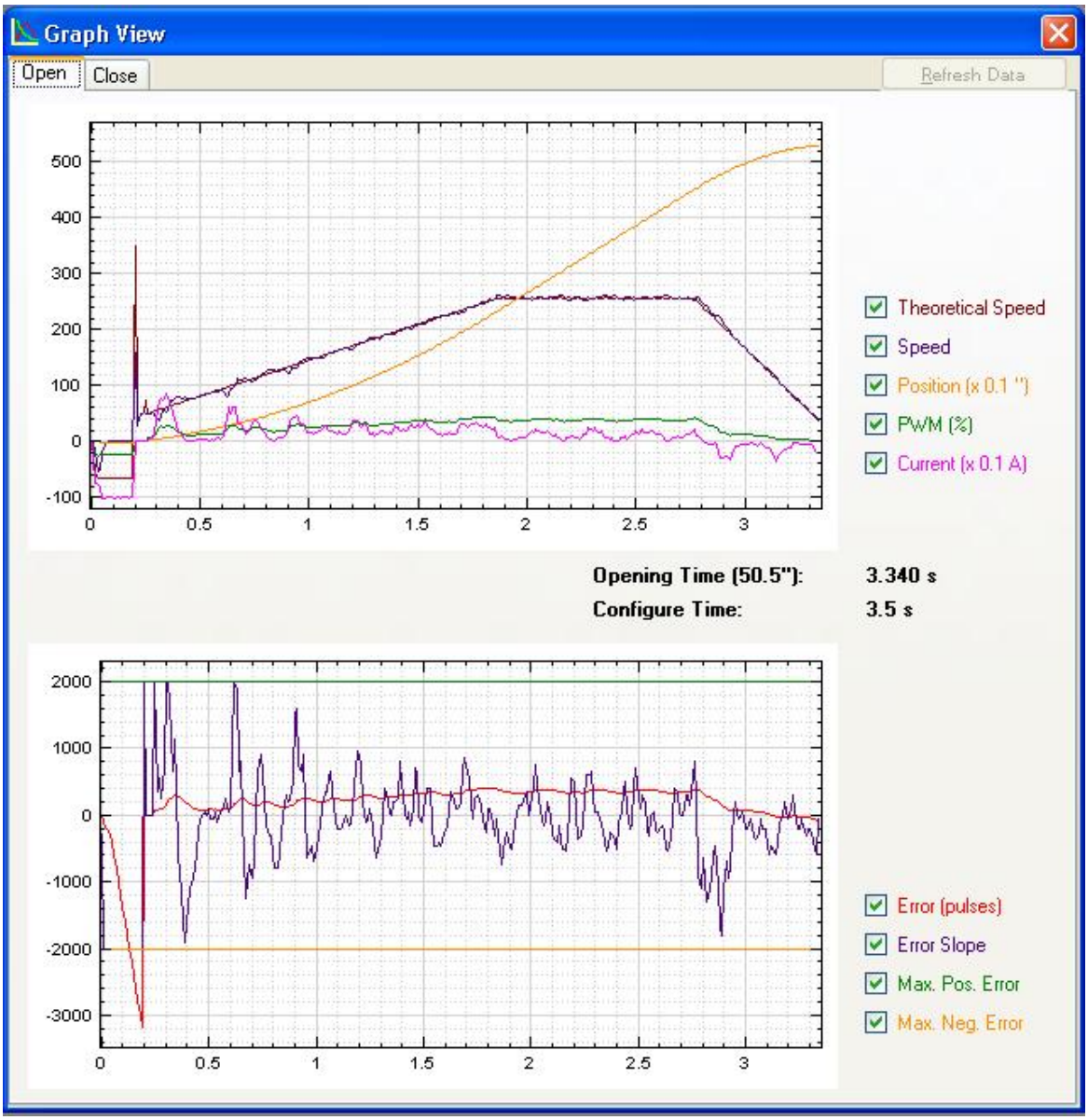


Figure 3-20 Graph View GUI

The Graph View GUI displays the door opening graphic and door closing graphic of the door.

The DCU must have completed a door open and door close cycle so that the file will contain data. If the PTA downloads an empty graph, the graph will appear blank.

The checkboxes to the right allow the user to control which line plot is displayed on the graph. If none are selected, the graph will appear blank. When checking and unchecking items, the graph will rescale itself to optimize display.

The top graph displays:

- Theoretical Speed
- Speed
- Position
- PWM
- Current

The bottom graph displays:

- Error pulses
- Error Slope
- Max. Pos. Error
- Max Neg. Error

The middle section displays the Opening and Closing time. The GUI displays the configured Opening and Closing time as well as the actual calculated values for comparison.

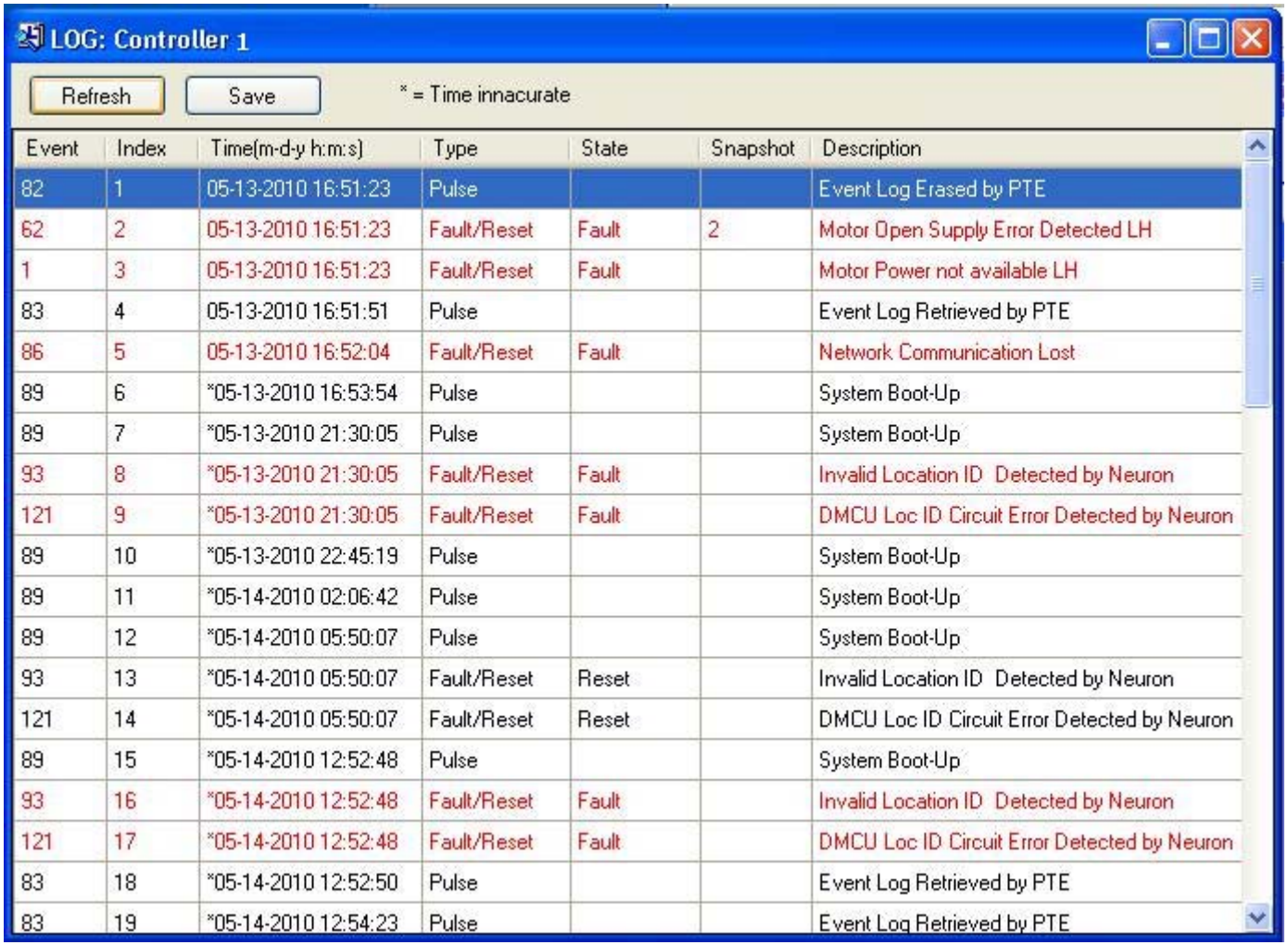
3.6.15 Event Log Erasure

Figure 3-21 Log Erasure GUI

The Log Erasure GUI pops up when selecting the Log Erasure option from the PTA. Before proceeding, the user must check the checkbox then press Erase Log. As an additional precaution, another popup confirmation message will ask the user if he is certain.

Pressing Cancel at any time during this process will close the GUI or popup and cancel the operation.

3.6.16 Event Log Table



Event	Index	Time(m-d-y h:m:s)	Type	State	Snapshot	Description
82	1	05-13-2010 16:51:23	Pulse			Event Log Erased by PTE
62	2	05-13-2010 16:51:23	Fault/Reset	Fault	2	Motor Open Supply Error Detected LH
1	3	05-13-2010 16:51:23	Fault/Reset	Fault		Motor Power not available LH
83	4	05-13-2010 16:51:51	Pulse			Event Log Retrieved by PTE
86	5	05-13-2010 16:52:04	Fault/Reset	Fault		Network Communication Lost
89	6	*05-13-2010 16:53:54	Pulse			System Boot-Up
89	7	*05-13-2010 21:30:05	Pulse			System Boot-Up
93	8	*05-13-2010 21:30:05	Fault/Reset	Fault		Invalid Location ID Detected by Neuron
121	9	*05-13-2010 21:30:05	Fault/Reset	Fault		DMCU Loc ID Circuit Error Detected by Neuron
89	10	*05-13-2010 22:45:19	Pulse			System Boot-Up
89	11	*05-14-2010 02:06:42	Pulse			System Boot-Up
89	12	*05-14-2010 05:50:07	Pulse			System Boot-Up
93	13	*05-14-2010 05:50:07	Fault/Reset	Reset		Invalid Location ID Detected by Neuron
121	14	*05-14-2010 05:50:07	Fault/Reset	Reset		DMCU Loc ID Circuit Error Detected by Neuron
89	15	*05-14-2010 12:52:48	Pulse			System Boot-Up
93	16	*05-14-2010 12:52:48	Fault/Reset	Fault		Invalid Location ID Detected by Neuron
121	17	*05-14-2010 12:52:48	Fault/Reset	Fault		DMCU Loc ID Circuit Error Detected by Neuron
83	18	*05-14-2010 12:52:50	Pulse			Event Log Retrieved by PTE
83	19	*05-14-2010 12:54:23	Pulse			Event Log Retrieved by PTE

Figure 3-22 Event Log Table GUI

The Log Table will display the event log downloaded from the DCU Application. The log displayed is not refreshed automatically.

The user can do the following actions:

- Clicking Refresh button allows the user to download the event log.
- Clicking Save button allows the user to save the event log in a CSV format.
- Clicking Export button (Not visible on Figure 3-22) allows the user to load a CSV format event log file.
- Clicking the column headers allows the user to sort the table.
- Double-clicking an event log entry will open the generic Event Attributes GUI for that event.
- Right-clicking an event will open the marks menu that allows you to do three things: Mark the event by highlighting it, Mark all events of the same type, Clear all Marks. Marking an event is only for readability.

The table has the following columns:

- Event: the identification number of event
- Index: Sequence number of event
- Date & Time : The time and date the event occurred
- Type: Event type
- State: Event state
- Snapshot: Related snapshot file number, if any.
- Event Code: Event code number (Not visible on Figure 3-22)
- Description: Event description

3.6.17 Memory Dump

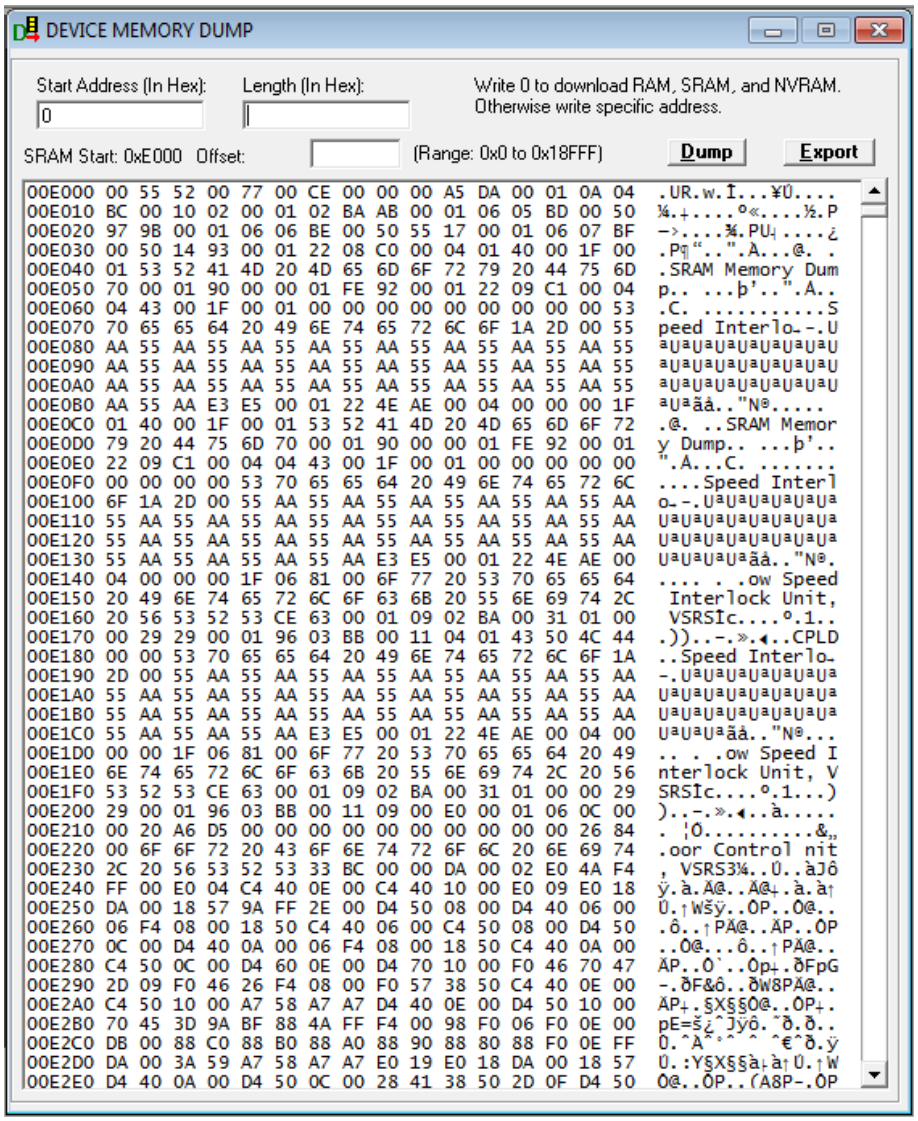


Figure 3-23 Memory Dump GUI

The Memory Dump window is used to download the DCU RAM contents (internal SRAM and NVRAM starts adresses are project's specific) and to display the contents in Hex and ASCII. During the download process, a progress bar displays the status of the data transfer.

3.6.18 Snapshot Erasure

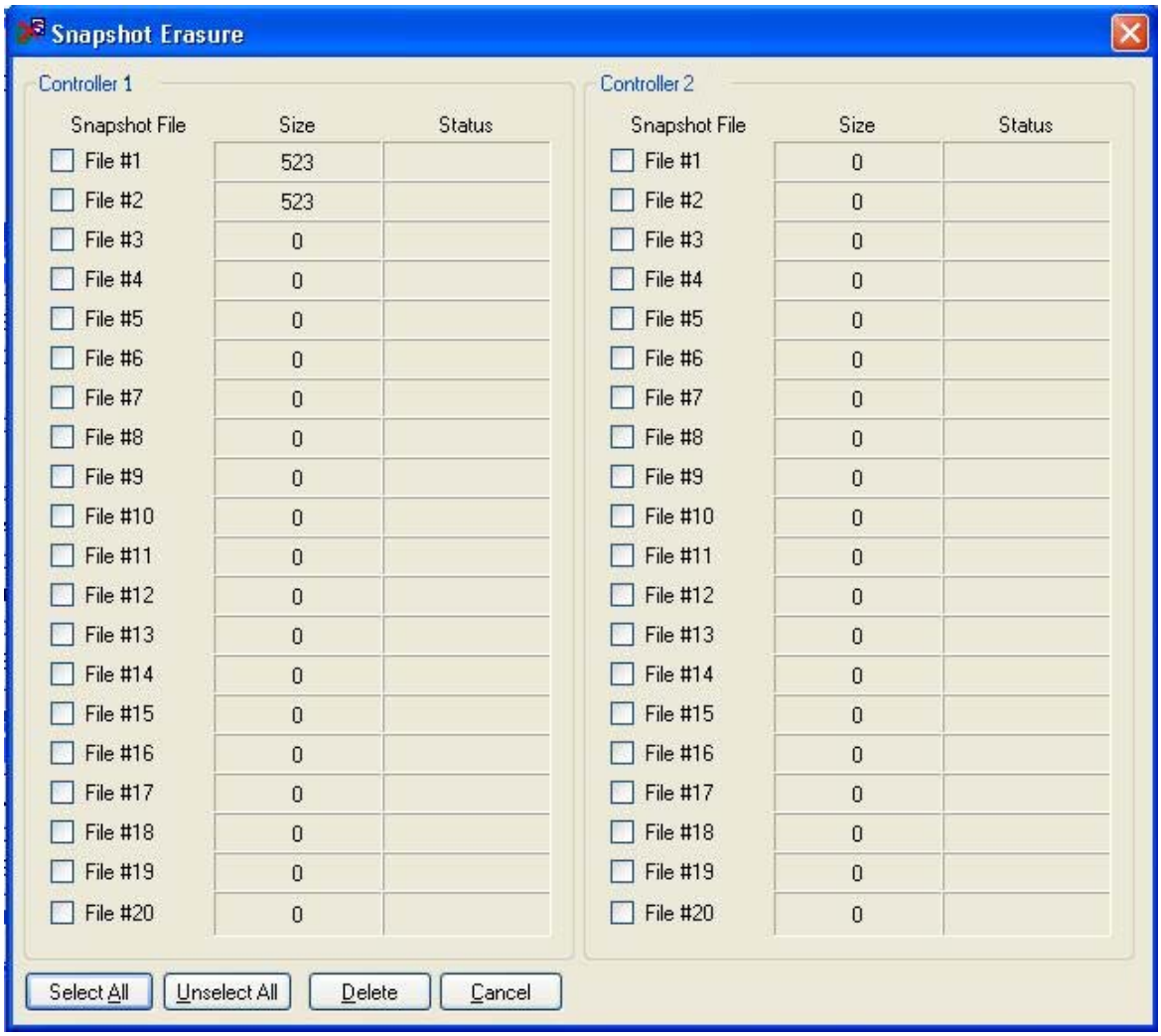


Figure 3-24 Snapshot Erasure GUI

The Snapshot Erasure GUI allows the user to delete specified snapshot files. The Size field represents the number of bytes the snapshot file uses on the DCU. A size of 0 means there is no snapshot file stored in that slot.

The user must check the snapshot files to be deleted using the checkboxes on the left. The user must then press the Delete button to delete the files. A confirmation window will appear. Once the files are deleted, the status field will indicate it.

The Select All and Unselect All buttons check the boxes but the files will not be deleted until the user presses Delete.
Pressing Cancel, even with boxes checked, will cancel the operation and no Delete request will be sent out.

3.6.19 Snapshot Table

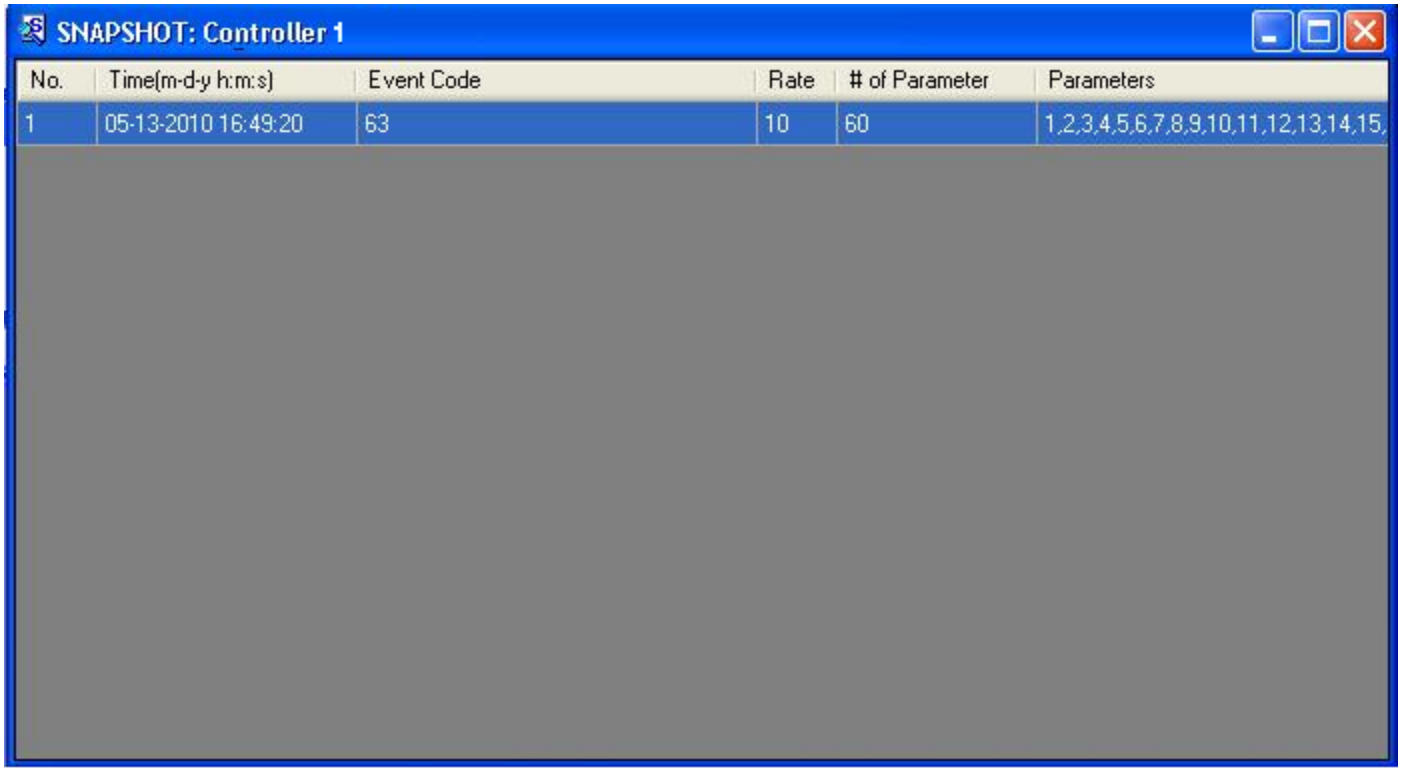


Figure 3-25 Snapshot Table GUI

The Snapshot Table is used to list all snapshot files of the DCU. In this table, the following snapshot file header information is presented:

- No: snapshot number
- Time: date and time stamp of the snapshot
- Event Code: reference event code and description
- Rate: sampling rate
- # of Parameter: The number of parameters in the file
- Parameters: parameters list

Clicking the headers of the columns will sort the table.
Double-clicking a snapshot will open the Snapshot View GUI.

3.6.20 Snapshot View

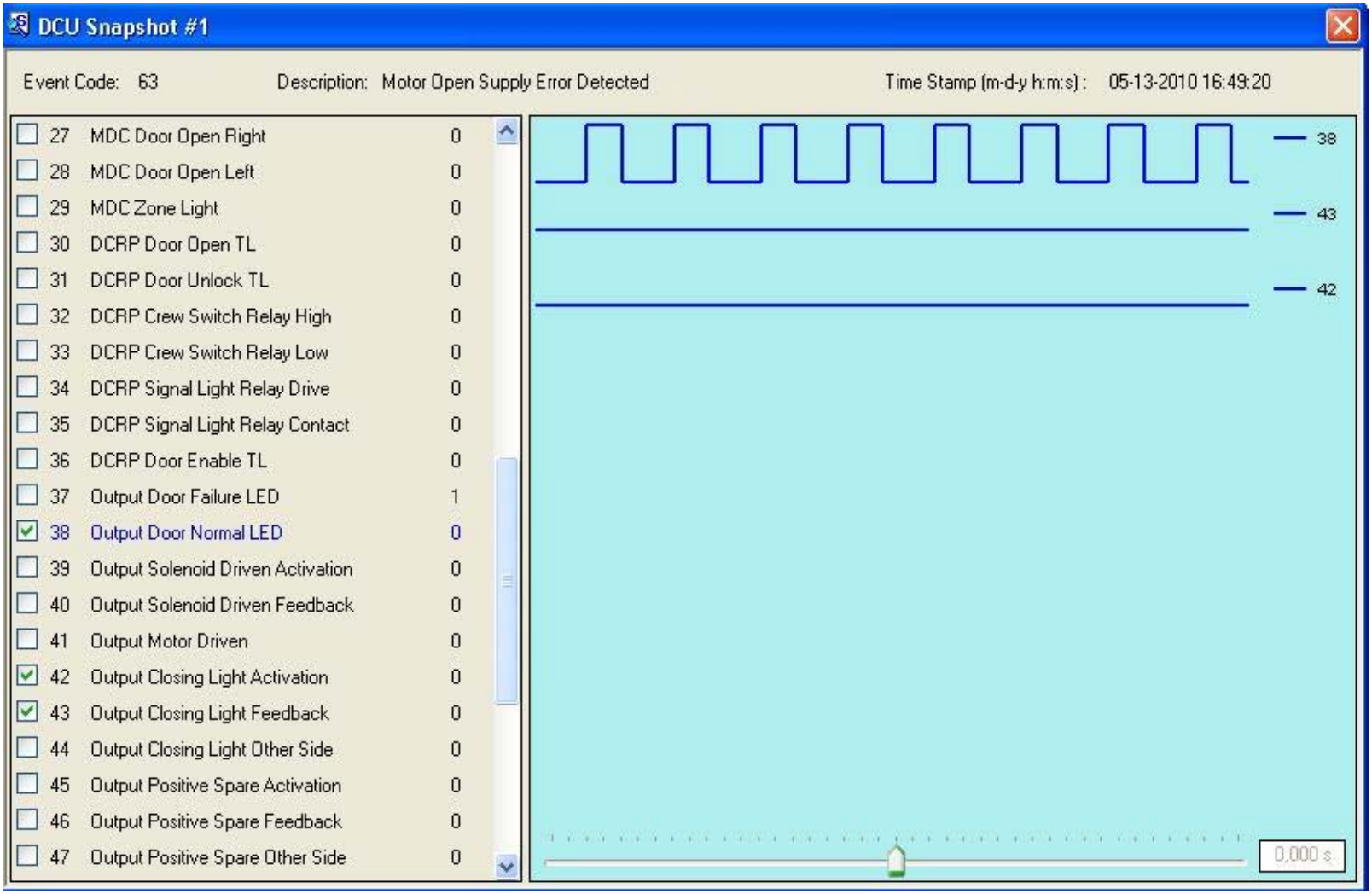


Figure 3-26 Snapshot View GUI

The Snapshot View allows the user to explore the snapshot contents. The GUI contains three sections:

- The header
- The parameters list
- The graphic view

The header presents the following information:

- Event Code: Code of the reference event
- Description: Description of the reference event
- Time Stamp: Date and time the snapshot was taken.

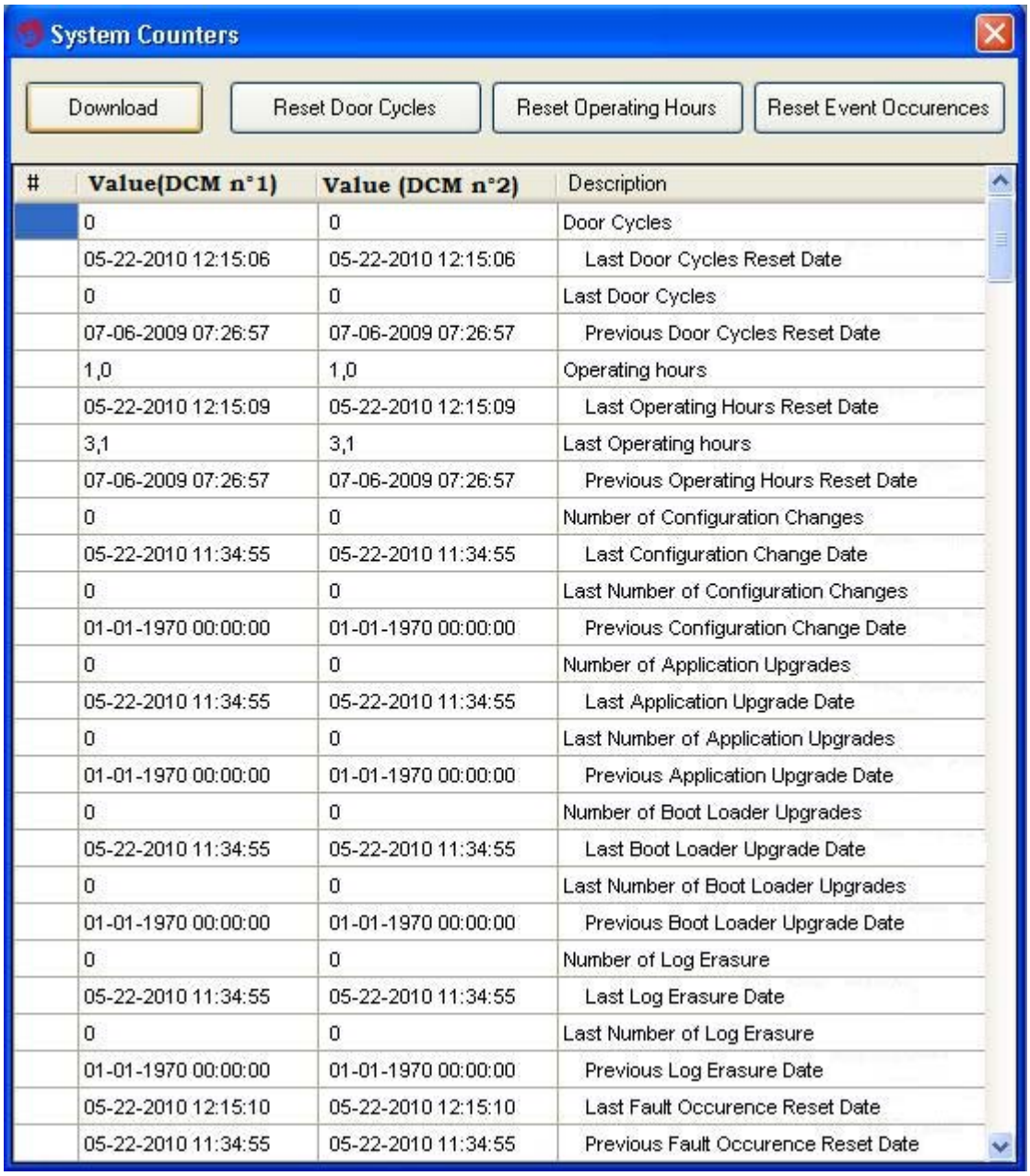
The parameters list presents the following information and controls:

- Parameter ID: ID of the operating parameter as seen in the OPDF
- Description: Description of the parameter as seen in the OPDF
- Value: Value of the parameter corresponding to the time indicated by the track bar, in the graphic section.
- Checkbox: Checking this box displays the parameter's plot in the graphic section.
- Highlighting: Parameters in blue mean that the signal value changes through the snapshot time period. Black means that the signal remained the same through the snapshot duration.

The parameters list presents the following information and controls:

- Graphic: For every checked parameter in the parameter list section, a graphic will be displayed. The graphic shows the fluctuation of the signal value through the time period of the snapshot.
- Track bar: An indicator in time shown at the bottom of the graphic section. Changing the track bar modifies the displayed signal value in the parameters section and the time box to the right of the track bar.
- Time box: Time currently indicated by the position of the track bar. The middle of the trackbar is 0, when the actual snapshot was taken. The time will not increment in regular intervals. Increments are calculated by using a delta.

3.6.21 System Counters



#	Value(DCM n°1)	Value (DCM n°2)	Description
	0	0	Door Cycles
	05-22-2010 12:15:06	05-22-2010 12:15:06	Last Door Cycles Reset Date
	0	0	Last Door Cycles
	07-06-2009 07:26:57	07-06-2009 07:26:57	Previous Door Cycles Reset Date
	1,0	1,0	Operating hours
	05-22-2010 12:15:09	05-22-2010 12:15:09	Last Operating Hours Reset Date
	3,1	3,1	Last Operating hours
	07-06-2009 07:26:57	07-06-2009 07:26:57	Previous Operating Hours Reset Date
	0	0	Number of Configuration Changes
	05-22-2010 11:34:55	05-22-2010 11:34:55	Last Configuration Change Date
	0	0	Last Number of Configuration Changes
	01-01-1970 00:00:00	01-01-1970 00:00:00	Previous Configuration Change Date
	0	0	Number of Application Upgrades
	05-22-2010 11:34:55	05-22-2010 11:34:55	Last Application Upgrade Date
	0	0	Last Number of Application Upgrades
	01-01-1970 00:00:00	01-01-1970 00:00:00	Previous Application Upgrade Date
	0	0	Number of Boot Loader Upgrades
	05-22-2010 11:34:55	05-22-2010 11:34:55	Last Boot Loader Upgrade Date
	0	0	Last Number of Boot Loader Upgrades
	01-01-1970 00:00:00	01-01-1970 00:00:00	Previous Boot Loader Upgrade Date
	0	0	Number of Log Erasure
	05-22-2010 11:34:55	05-22-2010 11:34:55	Last Log Erasure Date
	0	0	Last Number of Log Erasure
	01-01-1970 00:00:00	01-01-1970 00:00:00	Previous Log Erasure Date
	05-22-2010 12:15:10	05-22-2010 12:15:10	Last Fault Occurence Reset Date
	05-22-2010 11:34:55	05-22-2010 11:34:55	Previous Fault Occurence Reset Date

Figure 3-27 System Counters GUI

The System Counters GUI displays the downloaded system counters from the DCU Application.

The GUI has the following controls:

- Download: Allows the user to download the latest system counters
- Reset Door Cycles: Resets the door cycles counter.
- Reset Operating Hours: Resets the operating hours counter
- Reset Event Occurrences: Resets all the event occurrences counters (for each event)

The table has the following columns:

- #: Event Code. Only used for event counters.

- Value: Value of the counter named in description
- Description: Description of the counter or the value column.

The GUI table displays the following counters (counters have been grouped by category):

- Door Cycles: Number of door open/door close cycles.
- Operating Hours: Number of hours the DCU has been online.
- Number of Configuration Changes: Number of times the configuration has changed (through DCU Config GUI).
- Number of Application Upgrades: Number of times the DCU Application has been upgraded.
- Number of Boot Loader Upgrades: Number of times the Boot Loader has been upgraded.
- Number of Log Erasures: Number of times the event log has been erased.
- Fault Occurrences: One counter per event listed in the EDF. Number of times each event has occurred.

3.6.22 Self Test

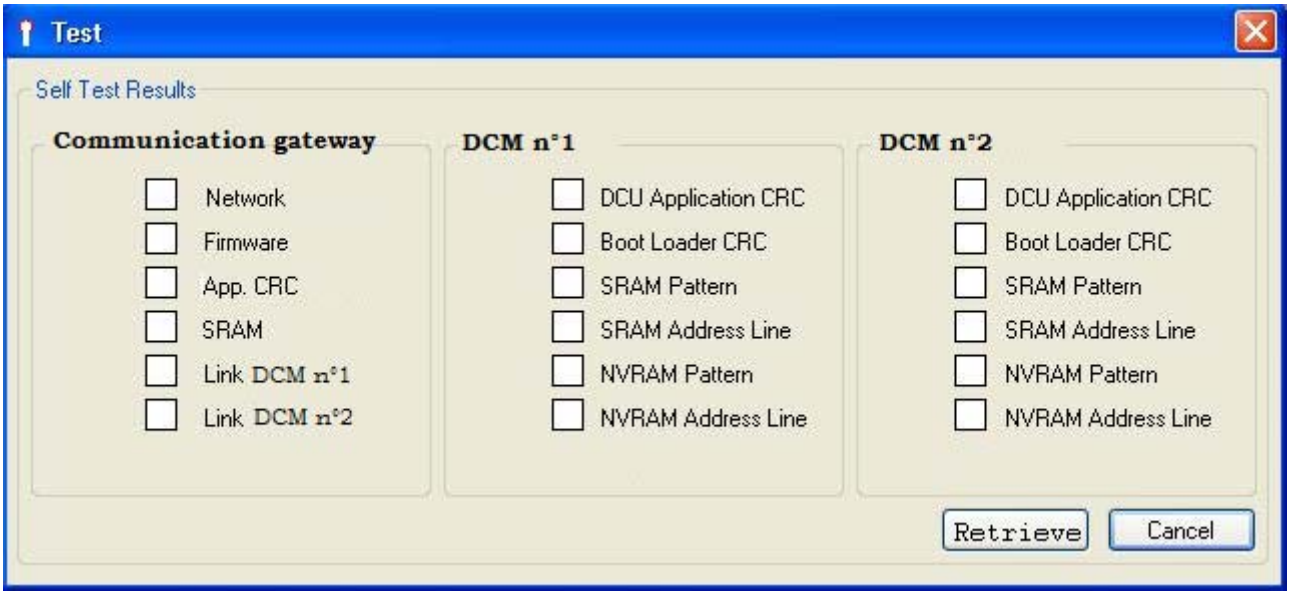


Figure 3-28 Self Test GUI

The Self-Test GUI displays current self-test results. The devices that are tested are listed in this window. If the device passes the self-test, the word “PASS” and the symbol ✓ appears in the front of the device name; otherwise, the word “FAIL” and the symbol ✗ appears.

The Self-Test GUI has the following controls:

- Redo: Redo the Self-Test.
- Cancel: Exit the Self-Test window.

3.6.23 Time

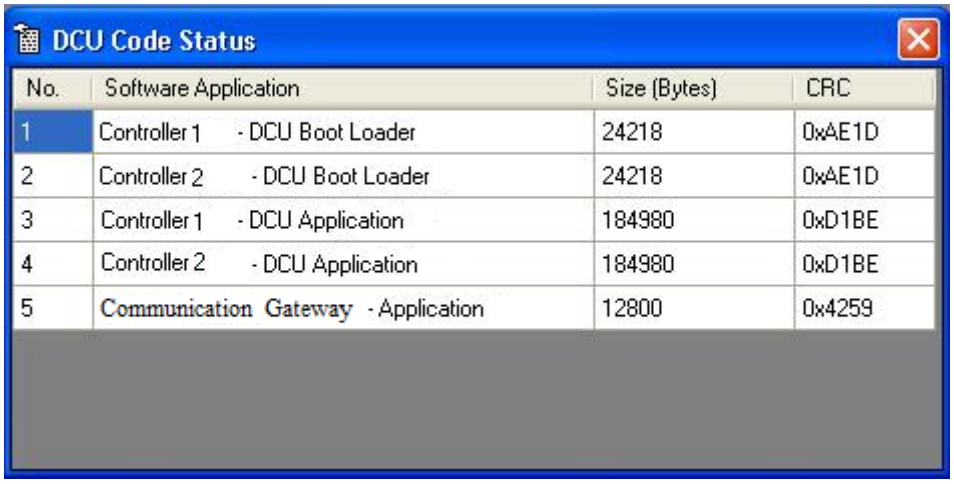
Figure 3-29 Time GUI

The Time GUI displays and sets the time in the DCU. The displayed time is not updated automatically in real-time; the user must manually retrieve it.

The GUI has the following controls:

- Update DCU Time: Sets the DCU date and time to the same as the Operating system of the computer the PTA is currently running on.
- Retrieve DCU Time: Retrieves and displays the DCU's time.
- Close: Closes the GUI.

3.6.24 DCU Code Status



DCU Code Status

No.	Software Application	Size (Bytes)	CRC
1	Controller 1 - DCU Boot Loader	24218	0xAE1D
2	Controller 2 - DCU Boot Loader	24218	0xAE1D
3	Controller 1 - DCU Application	184980	0xD1BE
4	Controller 2 - DCU Application	184980	0xD1BE
5	Communication Gateway - Application	12800	0x4259

Figure 3-30 DCU Code Status GUI

The Code Status GUI displays the applications loaded into the DCU along with details such as the size of the application file and the CRC of the application.

3.6.25 DCU Configuration

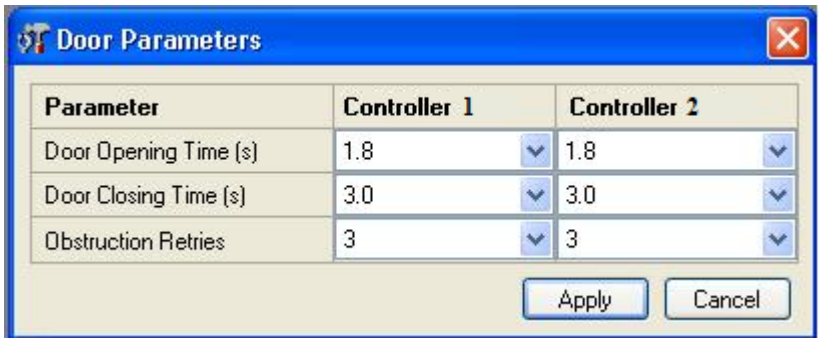


Figure 3-31 DCU Config GUI

The DCU Config GUI displays the current configuration in the DCU Application and allows the user to modify them.

All the configurabe parameters listed hereafter are described in the Appendix A and their values are defined in the SACS project's specific document.

The parameters available through this GUI are:

- SACS.Door.DefaultOpeningDelay (Door Opening Time (s) on Figure 3-31)
- SACS.Door.DefaultOpeningTime (Door Closing Time (s) on Figure 3-31)
- SACS.Door.OpeningTime.Range (Not showed on Figure 3-31)
- SACS.Door.DefaultClosingDelay (Not showed on Figure 3-31)
- SACS.Door.DefaultClosingTime (Not showed on Figure 3-31)
- SACS.Door.ClosingTime.Range (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Outside.ClosingDelay (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Inside.ClosingDelay (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Outside.ClosingTime (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Inside.ClosingTime (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Outside.OpeningDelay (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Inside.OpeningDelay (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Outside.OpeningTime (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.Inside.OpeningTime (Not showed on Figure 3-31)
- SACS.Door.CrewSwitch.LinkTiming (Not showed on Figure 3-31)
- SACS.Door.Obstruction.Distance (Not showed on Figure 3-31)
- SACS.Door.Obstruction.Count (Obstruction Retries on Figure 3-31)
- SACS.Door.Obstruction.EndBehavior (Not showed on Figure 3-31)
- SACS.Door.Obstruction.Range (Not showed on Figure 3-31)
- SACS.Door.AudibleAlarmActivation (Not showed on Figure 3-31)
- SACS.Door.VisualAlarmActivation (Not showed on Figure 3-31)
- SACS.Door.SecondaryObstruction.Position (Not showed on Figure 3-31)
- SACS.Door.SecondaryObstruction.Activation (Not showed on Figure 3-31)
- SACS.Door.SecondaryObstruction.Delay (Not showed on Figure 3-31)
- SACS.Door.EmergencyHandle.ClosePower (Not showed on Figure 3-31)
- SACS.Door.Snapshot.Period (Not showed on Figure 3-31)
- SACS.Door.Snapshot.Capacity (Not showed on Figure 3-31)
- SACS.Door.Snapshot.NbOfFiles (Not showed on Figure 3-31)

3.6.26 Event Log Erasure

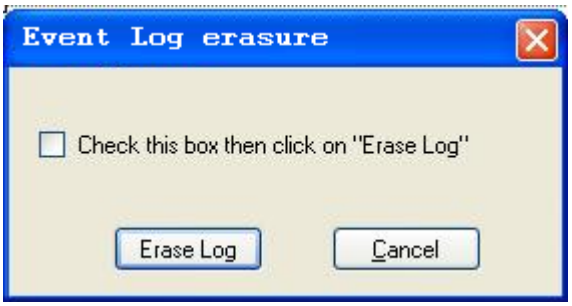


Figure 3-32 MDS Log Erasure GUI

This GUI allows the user to request the erasure of the Event Log file stored on the DCU.

The MDS Log Erasure GUI pops up when selecting the Event Log Erasure option from the PTA. Before proceeding, the user must check the checkbox then press Erase Log. As an additional precaution, another popup confirmation message will ask the user if he is certain.

Pressing Cancel at any time during this process will close the GUI or popup and cancel the operation.

3.6.27 About

Figure 3-33 About GUI

The About GUI displays the PTA version information and the copyright information.

3.7 Detailed Design

3.7.1 Main Module

The DCU Application applies the Multiple-Document Interface (MDI). The DCU Main organizes and incorporates all the application function modules. The DCU Application applies Windows event-driven program technology: when an event occurs, a message is sent to the DCU Application and the DCU Application has related process to handle it. For the Portable Test Application software, the following issues may trigger an event:

- Keyboard command
- Mouse click
- Communication Port
- Disk Error
- Printer

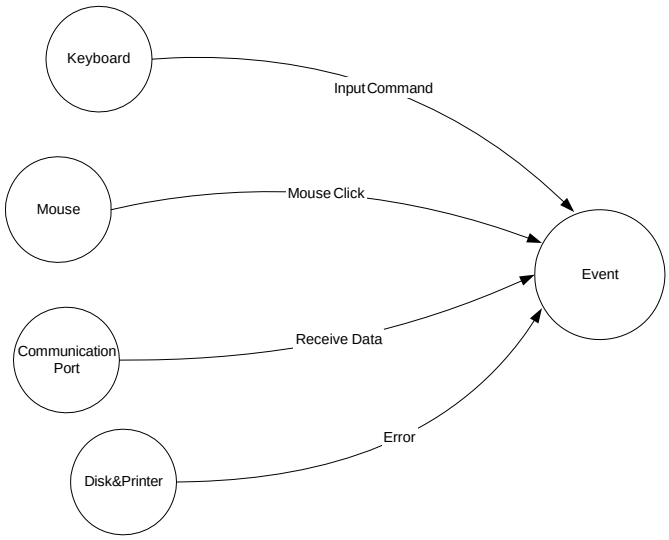


Figure 3-34 DCU Application Task Schedule Data Flow Diagram

3.7.1.1 Interfaces

The Main module interfaces with every other module through its MainGUI class. The MainGUI is a MDI (Multiple Document Interface) parent, meaning it “contains” all other GUIs and is their creator.

3.7.1.2 Functionalities

3.7.1.2.1 Startup of PTA

The Main Module includes two key classes: Main and the Main GUI. The Main is the startup class, which has the main() method to start the execution of the primary thread. The Main GUI is the first form created and the MDI parent to all other forms.

When starting the PTA, the following actions take place:

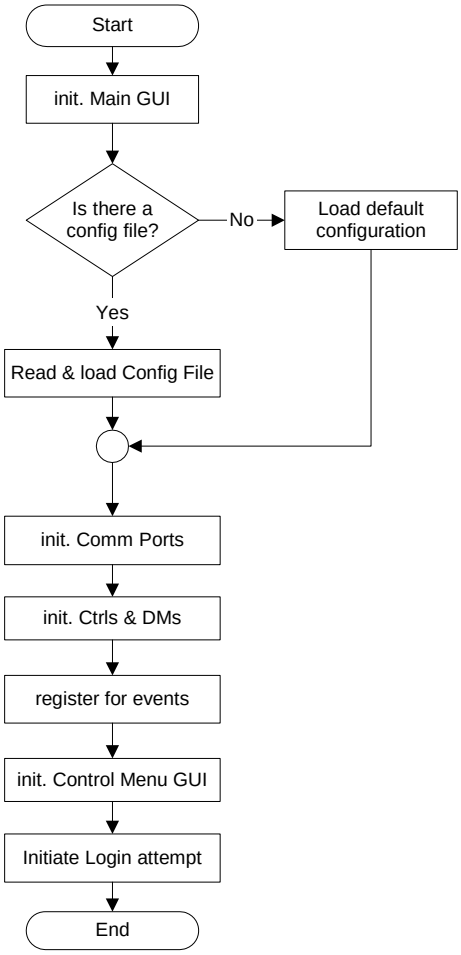


Figure 3-35 Flow Chart - DCU Application Startup

3.7.1.2.2 Shutdown of PTA

When the user requests to exit out of the PTA, some background actions take place. The Main GUI first sends a logout through the communicator. Afterwards, it asks the user if he wants to save the current PTA Project.

The PTA then closes all its communication ports and shuts down.

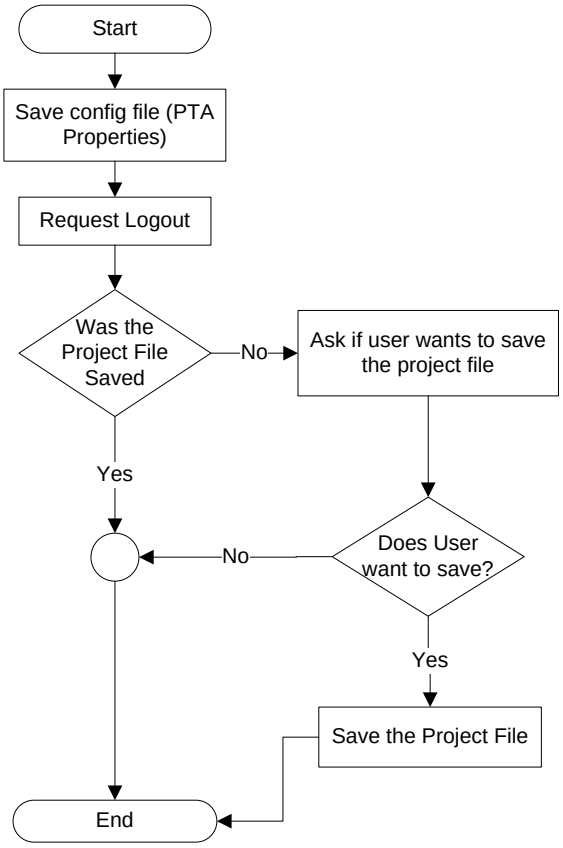


Figure 3-36 Flow Chart - PTA Shutdown

3.7.1.2.3 PTA Properties

This function allows the user to change the PTA properties related to the login. The options are changing the COM port and choosing a DCU target preference (DCU Application or Boot Loader).

The user preferences for PTA Properties are saved in a file with the following path and name:

```
X:\PTA\Data\Door\config.bin
```

Config.bin contains two elements: the login option and the communication device mode.

As shown in Figure 3-35, the PTA Properties are loaded from the file on startup. They are saved upon closing the PTA.

3.7.1.2.4 Password Security and Accessibility

Some functions are only accessible when logged in. Certain specific events will trigger the Main Module to check and adjust the accessibility of each of the menu items.

Figure 3-37 specifies the security requirements for each function:

Functions	Login to BootLoader	Login to DCU Application	Administration Security Level
SW Version Retrieve	x	x	
Log View			
Selft Test		x	
DataMonitoring		x	
Snapshot View			
Snapshot Download		x	
Log Download		x	
Graph View			
Graph Download		x	
Memory Dump Download	x	x	
Browser		x	
System Counters View			
System Counters Download		x	
Download All		x	
Log Erasure		x	x
SW Download	x		x
Boot Loader Upgrade			x
DCU Config Retrieve		x	
DCU Config Change		x	x
Snapshot Erasure		x	x
System Counters Reset		x	x
Time Retrieve		x	
Time Set		x	x
DCU Code Status	x	x	
EDF View			
OPDF View			
Update EDF & OPDF		x	

Figure 3-37 Function Accessibility

One thing is required to access the administrator functions of the PTA: a password.

3.7.1.2.4.1 Password

The password can be entered at any time while the PTA is running. The password is hardcoded into the application and is written in the PTA Software User Manual.

While no big effort is made to keep the password a secret, it is put into place to avoid accidental administrative function activations.

The flowchart below describes the simple sequence of actions taken to verify the password.

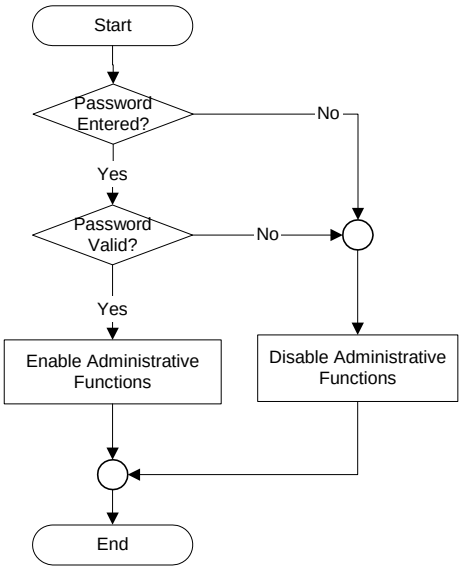


Figure 3-38 Flow Chart – Password Security

3.7.1.2.5 Login & Logout

The DCU Application requires the PTA to login before attempting any other operation. The login can be triggered manually by the user or automatically when starting up the PTA. There is one login per side. Being logged in with a target application means that all further communications will be directed towards this target application.

As shown in Figure 3-39, a successful login triggers a series of other communications. These automatic operations are triggered whether the login was automatic or requested by user. Note that the entire sequence is repeated once per targeted side.

Being logged in starts a periodical communication test which will test the link between the DCU and the PTA. The DCU requires periodical interaction with the PTA, else it will automatically log out the PTA.

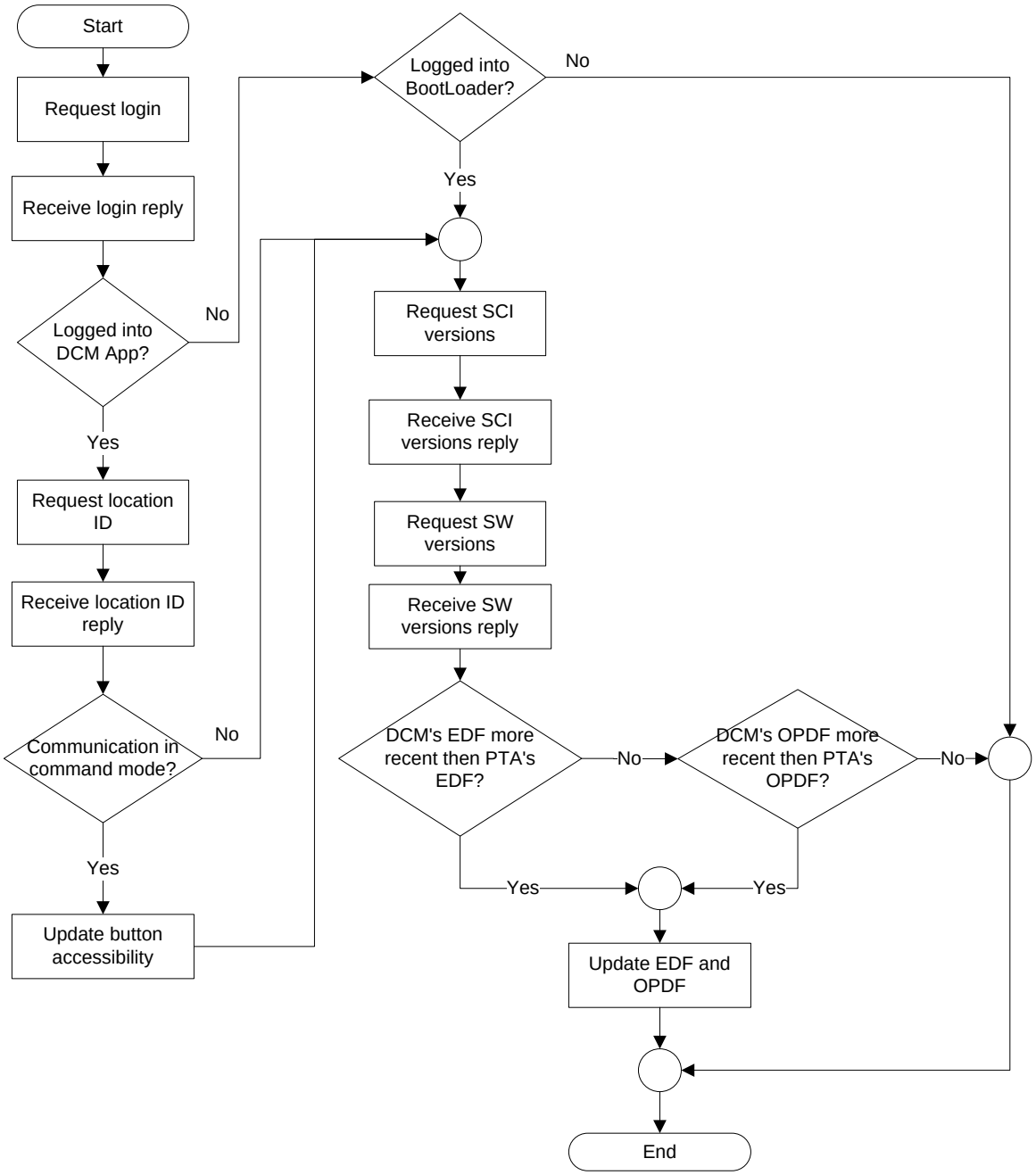


Figure 3-39 Flow Chart – Login

The logout sequence is simpler than the login. It can be triggered by the user or by exiting the PTA.

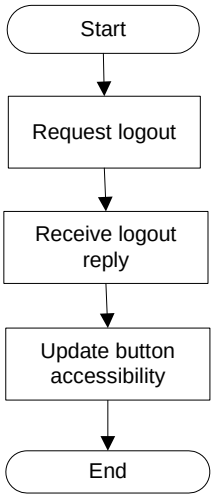


Figure 3-40 Flow Chart - Logout

3.7.1.2.6 Location ID

The Door Location in the Car Network is retrieved and displayed in the DCU Control Menu Location field. Refer to Vapor Stone Rail Systems ICD document Section #201111489 for more detail on location ID messages description.

3.7.1.2.7 Help

The help function simply opens a .pdf file located in the same folder as the PTA executable.

3.7.1.2.8 PTA Version & About

The PTA version is displayed in the Main GUI title bar, the status bar and the About GUI.

3.7.2 Communicator Module

The Communicator Module is composed of several entities shown in the figure below.

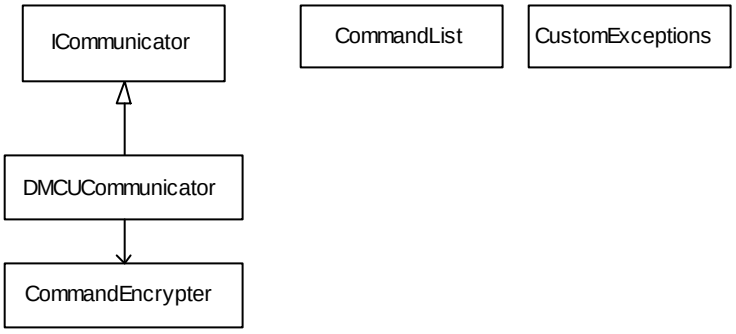


Figure 3-41 Communication Module Classes

A short description about the role of each class is as follows:

- CommandList holds the numerical value for all the commands sent through the Comm Port. The class is not linked to any other in the figures because it may be used by any class, in any module.
- CustomExceptions holds the definition for custom exceptions. The exceptions are related to communication and help with error handling on the GUI side. The class is not linked to any other in the figures because it may be used by any class, in any module.
- ICommunicator is a public interface that defines the skeleton of common inputs for all communicator classes. There is only one derivative class for.
- DCUCommunicator handles the serial port communication. It uses the CommandEncrypter which packets commands in the correct format, extracts meaningful bytes from received commands, checks the CRC, etc... The DCUCommunicator takes the commands from the other modules and goes through the sending sequence. It is also the entity that handles the sequence for receiving and sending out to the correct module.

3.7.2.1 Interfaces

As implied in the description above, the classes interfacing with other modules are ICommunicator, CommandList and CustomExceptions.

Because of the simple nature of CommandList and CustomExceptions, these will not be further explained as far as interfacing goes.

Classes derived from the type ICommunicator have the same inputs and outputs (also known as public functions or properties). The classes derived from ICommunicator are:

- NetworkCommunicator
- USBCommunicator
- DCUCommunicator

3.7.2.2 Functionalities

The following sections are of lower level (i.e. more detailed) then global functionalities of the system. These lower level processes are considered “functionalities” because of the dependencies with the communication protocol even if they are almost transparent to the user.

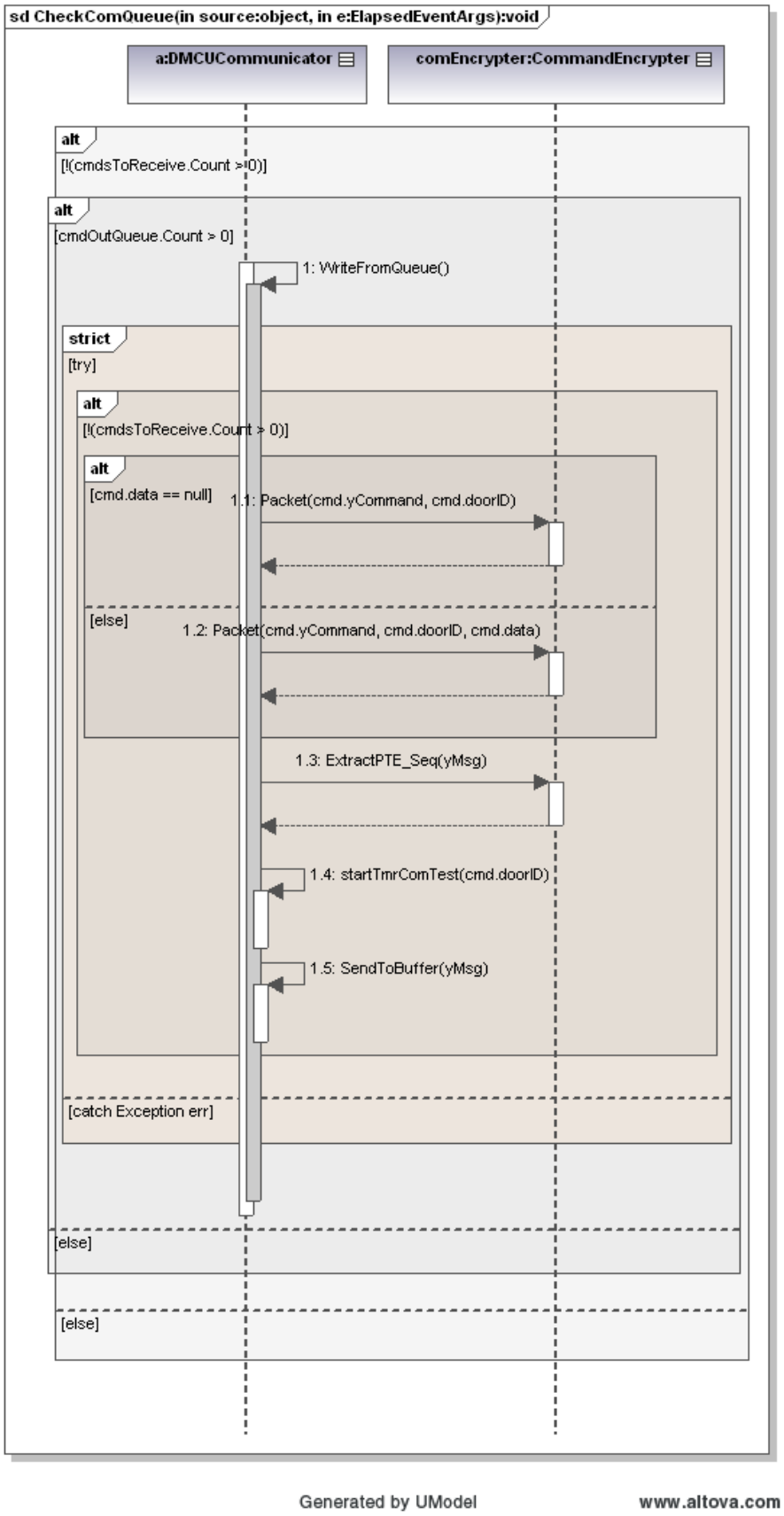
3.7.2.2.1 Command Transfer and Reception

The PTA is an event driven application. Because of this, the entire process of command transfer is actually a sequence of threads initiated independently. This means that the initial command request to the DCU is placed by a controller and the thread will do some operations before ending. The next thread may be initiated by a timer or a reception event and do the remainder of operations in order to complete the entire command transfer process.

CheckComQueue() is a function triggered periodically by a timer. It checks if any commands are waiting in the queue to be sent. These commands are placed in the queue by QueueCmdOut(). WriteFromQueue() prepares and sends commands to be sent (a.k.a. written) through the port.

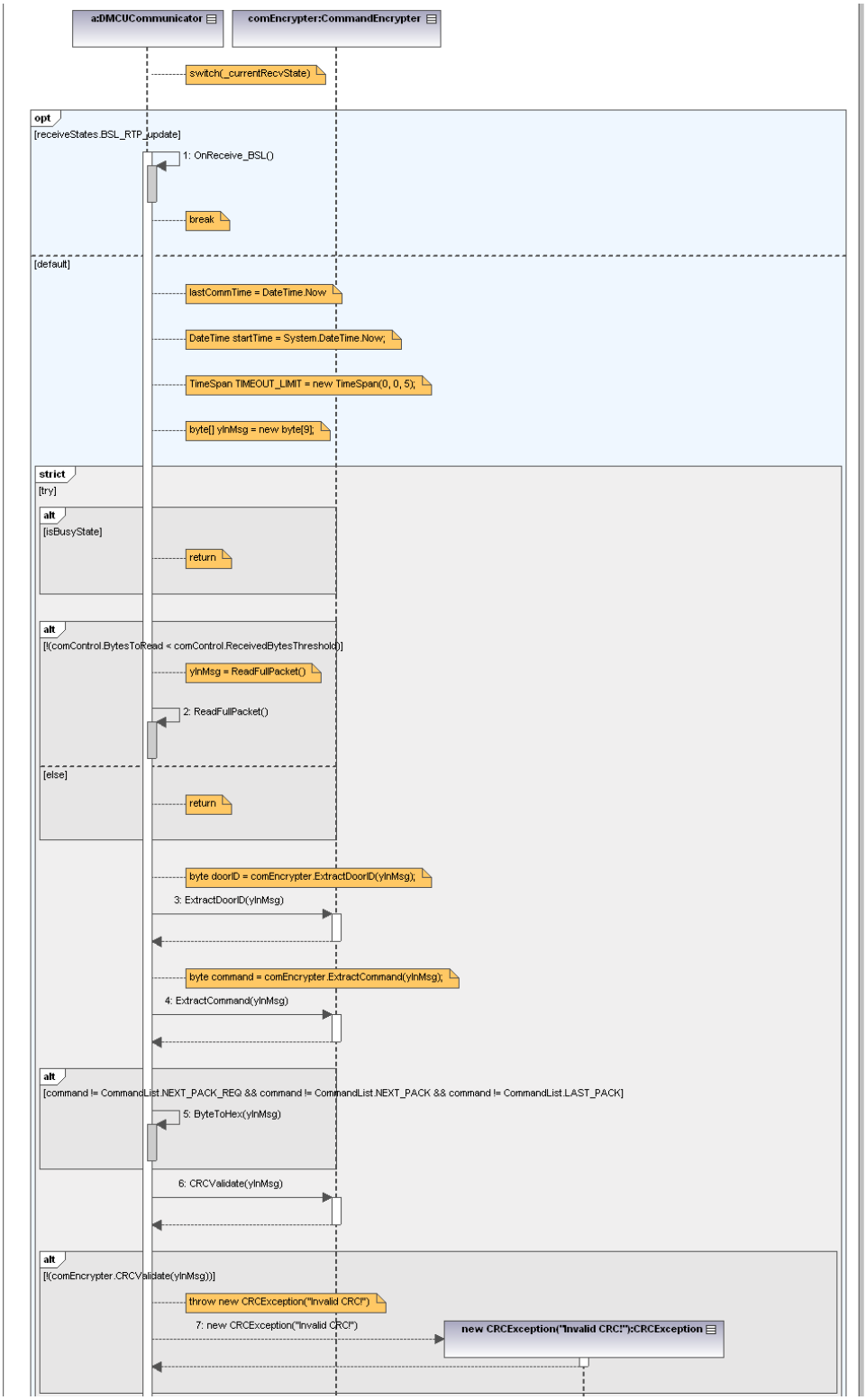
All functions starting with the keyword “On”, such as “OnReceive”, are functions triggered by an event. Usually the name of the function gives an indication as to what that event is. In the case of OnReceive(), it is triggered when receiving data through the communication port or socket. This OnReceive function is responsible for verifying that the entire command is read from the port and for redirecting the command to the appropriate parsing mechanism. This parsing mechanism is usually a sub function or a controller “Reply” type functions (or both).

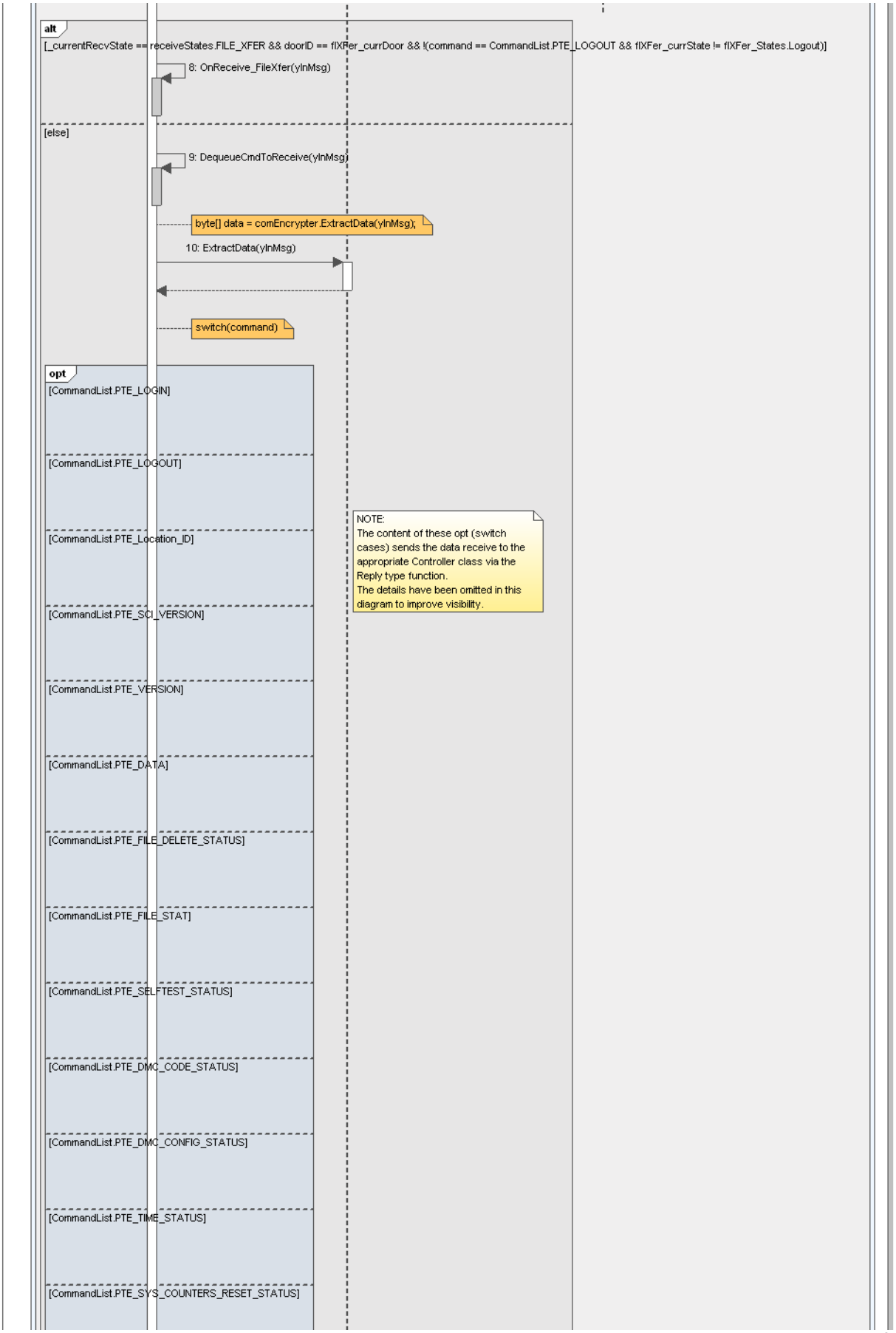
The following diagrams illustrate the inner workings of the CheckComQueue() and OnReceive functions.



Generated by UModel www.altova.com

Figure 3-42 Sequence Diagram - CheckComQueue


a:DMCUCommunicator
comEncrypter:CommandEncrypter
switch(_currentRecvState)
opt
[receiveStates.BSL_RTP_update]
1: OnReceive_BSL()
break
[default]
lastCommTime = DateTime.Now
DateTime startTime = System.DateTime.Now;
TimeSpan TIMEOUT_LIMIT = new TimeSpan(0, 0, 5);
byte[] ylnMsg = new byte[9];
strict
[try]
alt
[isBusyState]
return
alt
[!(comControl.BytesToRead < comControl.ReceivedBytesThreshold)]
ylnMsg = ReadFullPacket()
2: ReadFullPacket()
[else]
return
byte doorID = comEncrypter.ExtractDoorID(ylnMsg);
3: ExtractDoorID(ylnMsg)
byte command = comEncrypter.ExtractCommand(ylnMsg);
4: ExtractCommand(ylnMsg)
alt
[command != CommandList.NEXT_PACK_REQ && command != CommandList.NEXT_PACK && command != CommandList.LAST_PACK]
5: ByteToHex(ylnMsg)
6: CRCValidate(ylnMsg)
alt
[!(comEncrypter.CRCValidate(ylnMsg))]
throw new CRCException("Invalid CRC!")
7: new CRCException("Invalid CRC!")
new CRCException("Invalid CRC!"):CRCException


alt
[_currentRecvState == receiveStates.FILE_XFER && doorID == flXFer_currDoor && !(command == CommandList.PTE_LOGOUT && flXFer_currState != flXFer_States.Logout)]
8: OnReceive_FileXfer(ylnMsg)
[else]
9: DequeueCmdToReceive(ylnMsg)
byte[] data = comEncrypter.ExtractData(ylnMsg);
10: ExtractData(ylnMsg)
switch(command)
opt
[CommandList.PTE_LOGIN]
[CommandList.PTE_LOGOUT]
[CommandList.PTE_Location_ID]
[CommandList.PTE_SCI_VERSION]
[CommandList.PTE_VERSION]
[CommandList.PTE_DATA]
[CommandList.PTE_FILE_DELETE_STATUS]
[CommandList.PTE_FILE_STAT]
[CommandList.PTE_SELFTEST_STATUS]
[CommandList.PTE_DMC_CODE_STATUS]
[CommandList.PTE_DMC_CONFIG_STATUS]
[CommandList.PTE_TIME_STATUS]
[CommandList.PTE_SYS_COUNTERS_RESET_STATUS]
NOTE:
The content of these opt (switch cases) sends the data receive to the appropriate Controller class via the Reply type function.
The details have been omitted in this diagram to improve visibility.

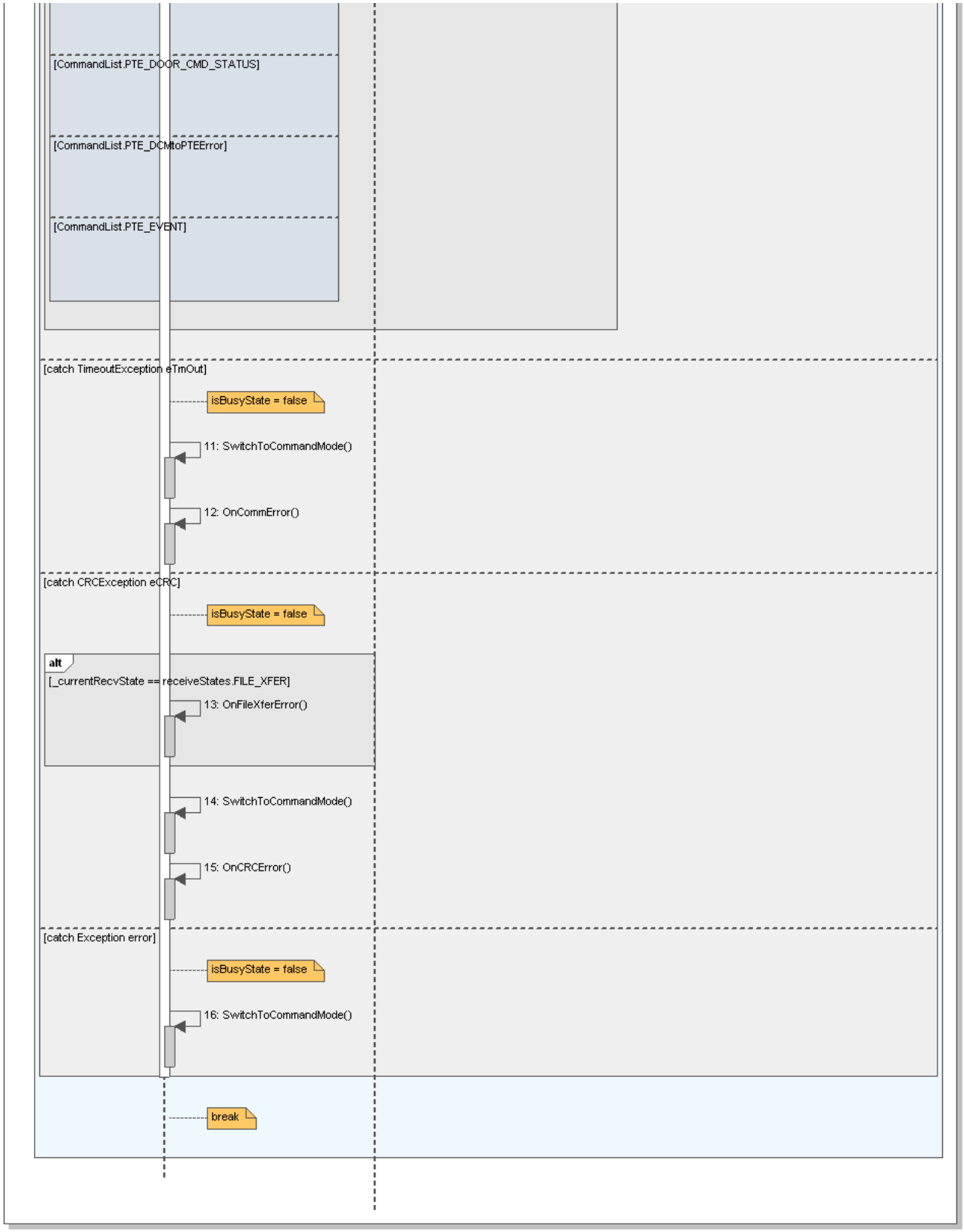


Figure 3-43 Sequence Diagram - OnReceive

3.7.2.2.2 File Transfer (send to DCU)

StartFileXfer is called by controllers to start a file transfer. This function is responsible for setting the file transfer variables and the correct flags to go into file transfer mode.

The variable _currentRecvState is mentioned in several diagrams in this document. It serves as a flag for the PTA to know if it is transferring commands or it is in the middle of a file transfer.

When sending the file to the DCU, the PTA is performing a file upload. The PTA needs to be logged into the Boot Loader. Therefore, at the end of the StartFileXfer, a logout is sent followed later by a login to ensure that the PTA is logged into the Boot Loader. Using the flags and a state machine strategy, the PTA then sends a sequence of commands to complete the transfer.

The following diagrams Figure 3-44 and Figure 3-45 illustrate the inner workings of the StartFileXfer() and OnreceiveFileXfer() functions:

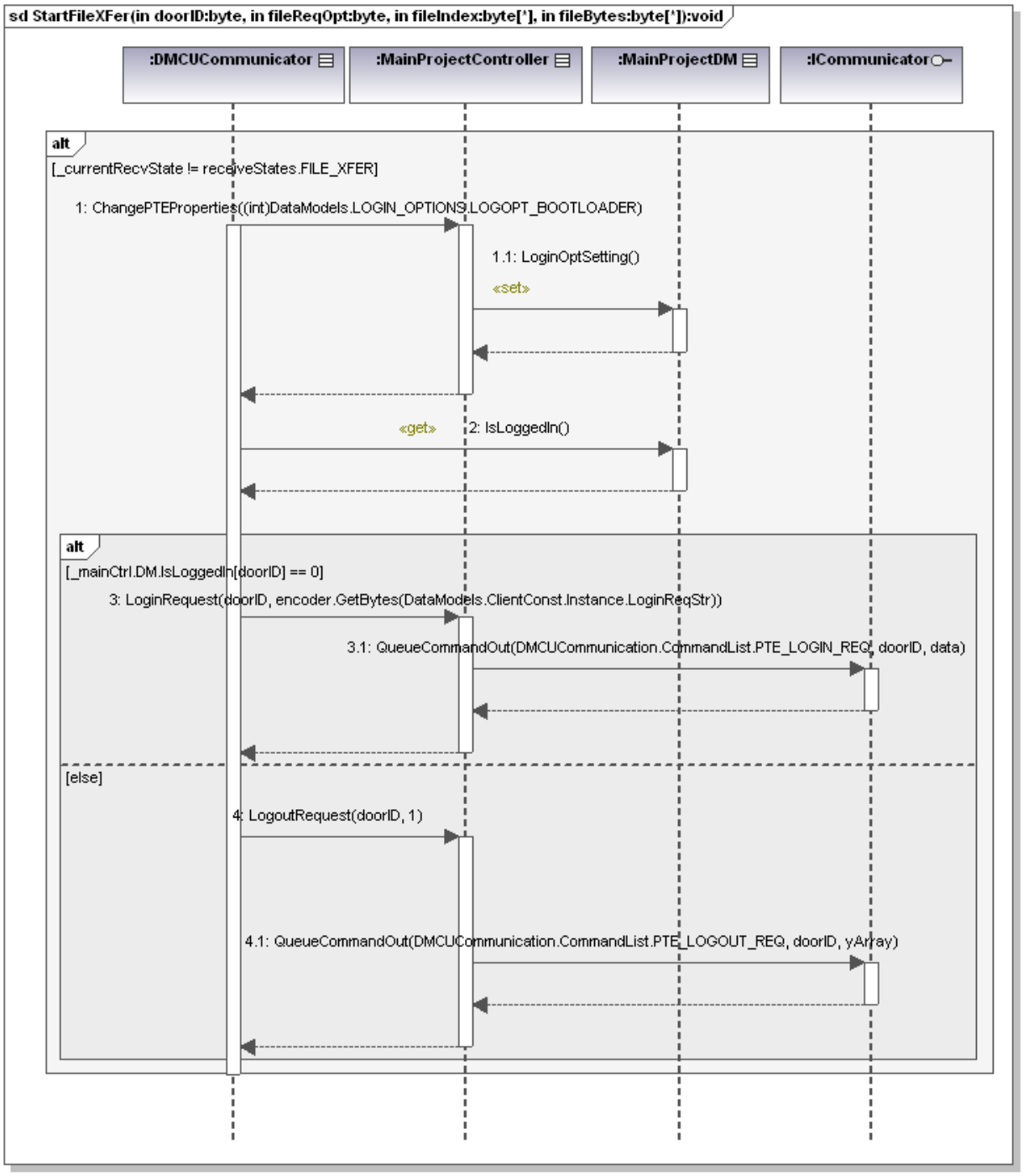
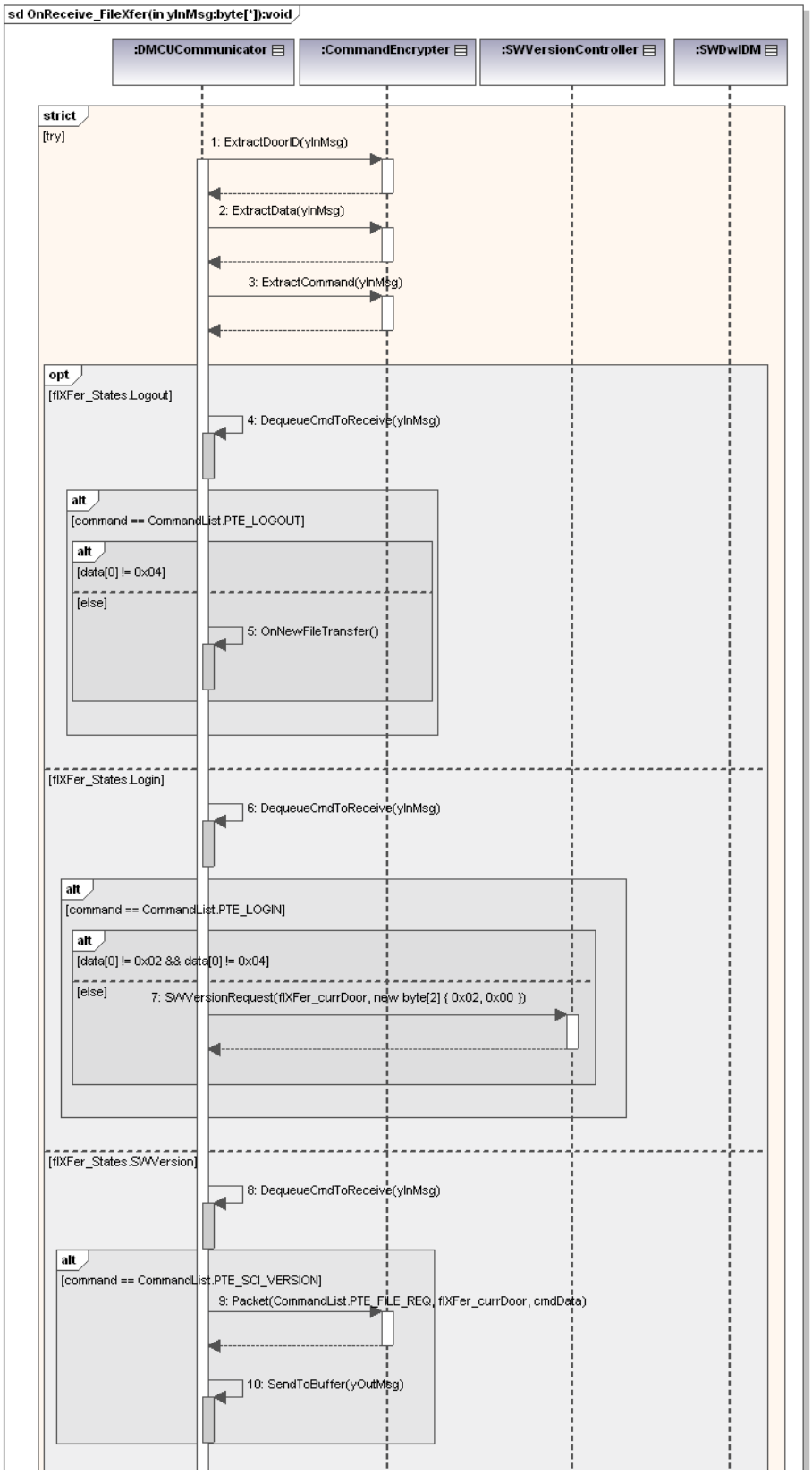
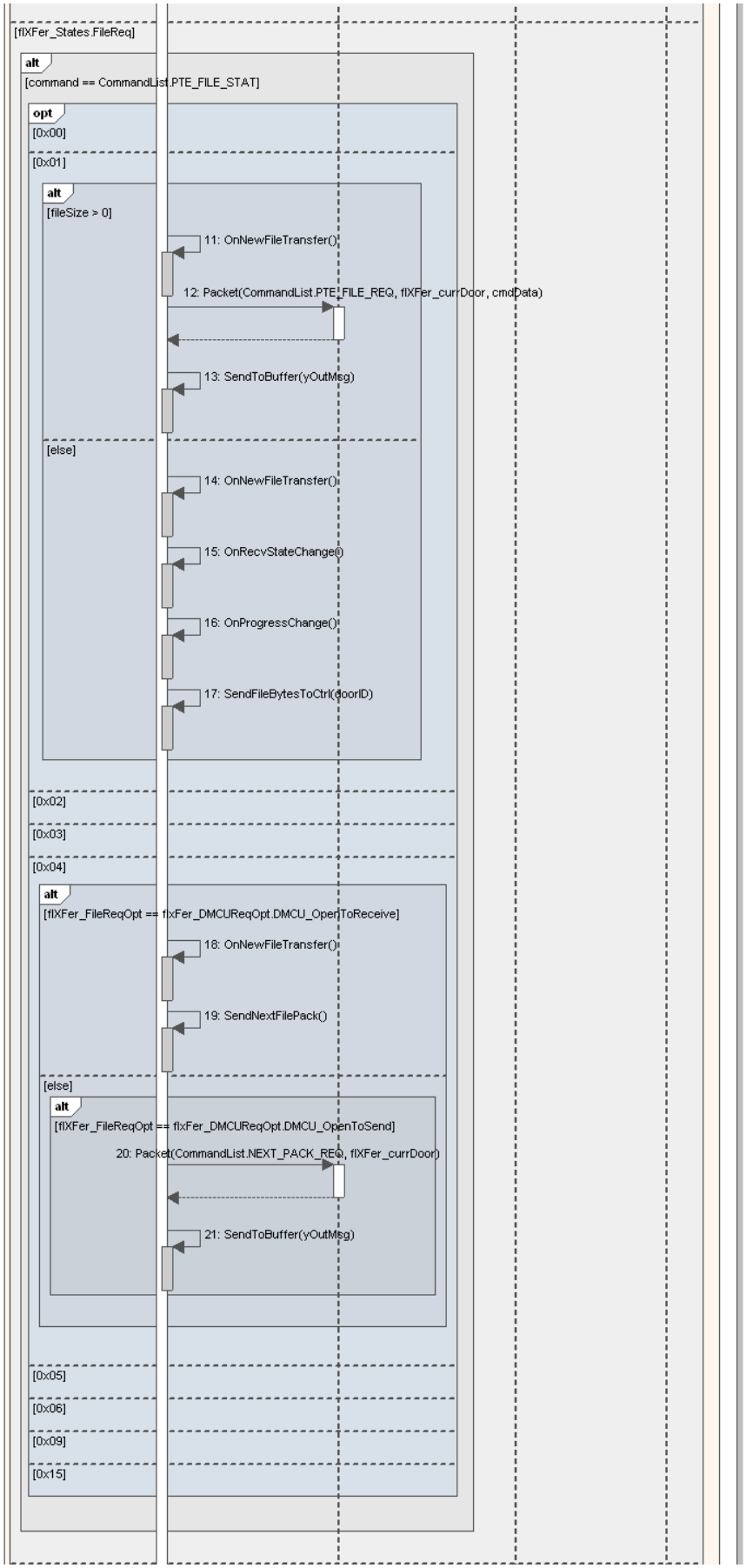
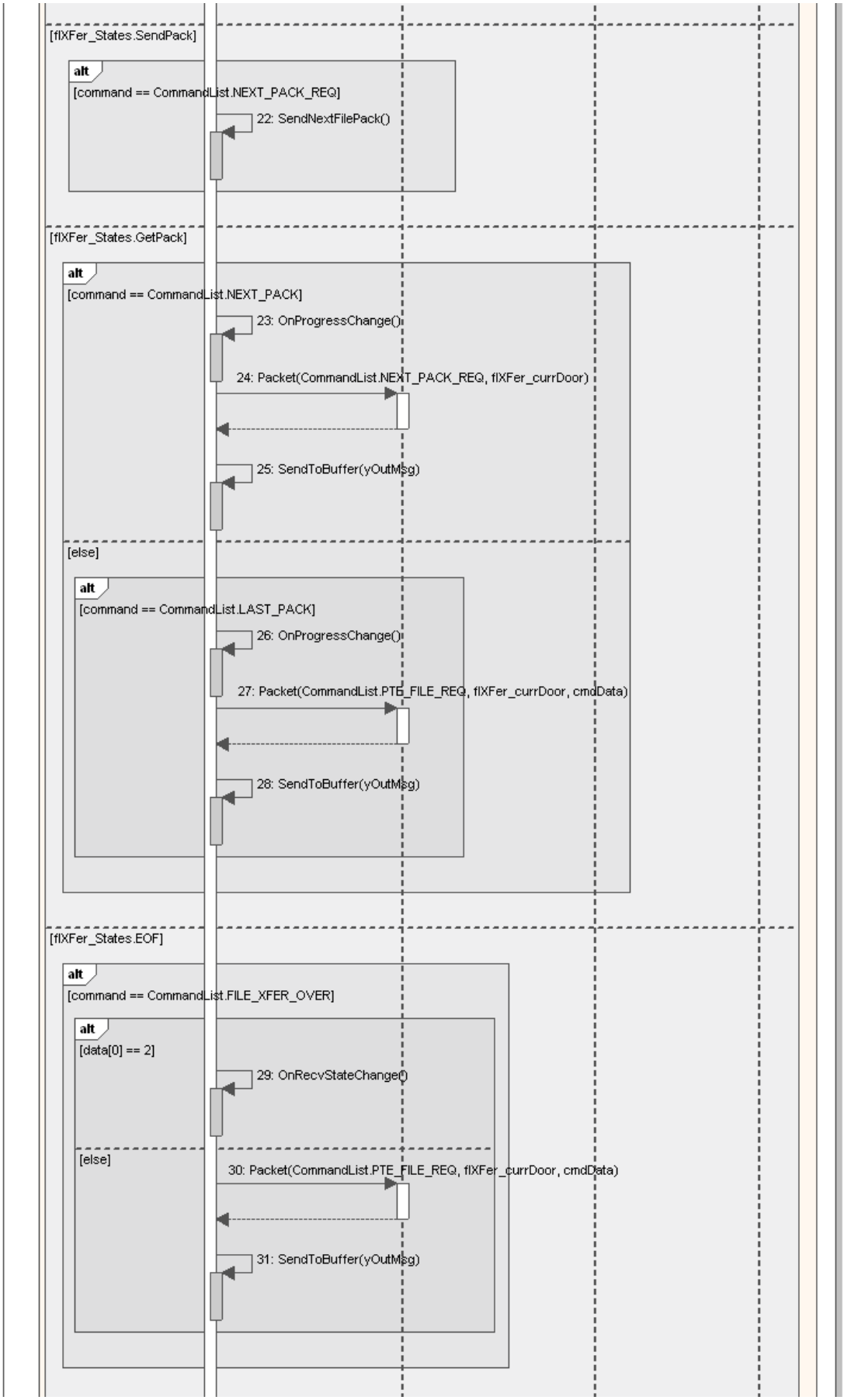


Figure 3-44 Sequence Diagram - StartFileXfer() (to DCU)


sd OnReceive_FileXfer(in yInMsg:byte[']):void
:DMCUCommunicator
:CommandEncrypter
:SWVersionController
:SWDwIDM
strict
[try]
1: ExtractDoorID(yInMsg)
2: ExtractData(yInMsg)
3: ExtractCommand(yInMsg)
opt
[flXFer_States.Logout]
4: DequeueCmdToReceive(yInMsg)
alt
[command == CommandList.PTE_LOGOUT]
alt
[data[0] != 0x04]
[else]
5: OnNewFileTransfer()
[flXFer_States.Login]
6: DequeueCmdToReceive(yInMsg)
alt
[command == CommandList.PTE_LOGIN]
alt
[data[0] != 0x02 && data[0] != 0x04]
[else]
7: SWVersionRequest(flXFer_currDoor, new byte[2] { 0x02, 0x00 })
[flXFer_States.SWVersion]
8: DequeueCmdToReceive(yInMsg)
alt
[command == CommandList.PTE_SCI_VERSION]
9: Packet(CommandList.PTE_FILE_REQ, flXFer_currDoor, cmdData)
10: SendToBuffer(yOutMsg)


[flXFer_States.FileReq]
alt
[command == CommandList.PTE_FILE_STAT]
opt
[0x00]
[0x01]
alt
[fileSize > 0]
11: OnNewFileTransfer()
12: Packet(CommandList.PTE_FILE_REQ, flXFer_currDoor, cmdData)
13: SendToBuffer(yOutMsg)
[else]
14: OnNewFileTransfer()
15: OnRecvStateChange()
16: OnProgressChange()
17: SendFileBytesToCtrl(doorID)
[0x02]
[0x03]
[0x04]
alt
[flXFer_FileReqOpt == flxFer_DMCUReqOpt.DMCU_OpenToReceive]
18: OnNewFileTransfer()
19: SendNextFilePack()
[else]
alt
[flXFer_FileReqOpt == flxFer_DMCUReqOpt.DMCU_OpenToSend]
20: Packet(CommandList.NEXT_PACK_REQ, flXFer_currDoor)
21: SendToBuffer(yOutMsg)
[0x05]
[0x06]
[0x09]
[0x15]


[flXFer_States.SendPack]
alt
[command == CommandList.NEXT_PACK_REQ]
22: SendNextFilePack()
[flXFer_States.GetPack]
alt
[command == CommandList.NEXT_PACK]
23: OnProgressChange()
24: Packet(CommandList.NEXT_PACK_REQ, flXFer_currDoor)
25: SendToBuffer(yOutMsg)
[else]
alt
[command == CommandList.LAST_PACK]
26: OnProgressChange()
27: Packet(CommandList.PTE_FILE_REQ, flXFer_currDoor, cmdData)
28: SendToBuffer(yOutMsg)
[flXFer_States.EOF]
alt
[command == CommandList.FILE_XFER_OVER]
alt
[data[0] == 2]
29: OnRecvStateChange()
[else]
30: Packet(CommandList.PTE_FILE_REQ, flXFer_currDoor, cmdData)
31: SendToBuffer(yOutMsg)

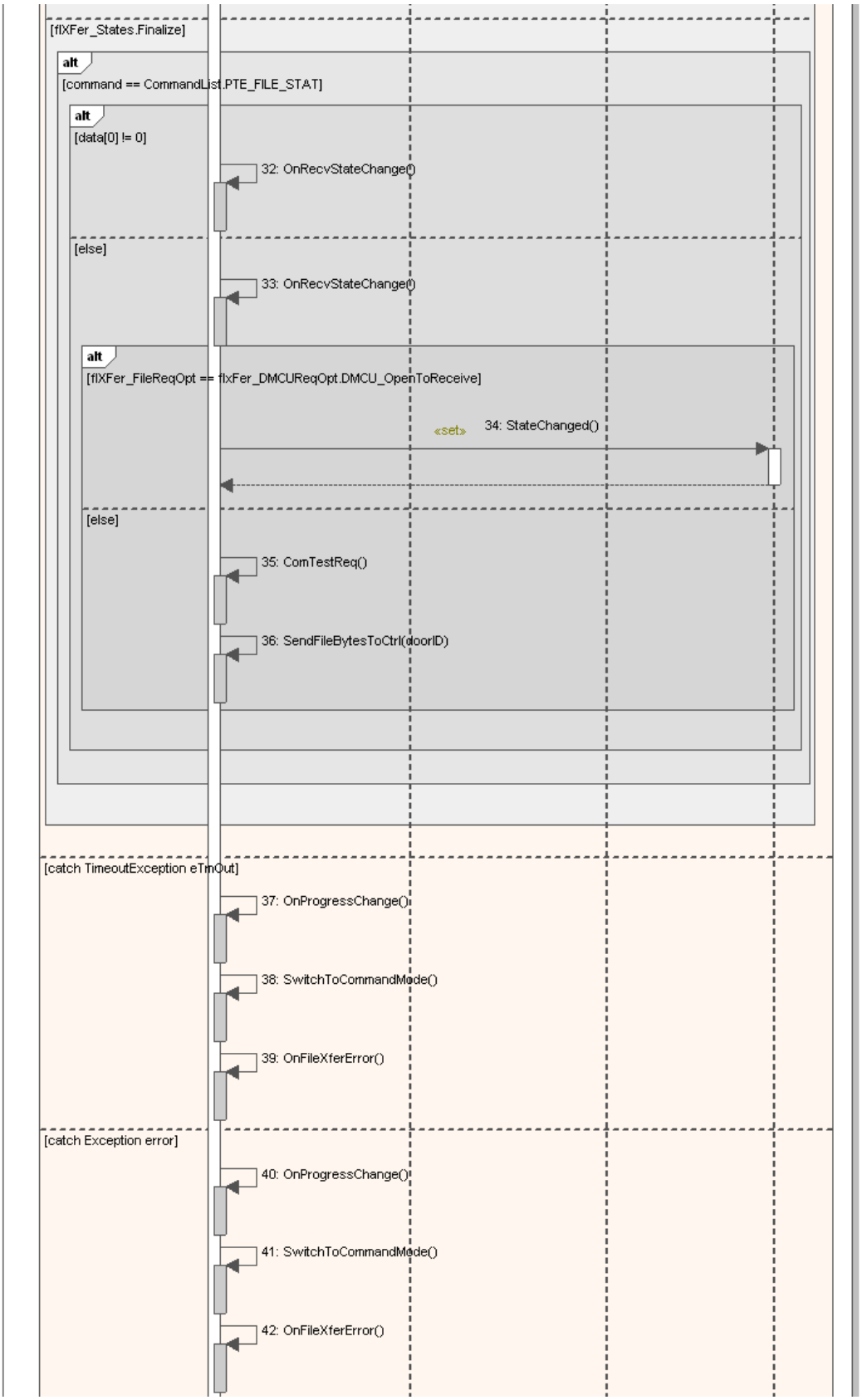


Figure 3-45 Sequence Diagram - OnReceive_FileXfer

3.7.2.2.3 File Transfer (receive from DCU)

The File_Transfer handles the file transfer commands. They are:

- PTA_FileReq
- PTA_FileRsp
- PTA_NextPackData
- PTA_NextPackAck
- PTA_LastPackData
- PTA_FileXferOverAck

When receiving the file from the DCU the PTA is doing a download. Unlike an upload, there is no need to send a logout to change from DCU Application to Boot Loader. If the PTA is already logged into Boot Loader, files downloads not allowed by the Boot Loader will not be available through the GUI.

Figure 3-46 describes the file transfer:

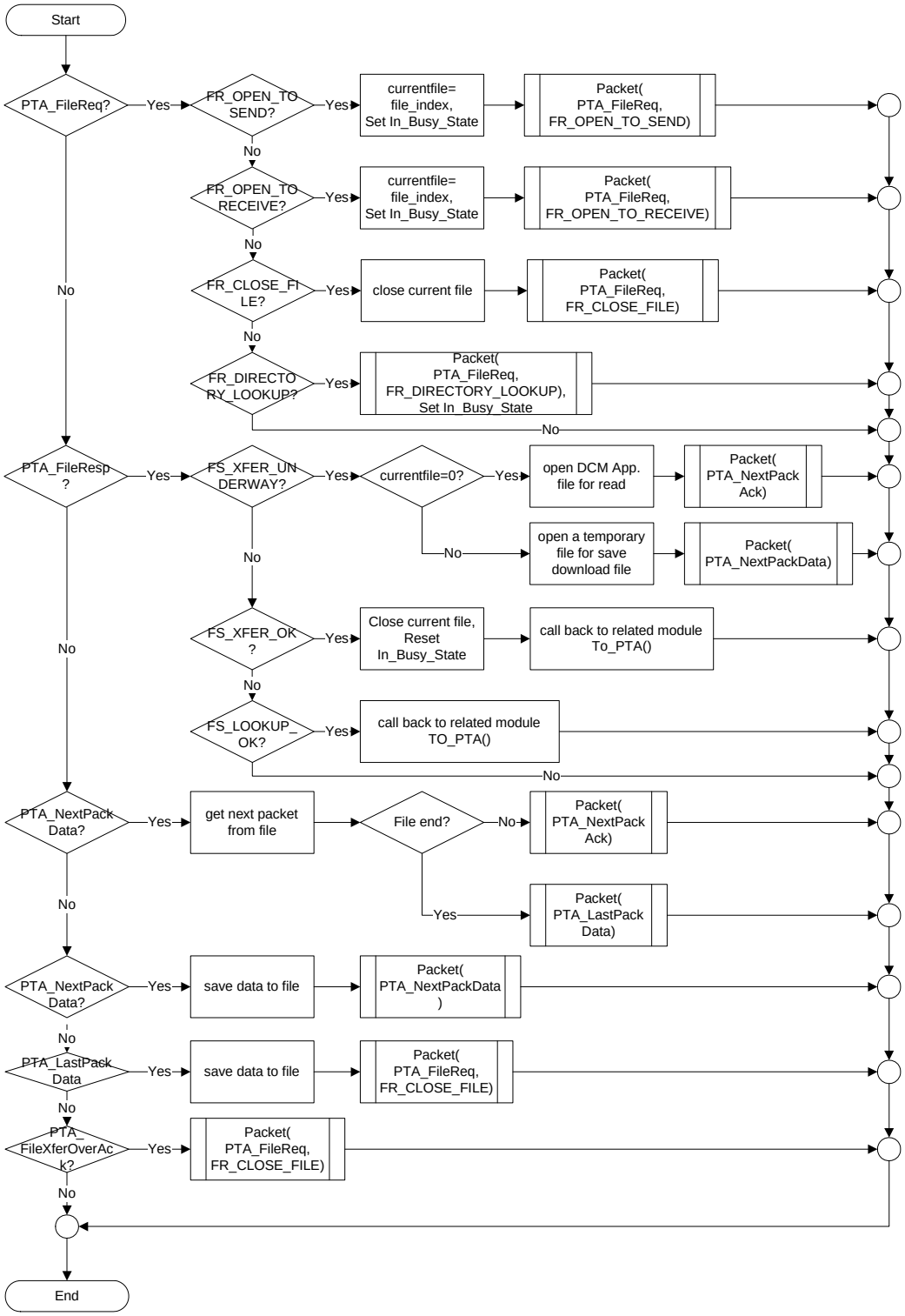


Figure 3-46 Data Communication – File Transfer Flow Chart (Receive from DCU)

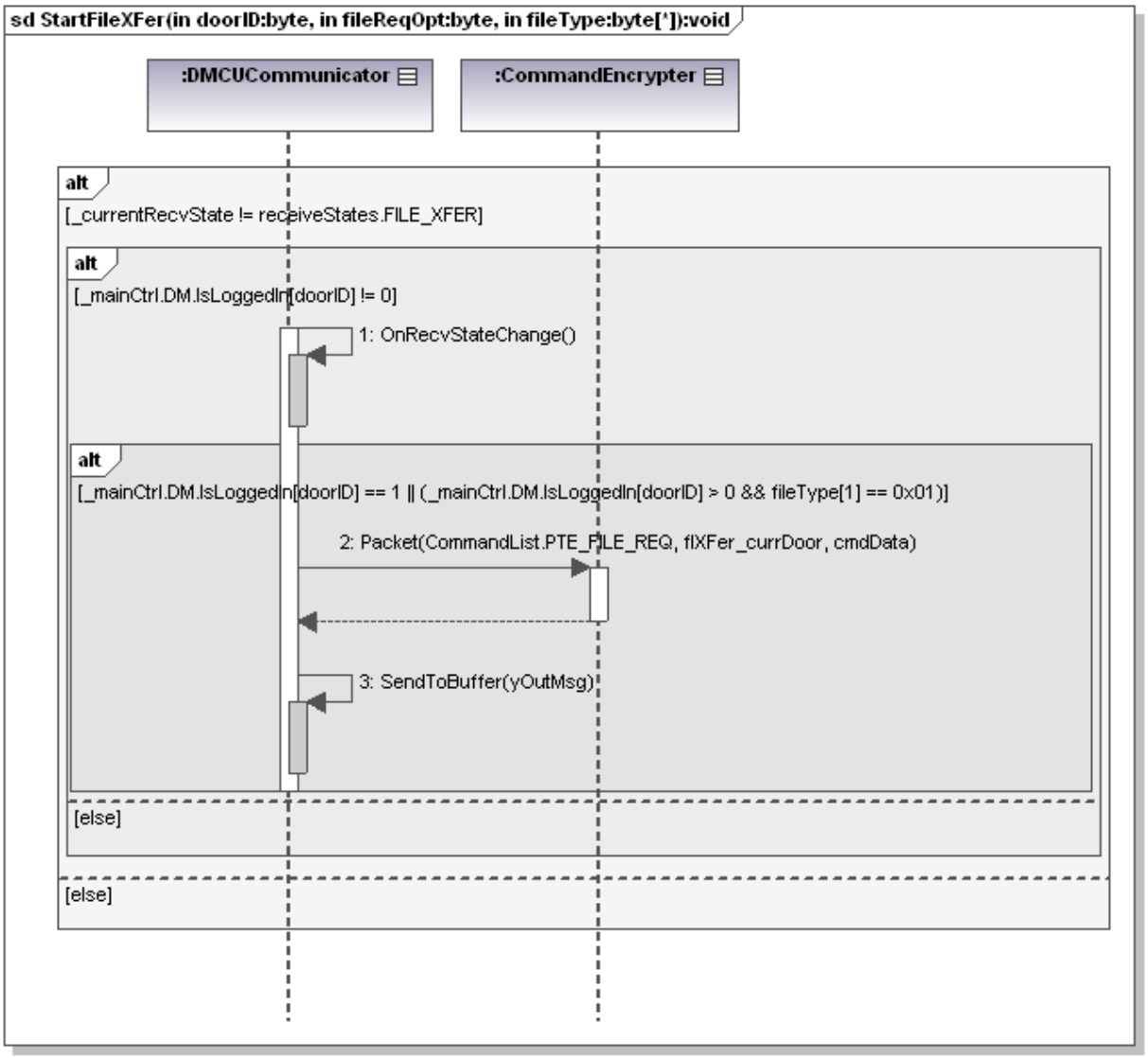


Figure 3-47 Sequence Diagram - StartFileXfer (receive from DCU)

The reception of a file transfer is handled through the same function as the function for sending a file to the DCU. See Figure 3-45.

3.7.2.2.4 Boot Loader Upgrade

The Boot Loader update handles the transfer of the BootLoader file to the DCU. It can also transfer an RTP tool file since the process is exactly the same on the PTA side. The reason why the BootLoader update is not classified as a file transfer is that it follows an entirely different protocol. The Boot Loader update must go through a series of communication that do not follow the regular packaging of bytes.

The Boot Loader upgrade is done in two parts. The first part is loading and verifying the flash file which will be sent to the DCU (See Figure 3-48). The second part is the actual sending of the boot loader flash file following a precise sequence. The entire function operates like a state machine (See Figure 3-49).

Note: OnReceive_BSL is called by the OnReceive() function detailed in Figure 3-43

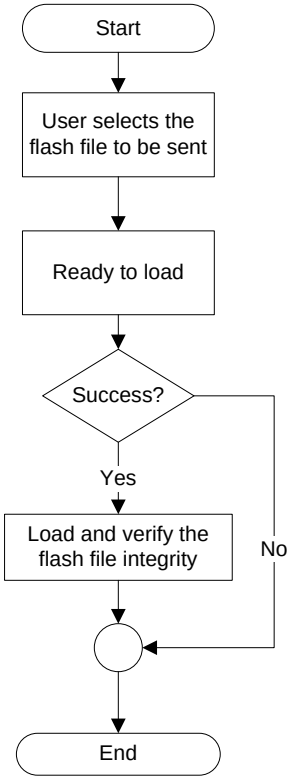


Figure 3-48 Flowchart - Load Flash File

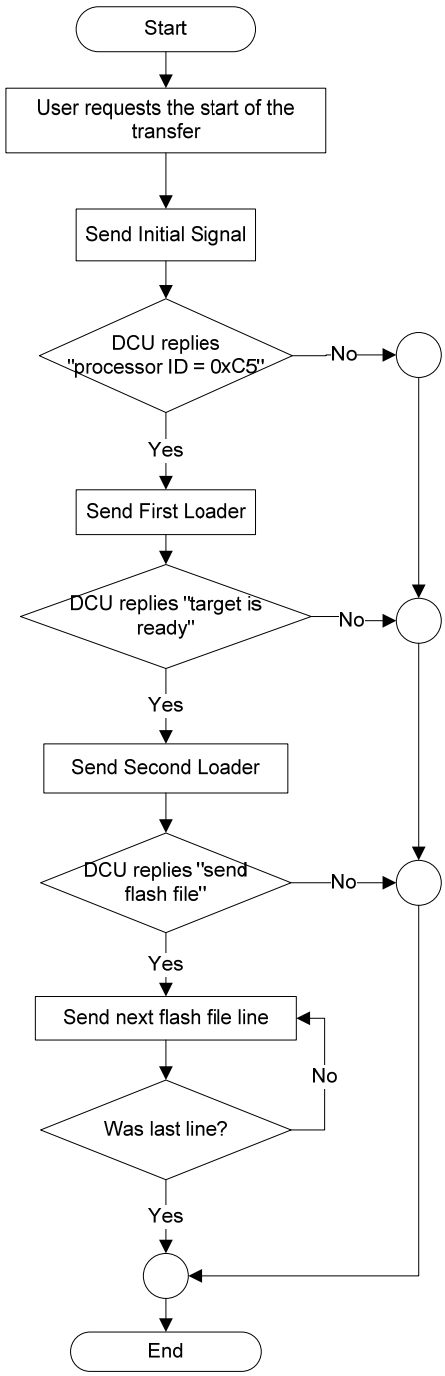
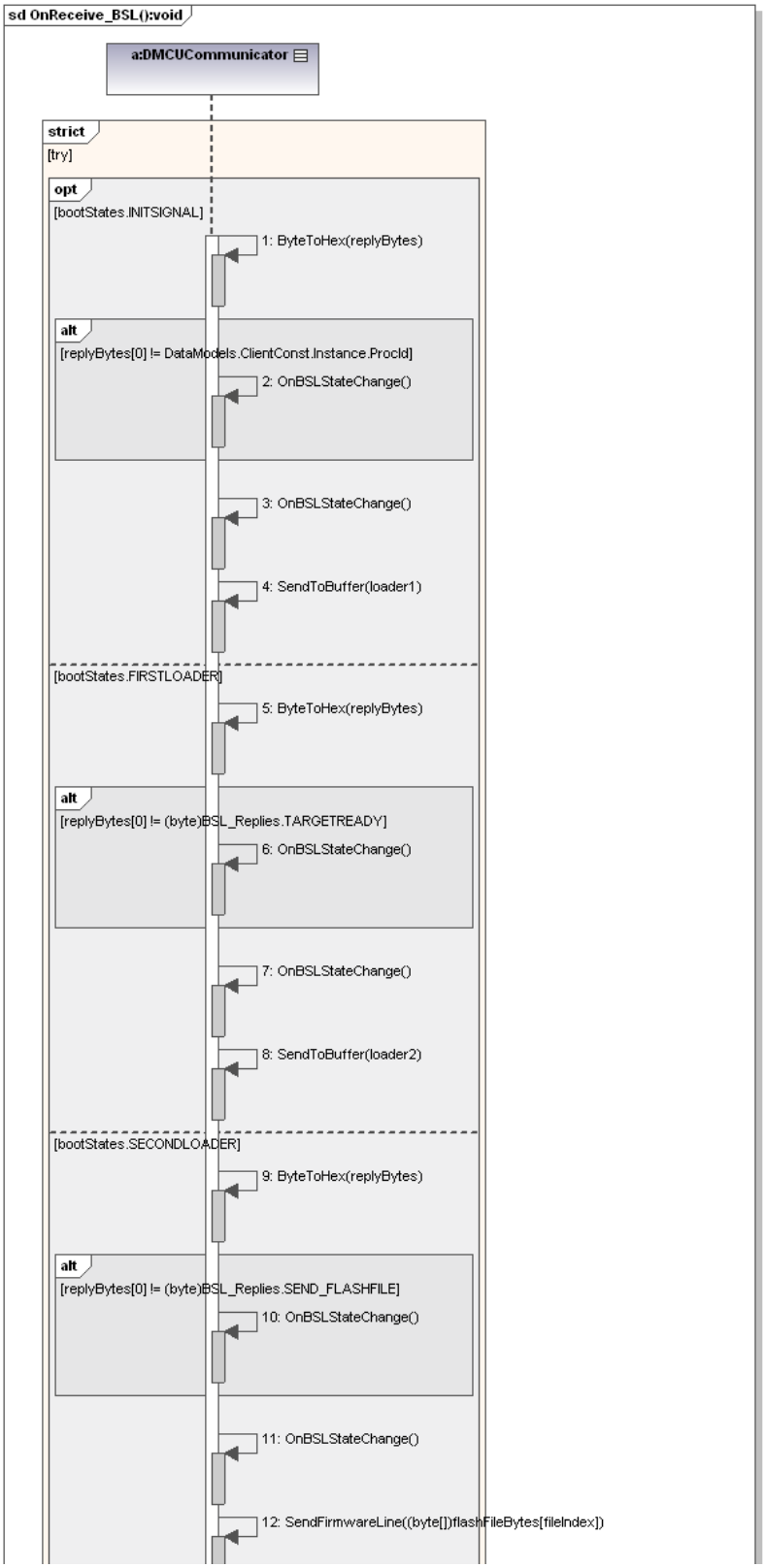


Figure 3-49 FlowChart – Update Boot Loader


sd OnReceive_BSL():void
a:DMCUCommunicator
strict
[try]
opt
[bootStates.INITSIGNAL]
1: ByteToHex(replyBytes)
alt
[replyBytes[0] != DataModels.ClientConst.Instance.ProcId]
2: OnBSLStateChange()
3: OnBSLStateChange()
4: SendToBuffer(loader1)
[bootStates.FIRSTLOADER]
5: ByteToHex(replyBytes)
alt
[replyBytes[0] != (byte)BSL_Replies.TARGETREADY]
6: OnBSLStateChange()
7: OnBSLStateChange()
8: SendToBuffer(loader2)
[bootStates.SECONDLOADER]
9: ByteToHex(replyBytes)
alt
[replyBytes[0] != (byte)BSL_Replies.SEND_FLASHFILE]
10: OnBSLStateChange()
11: OnBSLStateChange()
12: SendFirmwareLine((byte[])flashFileBytes[fileIndex])

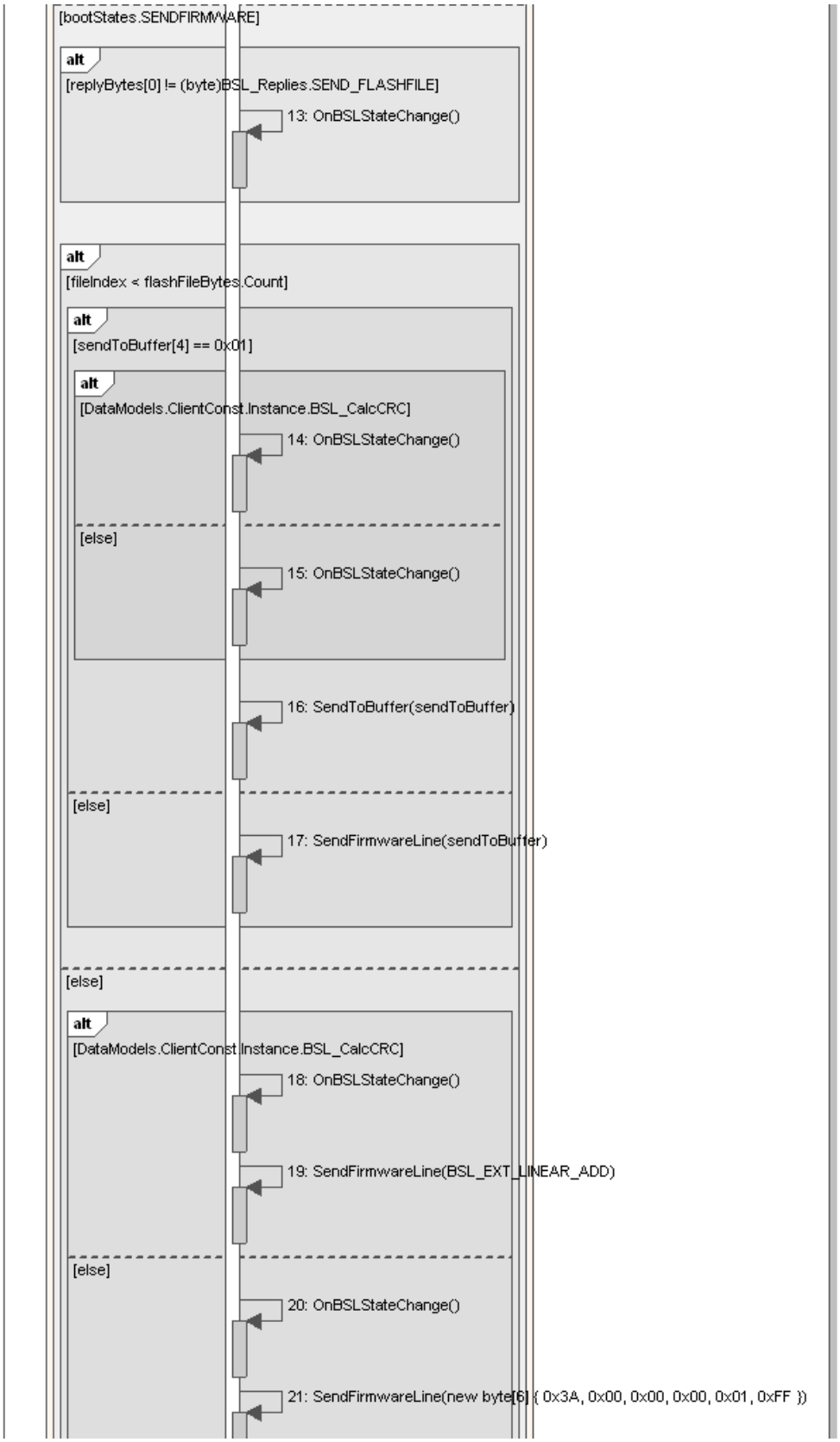

[bootStates.SENDFIRMWARE]
alt
[replyBytes[0] != (byte)BSL_Replies.SEND_FLASHFILE]
13: OnBSLStateChange()
alt
[fileIndex < flashFileBytes.Count]
alt
[sendToBuffer[4] == 0x01]
alt
[DataModels.ClientConst.Instance.BSL_CalcCRC]
14: OnBSLStateChange()
[else]
15: OnBSLStateChange()
16: SendToBuffer(sendToBuffer)
[else]
17: SendFirmwareLine(sendToBuffer)
[else]
alt
[DataModels.ClientConst.Instance.BSL_CalcCRC]
18: OnBSLStateChange()
19: SendFirmwareLine(BSL_EXT_LINEAR_ADD)
[else]
20: OnBSLStateChange()
21: SendFirmwareLine(new byte[6] { 0x3A, 0x00, 0x00, 0x00, 0x01, 0xFF })

Figure 3-50 Sequence Diagram - OnReceive_BSL ()

3.7.3 SW Download Module

The SW Download Module is composed of three main entities that operate in an MVC fashion. In addition, the GUI has access to part of the SW Version Module, in order to display the header. The Software Download Module classes are presented in Figure 3-51.

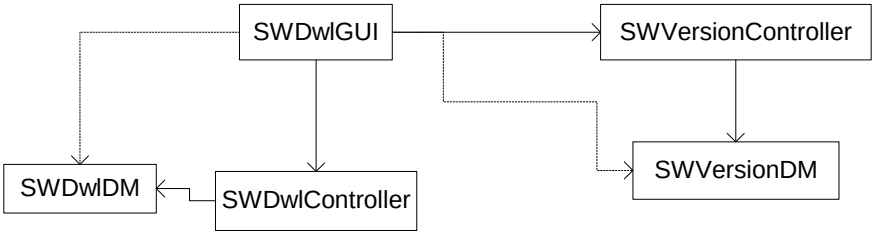


Figure 3-51 SW Download Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

3.7.3.1 Interfaces

As per the global architecture described in Section 3.5, the GUI in this module interacts with the PTA Main module by being a child GUI of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.3.2 Functionalities

3.7.3.2.1 DCU Application Upgrade

This functionality is used to upgrade the DCU Application. It is available only when a valid password is entered at start-up. The DCU Application does the DCU Application Upgrade by transferring a binary file to the DCU. When a DCU Application file is selected, the file CRC32 is validated. If a CRC32 error happens, the application upgrade is not available. Any error during upgrade is reported.Figure 3-52 is showing the flowchart of the DCU application upgrade functionality. The numerotation of this communication diagram indicates the order of appearance of the messages. When 2 or more messages have the same index with a different sub-index (i.e. 2.1, 2.2), it indicates that they are sent in the same time sequence.

Figure 3-53 shows the communication diagram for the DCU application upgrade functionality.

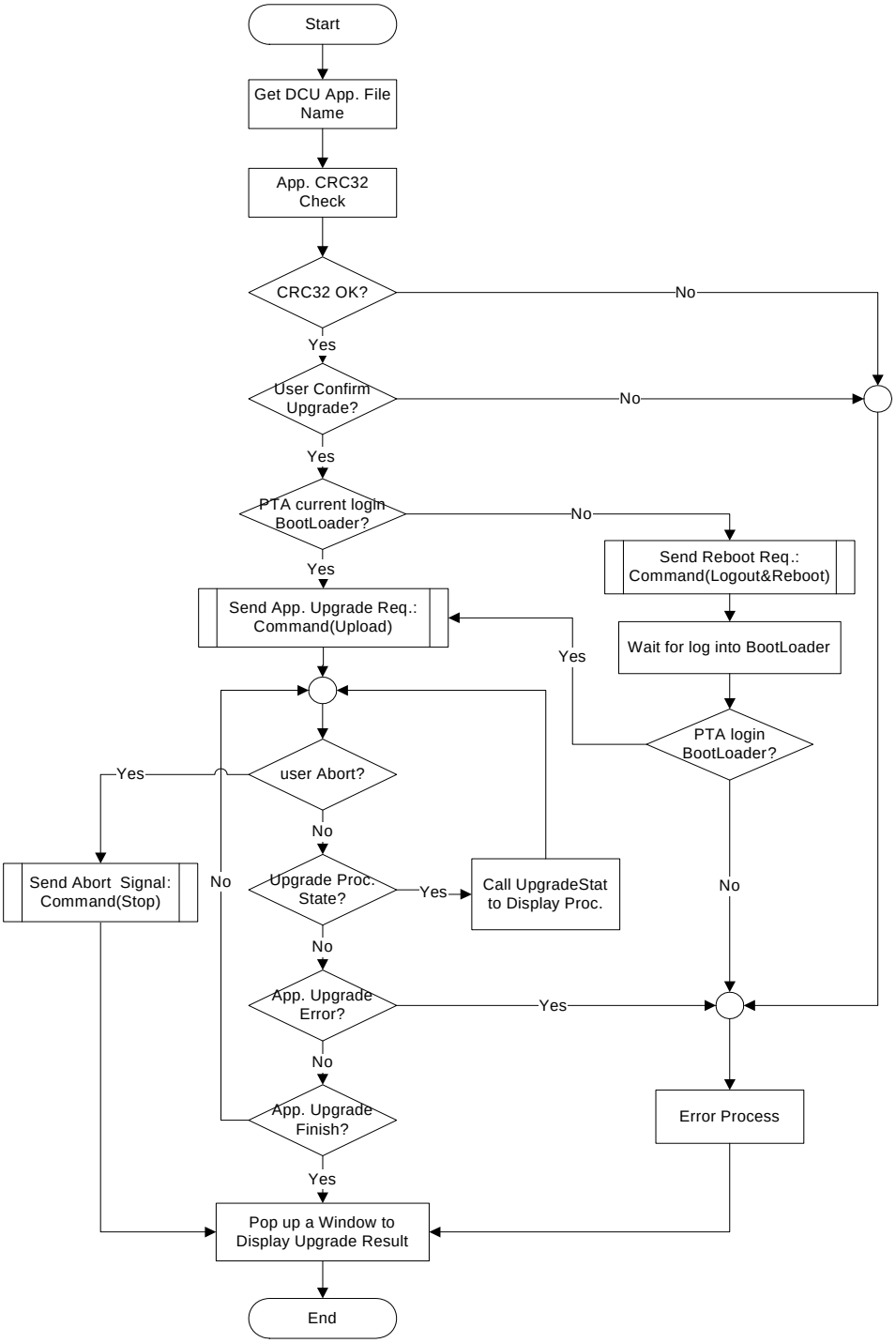
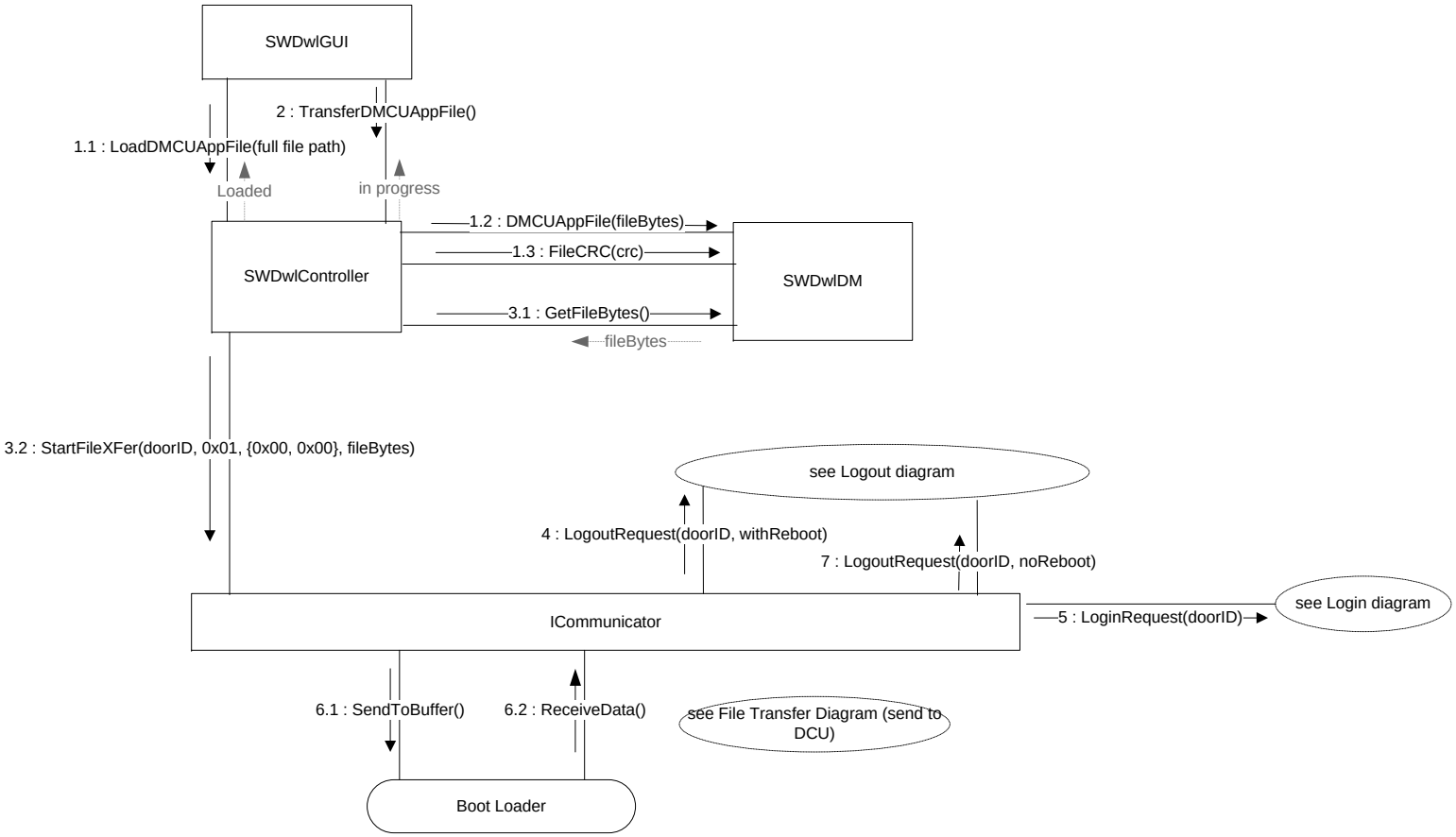


Figure 3-52 Flow Chart - DCU Application Upgrade



The numerotation of this communication diagram indicates the order of appearance of the messages. When 2 or more messages have the same index with a different sub-index (i.e. 2.1, 2.2), it indicates that they are sent in the same time sequence.

Figure 3-53 Communication Diagram - DCU Application Upgrade

This page was intentionally left blank.

3.7.4 Events & Log Module

The Events Module operates in an MVC fashion, however it has several views. Some views are merely observers. The views acting solely as observers are grayed out in Figure 3-54.

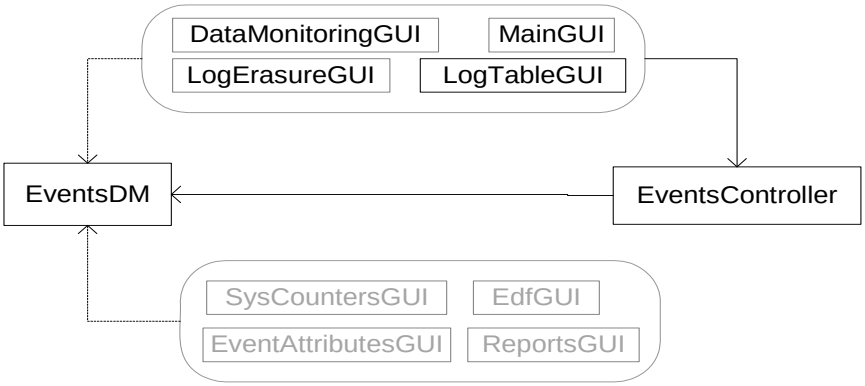


Figure 3-54 Events & Log Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

3.7.4.1 Interfaces

As per the global architecture described in Section 3.5, the GUIs in this module interact with the PTA Main module by being child GUIs of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.4.2 Functionalities

3.7.4.2.1 EDF Download & display

The EDF Download and display behaves in the same way as other file transfers. For more details on file transfers, see section 3.7.2.2.3.

The following Diagrams show the sequence in which the EDF download and display occurs, and how it coordinates with another function: OPDF Update.

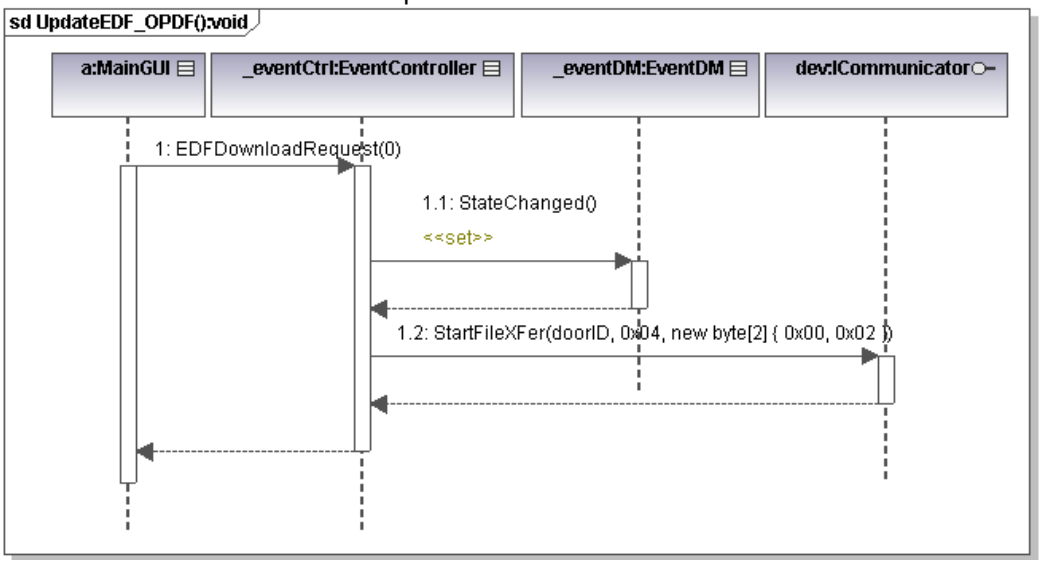


Figure 3-55 Sequence Diagram - UpdateEDF_OPDF()

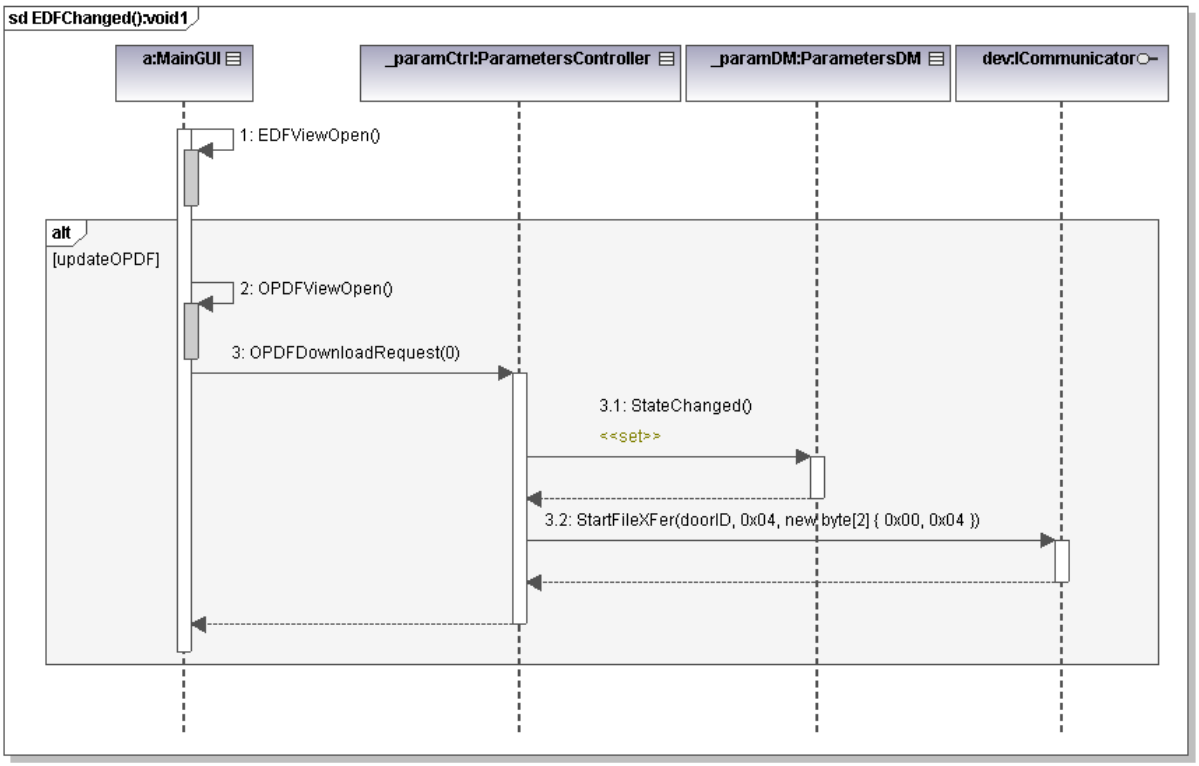


Figure 3-56 Sequence Diagram - EDFChanged()

3.7.4.2.2 EDF Read & Write

The EDF must always be loaded to ensure proper PTA functionality. Several mechanisms are in place to ensure this.

Upon starting the PTA, the application looks for the following file:

```
X:\PTA\Data\Door\edf.xml
```

From this file, the EDF is loaded. If the *EDF.xml* file is missing or corrupted, the PTA will load a default hardcoded copy of the EDF. This version was hardcoded during development prior to building the installer. This hardcoded version is not a file that can be deleted by the user.

When the EDF is successfully loaded, the PTA re-writes the *EDF.xml* file. The Figure 3-57shows the flowchart of the EDF Read and Write function. The Figure 3-58 presents the sequence diagram of the ReadEDFFile() function.

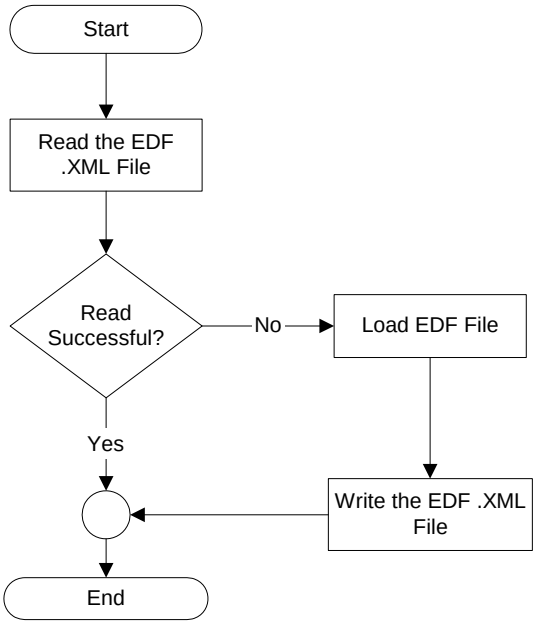
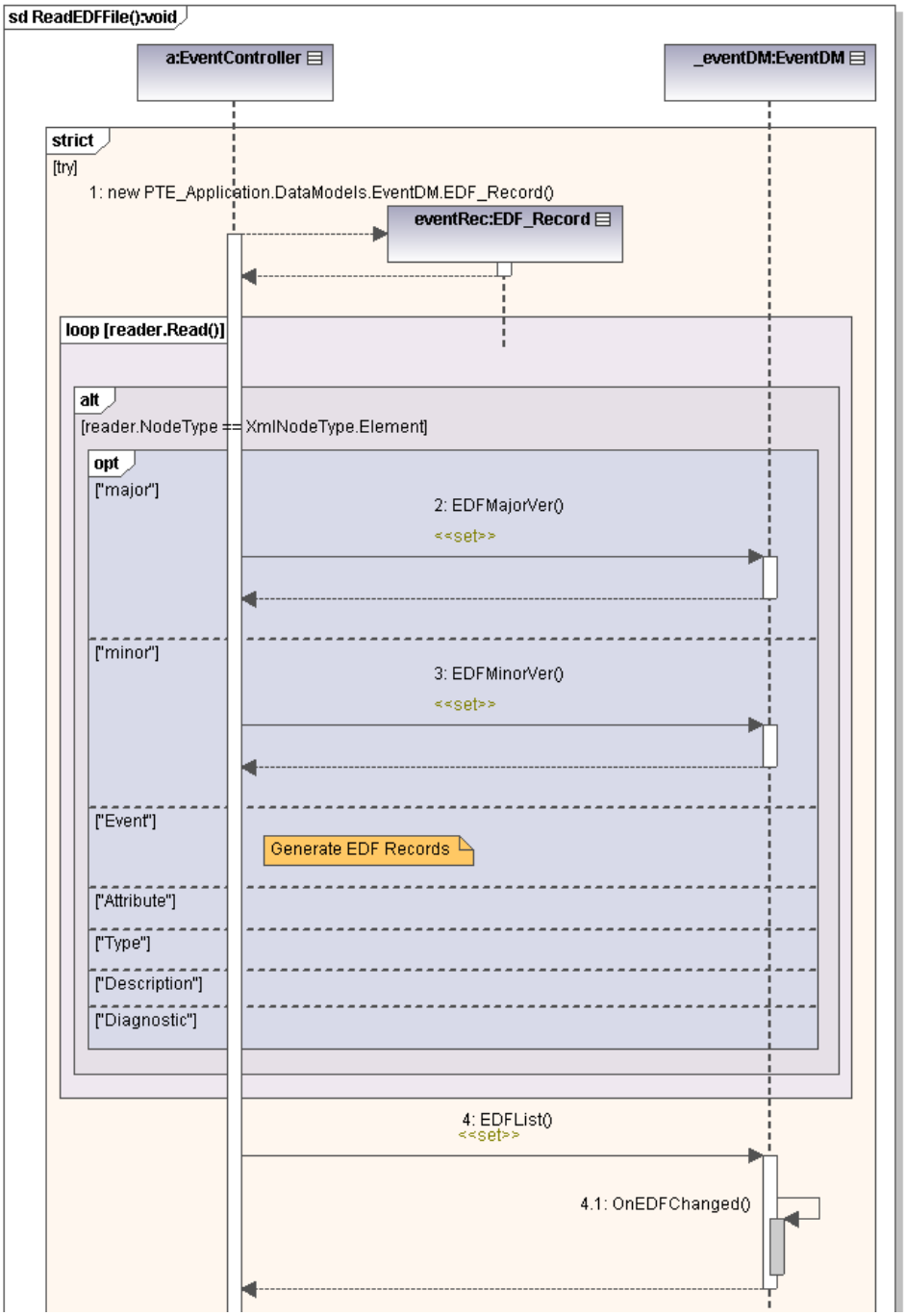


Figure 3-57 Flow Chart - EDF Read & Write


sd ReadEDFFile():void
a:EventController
_eventDM:EventDM
strict
[try]
1: new PTE_Application.DataModels.EventDM.EDF_Record()
eventRec:EDF_Record
loop [reader.Read()]
alt
[reader.NodeType == XmlNodeType.Element]
opt
["major"]
2: EDFMajorVer()
<<set>>
["minor"]
3: EDFMinorVer()
<<set>>
["Event"]
Generate EDF Records
["Attribute"]
["Type"]
["Description"]
["Diagnostic"]
4: EDFList()
<<set>>
4.1: OnEDFChanged()

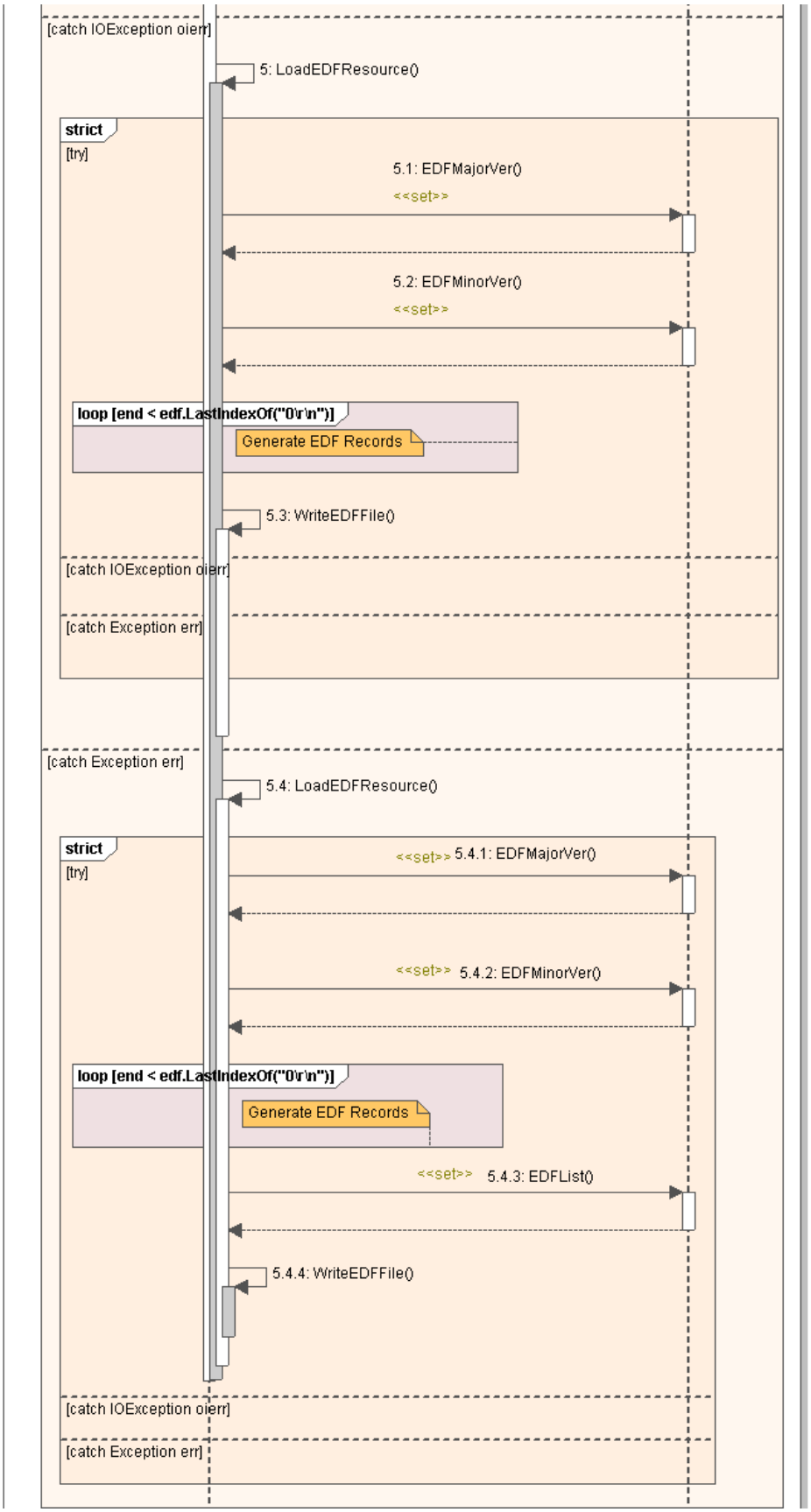


Figure 3-58 Sequence Diagram - ReadEDFFile()

3.7.4.2.3 Event Sync

The Event Sync is triggered in two ways. The first is by request from the user, through the DataMonitoringGUI, which gives the events currently flagged as active. The second is automatically done right after receiving a Data Reply (see section 3.7.5.2.3). The later will retrieve any new events flagged.

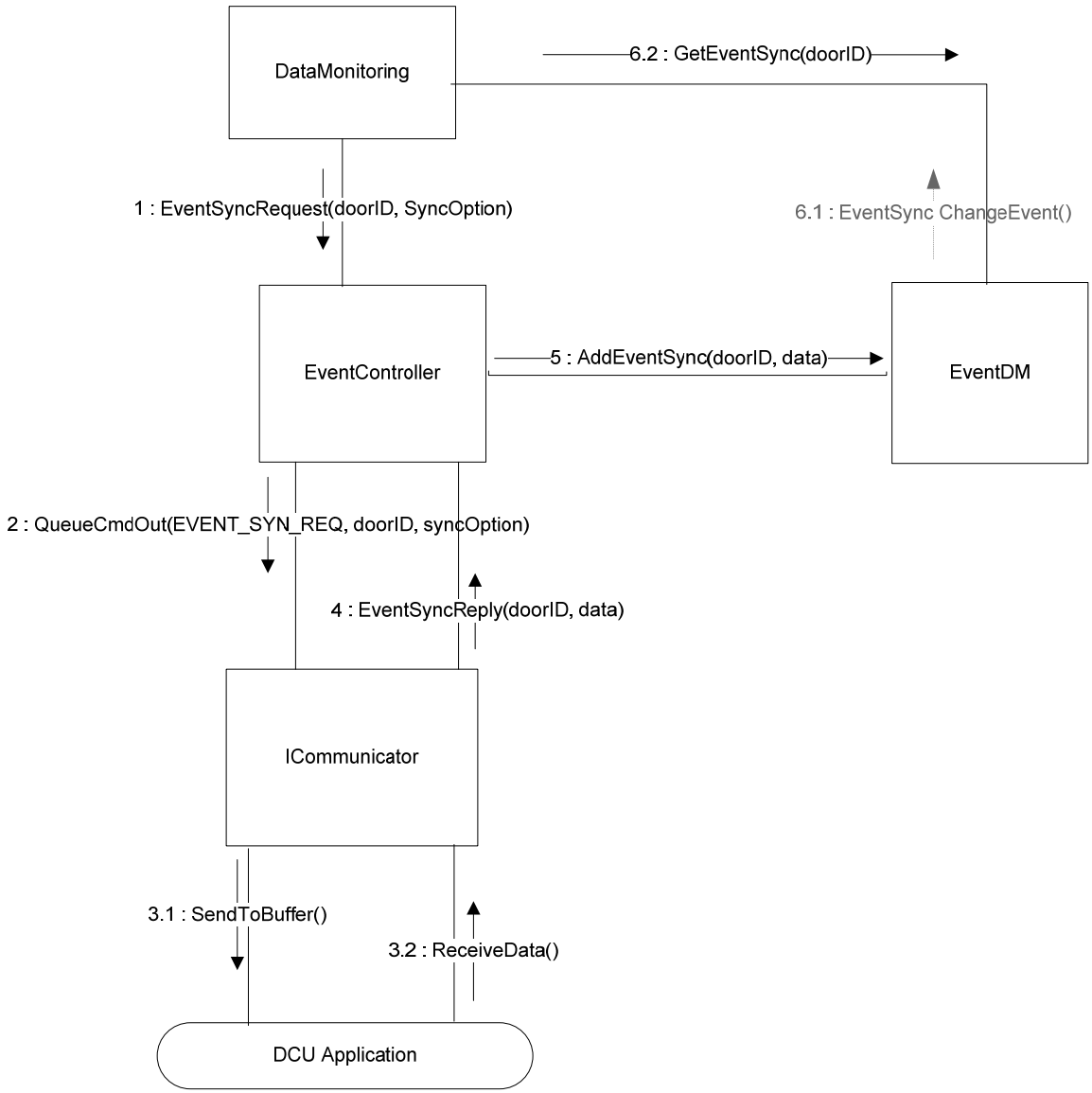


Figure 3-59 Communication Diagram - EventSync

3.7.4.2.4 Log Download & Display

The Log Download and display behaves in the same way as other file transfers. For more details on file transfers, see section 3.7.2.2.3. The Figure 3-60 shows the communication diagram of the Log Download and Display function.

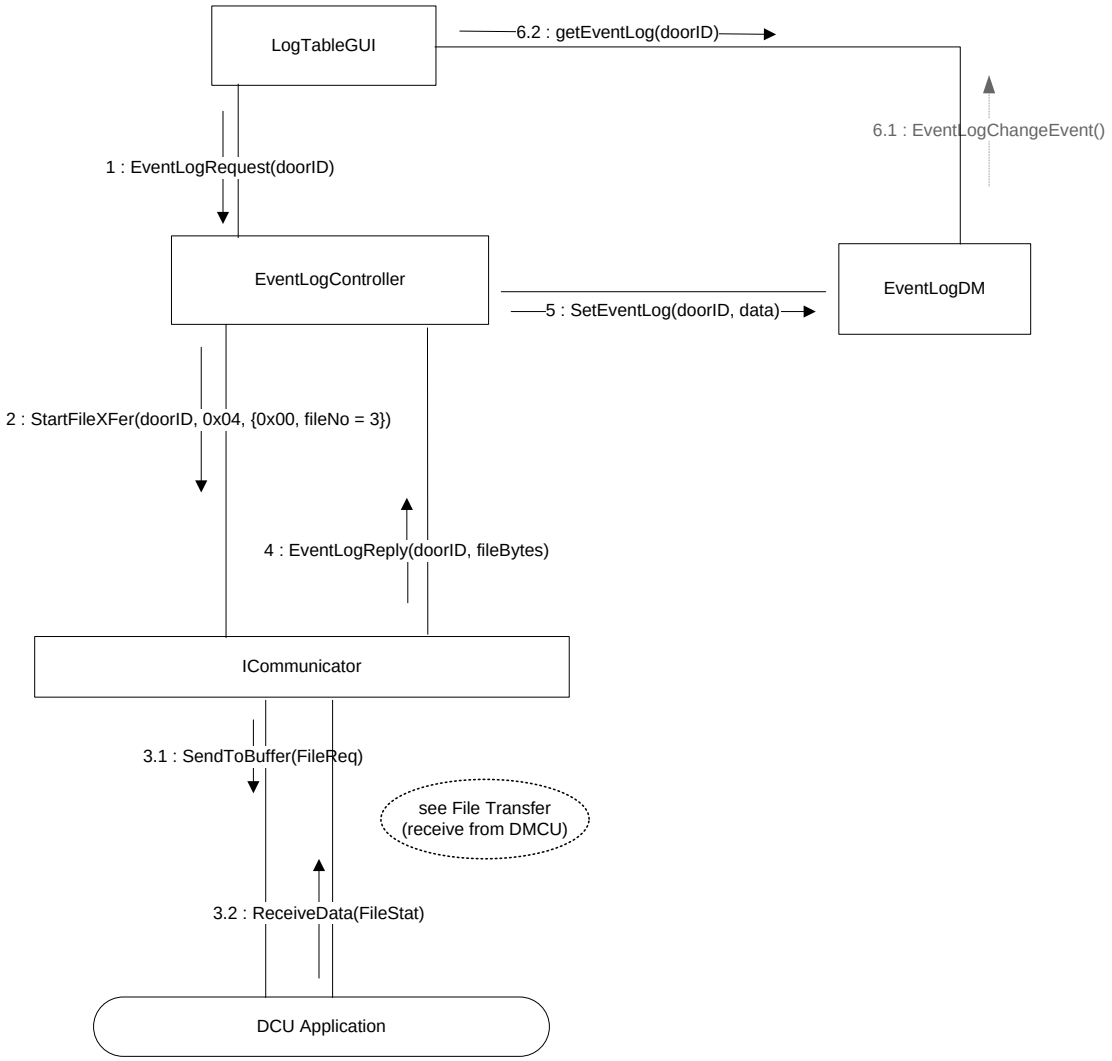


Figure 3-60 Communication Diagram - Log Download & Display

3.7.4.2.5 Log Erasure

The Log Erasure is initiated by the Log Erasure GUI, after the user has gone through all the confirmation steps.

After the Log Erasure, the GUI sends out a system counters reset. The details of this reset are described in section 3.7.12. Figure 3-61 and Figure 3-62 show respectively the flowchart and communication diagram of the Log Erasure functionality.

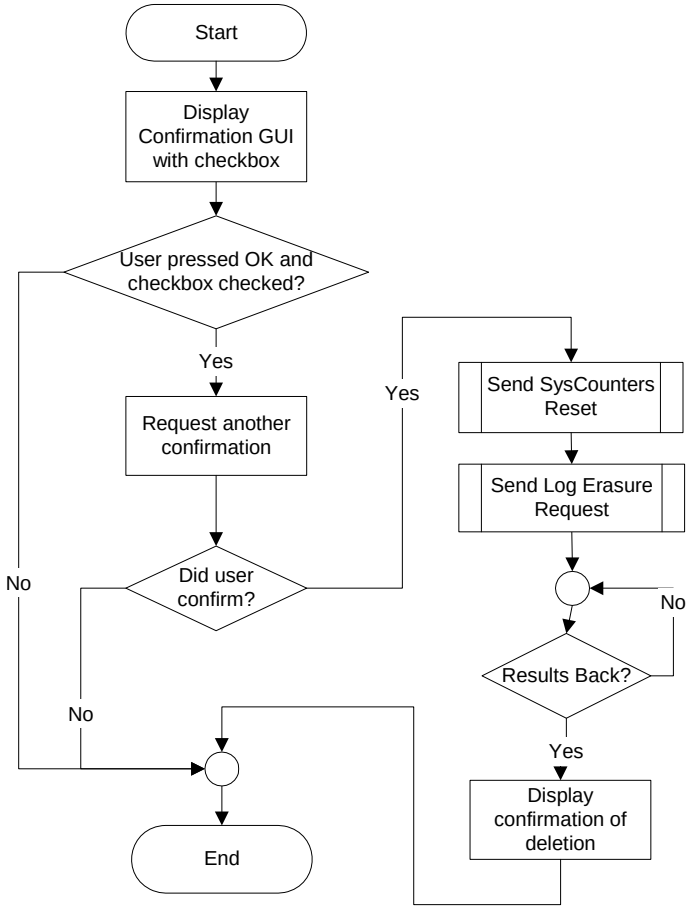


Figure 3-61 Flow Chart - Log Erasure

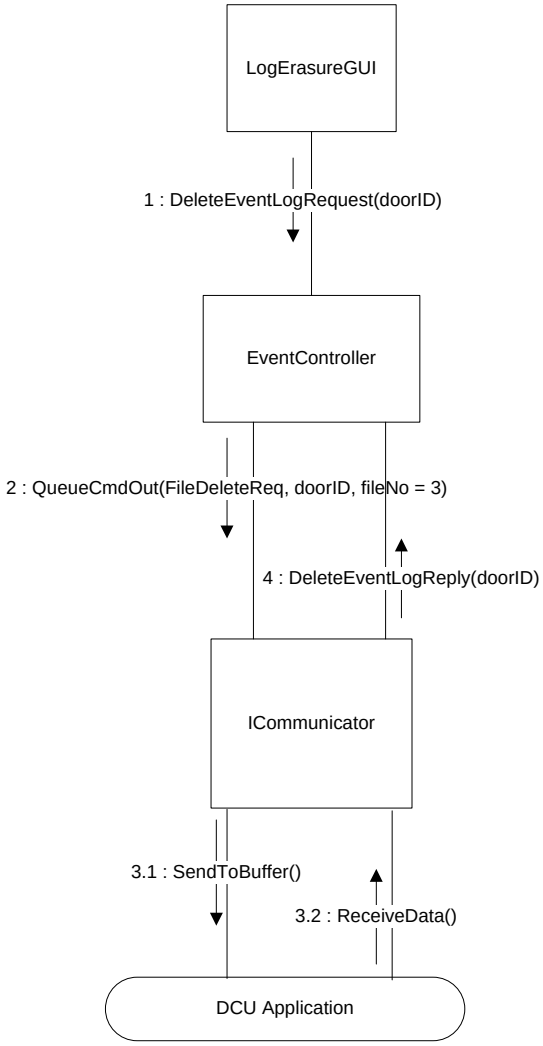


Figure 3-62 Communication Diagram - Log Erasure

3.7.5 Parameters & Snapshots Module

The Parameters & Snapshot Module operates in an MVC fashion, however it has several views. Some views are merely observers. The views acting solely as observers are grayed out in Figure 3-63.

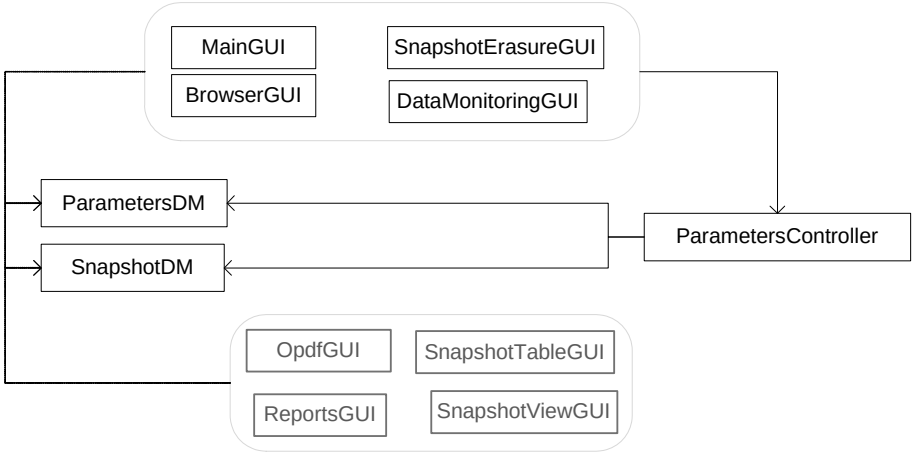


Figure 3-63 Parameters & Snapshot Module Classes

3.7.5.1 Interfaces

As per the global architecture described in Section 3.5, the GUIs in this module interact with the PTA Main module by being child GUIs of the MainGUI. The Controller in this module interacts with the Communicator Module.

The SnapshotViewGUI also interfaces with the NPlot library, an external library that facilitates the drawing of plots and graphics.

3.7.5.2 Functionalities

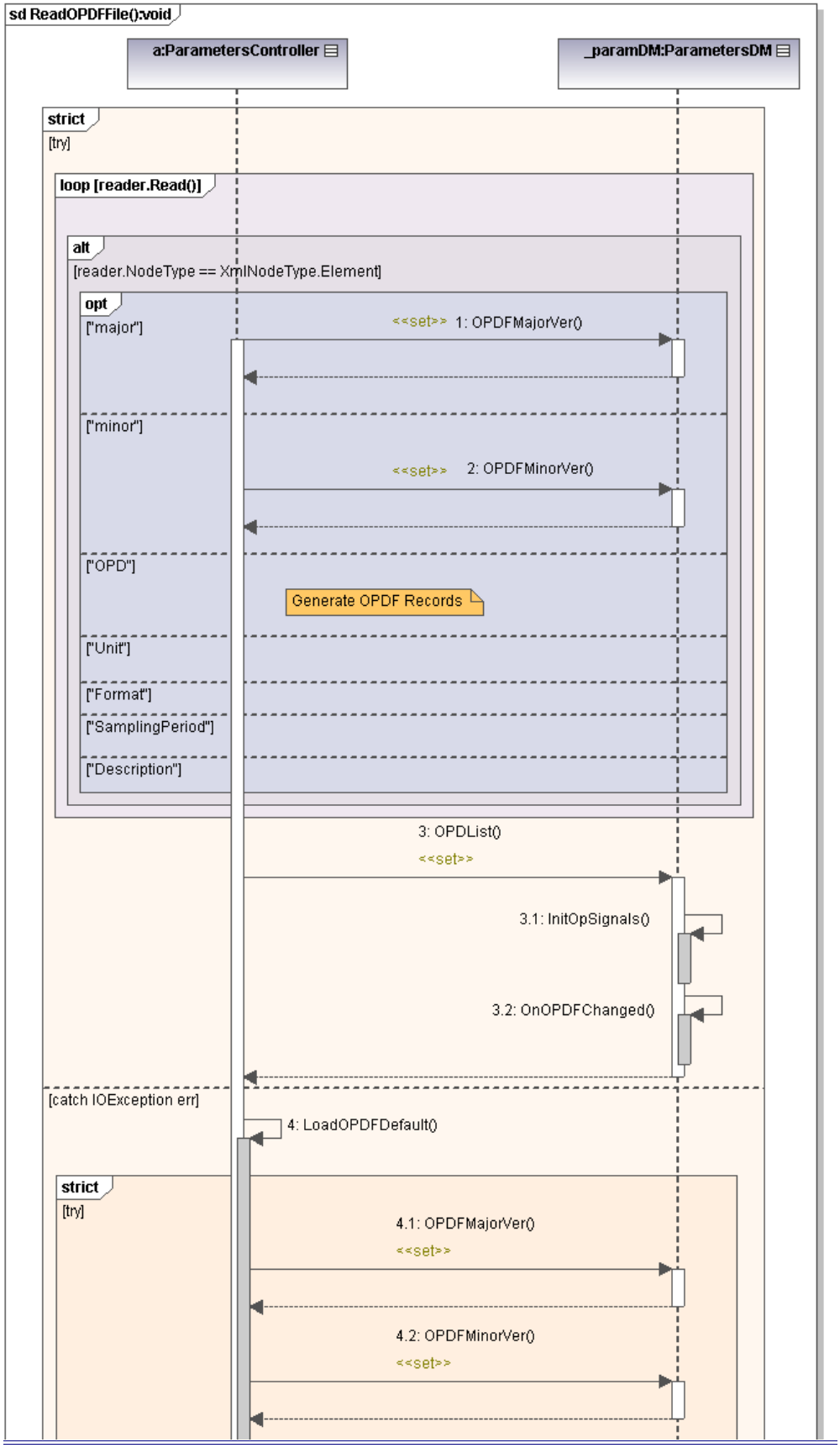
3.7.5.2.1 OPDF Download & Display

The OPDF download and display behaves in the same way as other file transfers. For more details on file transfers, see section 3.7.2.2.3.

The OPDF Download is always preceded with an EDF Download. Please view Figure 3-56 for more details.

3.7.5.2.2 OPDF Read & Write

The OPDF read and write sequence behaves exactly like the EDF Read and Write. Refer to Section 3.7.4.2.2 for more details. Figure 3-64 shows the Sequence Diagram of the ReadOPDFFile() function.


sd ReadOPDFFile():void
a:ParametersController
_paramDM:ParametersDM
strict
[try]
loop [reader.Read()]
alt
[reader.NodeType == XmlNodeType.Element]
opt
["major"]
<<set>> 1: OPDFMajorVer()
["minor"]
<<set>> 2: OPDFMinorVer()
["OPD"]
Generate OPDF Records
["Unit"]
["Format"]
["SamplingPeriod"]
["Description"]
3: OPDList()
<<set>>
3.1: InitOpSignals()
3.2: OnOPDFChanged()
[catch IOException err]
4: LoadOPDFDefault()
strict
[try]
4.1: OPDFMajorVer()
<<set>>
4.2: OPDFMinorVer()
<<set>>

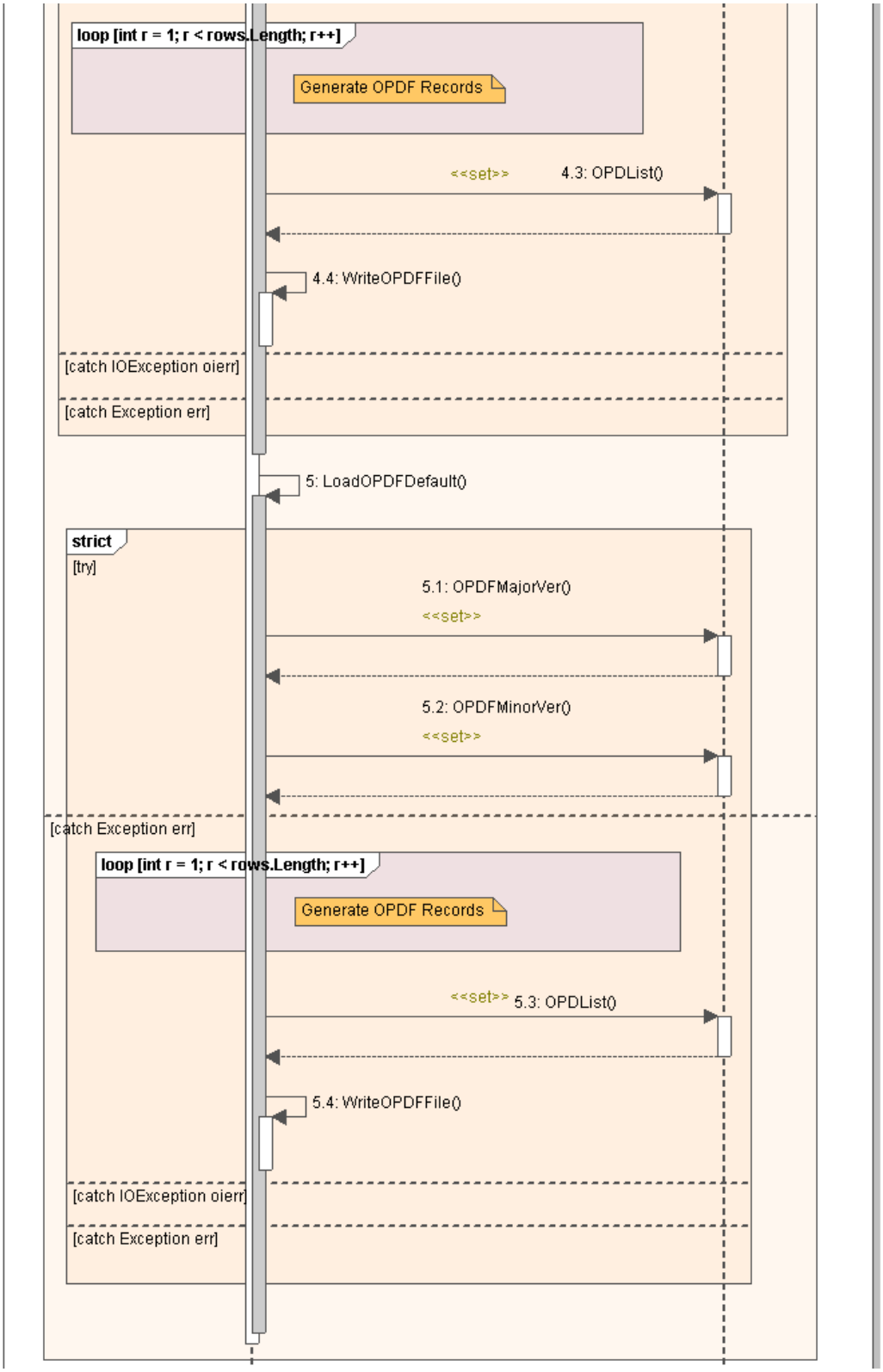


Figure 3-64 Sequence Diagram - ReadOPDFFile()

3.7.5.2.3 Poll Data & Display

The data polling behaves differently than other commands as the actual request is not initiated by a GUI. This is because two GUIs can poll for Data request and to minimize traffic, the polling has been centralized in the Controller. Instead, each GUI "registers" itself to the controller to indicate that it wishes to poll. If one or more GUIs is requesting to poll, then the controller takes care of the polling. The rest (notification and update of GUIs) behaves in the same fashion.

Figure 3-65 shows how the start polling is initiated. StopPolling() behaves in the same way as NeedsToPoll(), except that it unregisters the GUI's name from the list of GUI's which are currenlty 'requesting to poll'. The details of the actual Data Request polling are displayed in Figure 3-66 and Figure 3-67

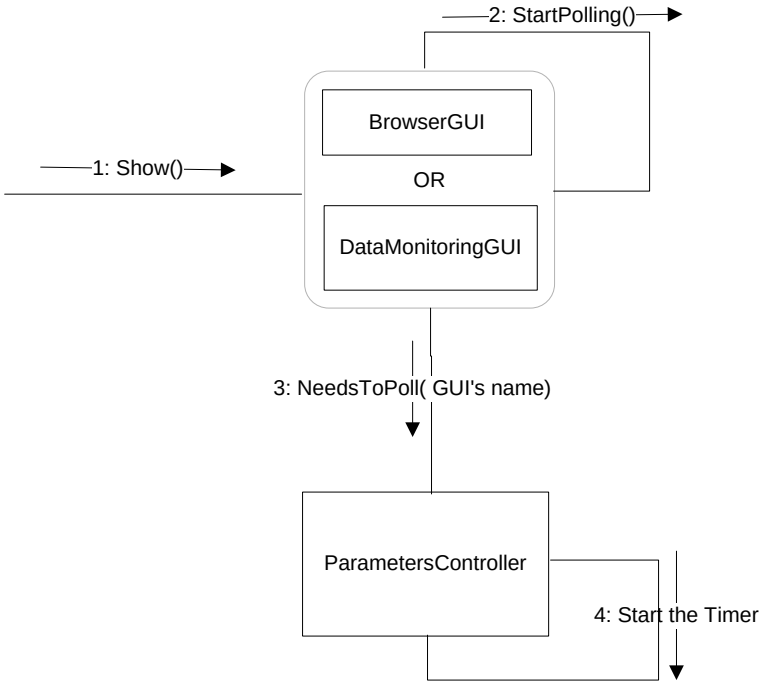


Figure 3-65 Communication Diagram - Start Polling Data

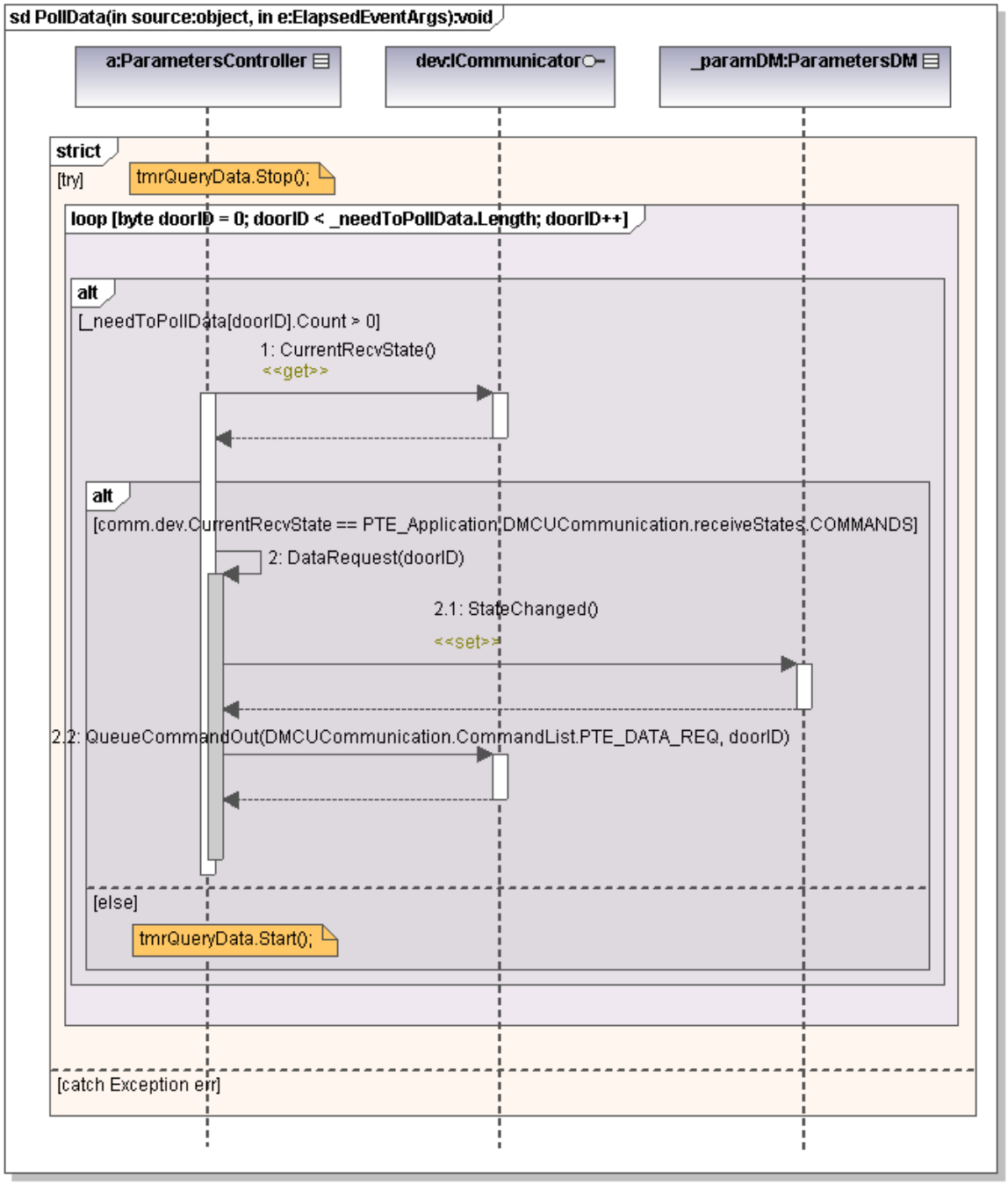


Figure 3-66 Sequence Diagram - Poll Data

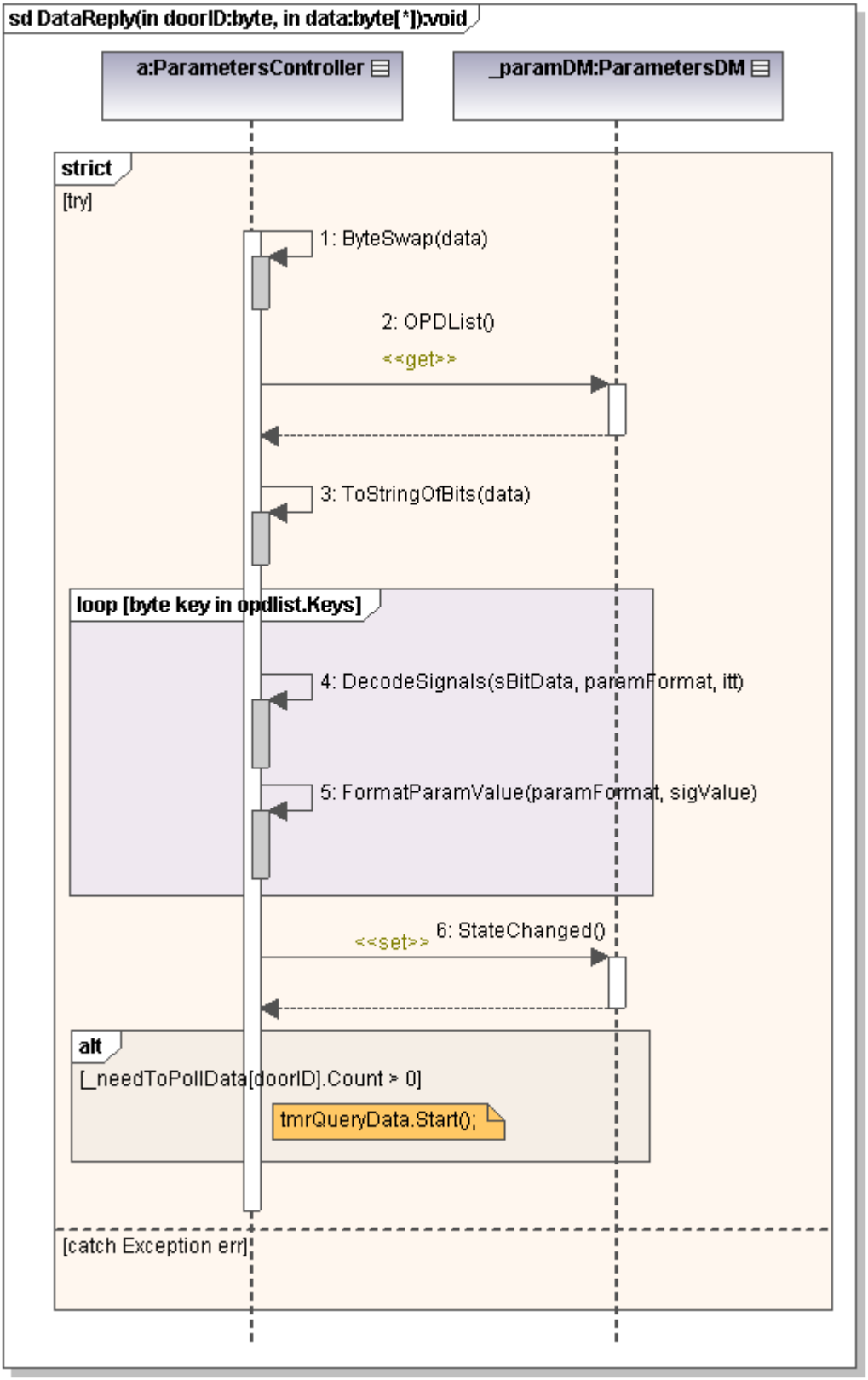


Figure 3-67 Sequence Diagram - Data Reply

3.7.5.2.4 Snapshot Download & Display

The Snapshot Download and Display behaves in the same way as other file transfers. For more details on file transfers, see section 3.7.2.2.3.

Upon reception of the user download DCU snapshot command, the DCU downloads all snapshots residing in the DCU. After the snapshots are downloaded from the DCU, the snapshot file header is listed on the snapshot file table. The snapshot file data are checked before being sent to the display. The Snapshot Download flowchart is shown in Figure 3-68. Figure 3-69 presents the sequence diagram of the SnapshotDownloadReply function.

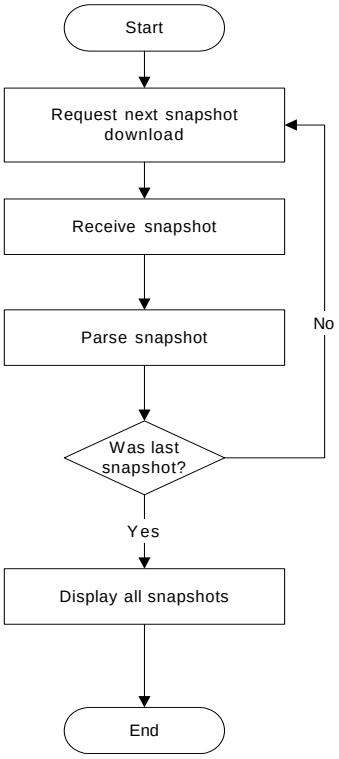


Figure 3-68 Flowchart - Snapshot Download

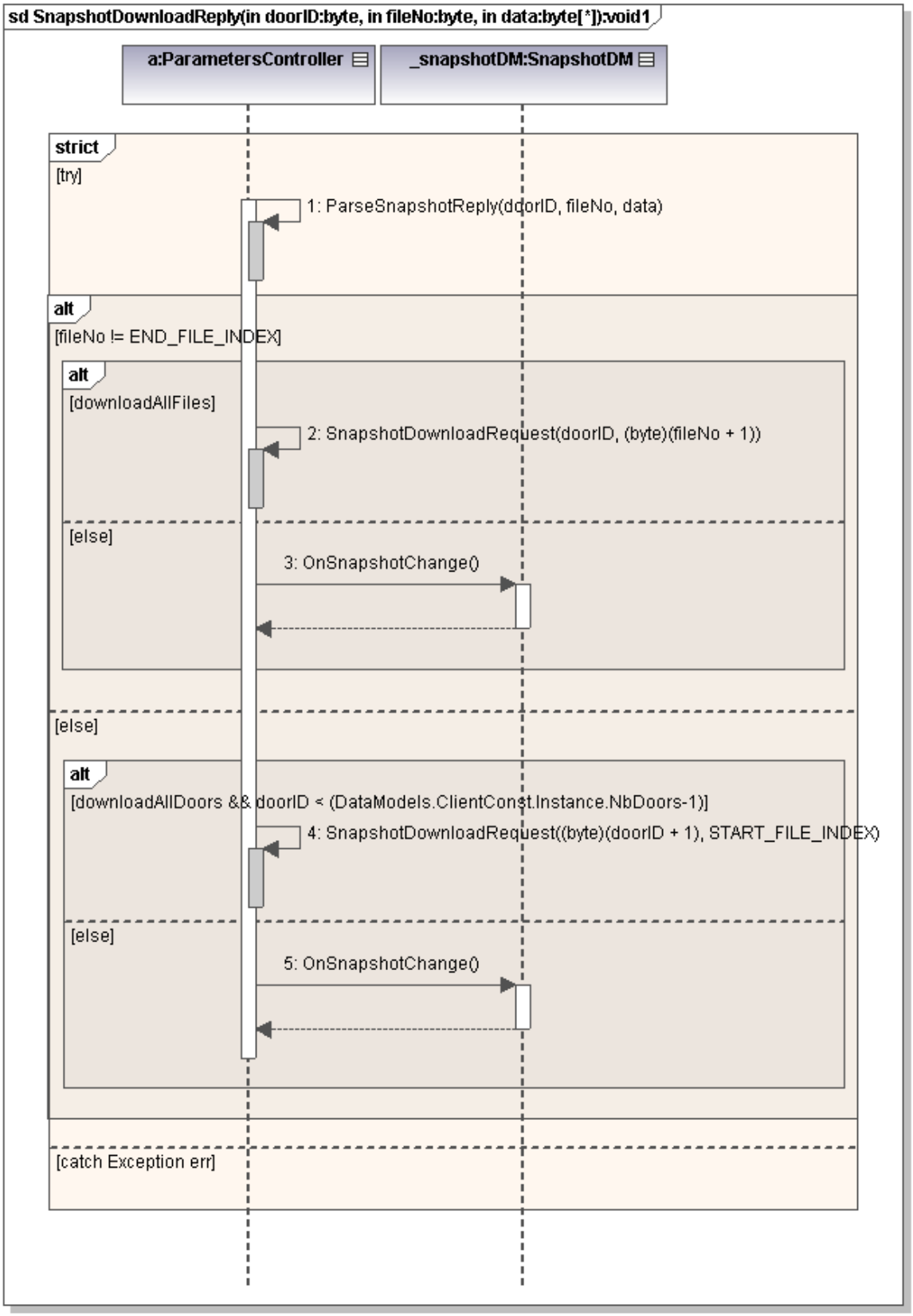


Figure 3-69 Sequence Diagram - SnapshotDownloadReply

3.7.5.2.5 Snapshot Erasure

The snapshot erasure request is sent out for each of the snapshot selected in the GUI. This means that the cycle describes in the communication diagram will be repeated for all those snapshots. The Communication Diagram and Flowchart of the Snapshot Erasure functions are respectively presented in Figure 3-70 and Figure 3-71.

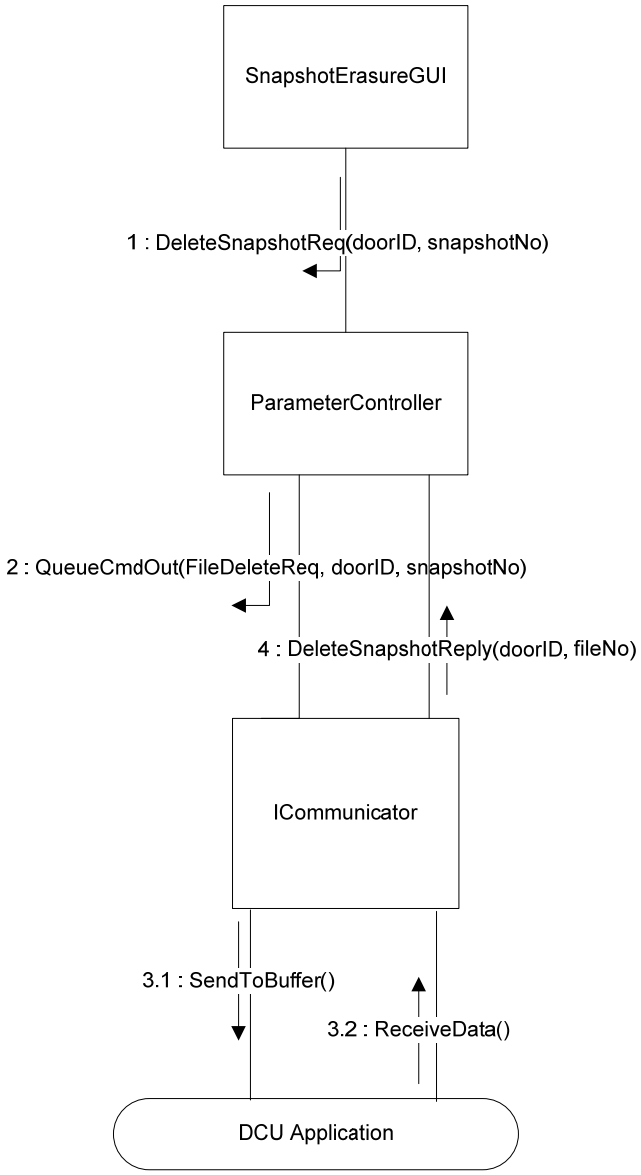


Figure 3-70 Communication Diagram - Snapshot Erasure

The DCU user is able to request the DCU to erase Snapshots when a password has been entered. When the DCU receives that request, it prompts the user to confirm the delete action. When the user confirms the delete action, the DCU sends a command to delete each snapshot sequentially.

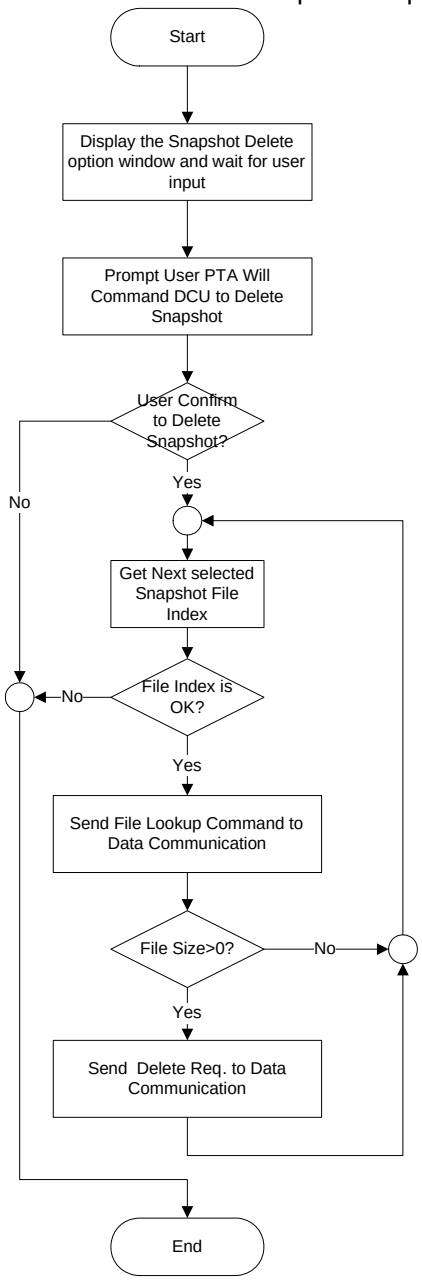


Figure 3-71 Flow Chart – Snapshot Erasure

3.7.6 Self-Test Module

The Self-Test Module is composed of three main entities that operate in an MVC fashion. Figure 3-72 presents the Self-Test module classes.

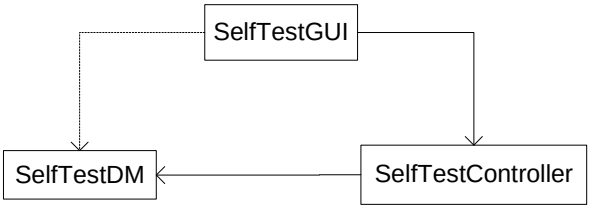


Figure 3-72 Self-Test Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

The DCU independently performs a self-test on its components. The PTA allows the user to retrieve and display the results of those periodic self-tests. The following devices are tested during the self-test procedure:

- DCU Application CRC32
- DCU Boot Loader Application CRC32
- SRAM Pattern
- SRAM Address Line
- NVRAM Pattern
- NVRAM Address/Data Line
- Network Link

3.7.6.1 Interfaces

As per the global architecture described in Section 3.5, the GUI in this module interacts with the PTA Main module by being a child GUI of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.6.2 Functionalities

3.7.6.2.1 Self-Test Result Retrieval & Display

The DCU self-test results are sent to the Portable Test Application by the DCU upon request. The results are parsed and displayed. Figure 3-73 and Figure 3-74 present the Flowchart and Communication Diagram of the Self-Test functionality.

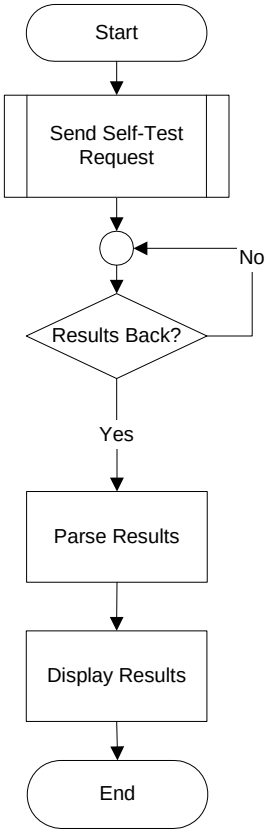


Figure 3-73 Flow Chart - Self-Test

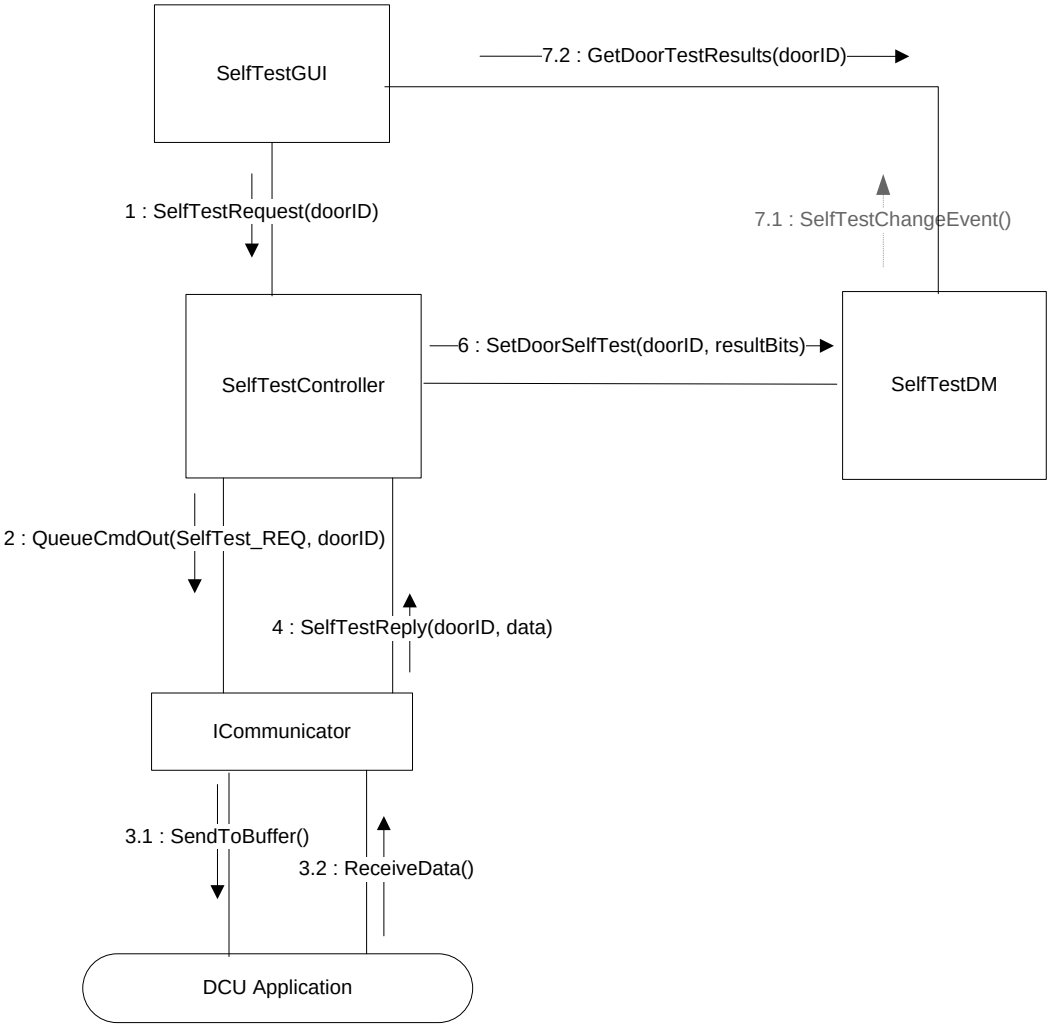


Figure 3-74 Communication Diagram - Self-Test

3.7.7 Memory Dump Module

The Memory Dump Module is composed of three main entities that operate in an MVC fashion. Figure 3-75 shows the memory Dump Module Classes

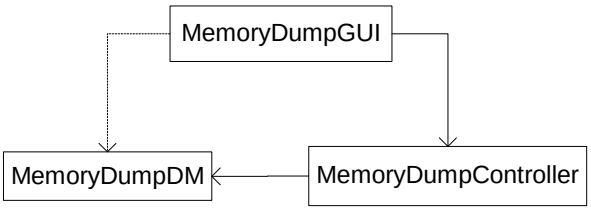


Figure 3-75 Memory Dump Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

The Memory Dump allows the DCU user to view the DCU RAM contents. The DCU SRAM is used to store temporary volatile data. The DCU NVRAM is used to store Log files, snapshot files, error messages, and configurable data. The DCU Application transfers the RAM memory content in one file transfer that contains both SRAM and NVRAM data.

3.7.7.1 Interfaces

As per the global architecture described in Section 3.5, the GUI in this module interacts with the PTA Main module by being a child GUI of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.7.2 Functionalities

3.7.7.2.1 Memory Dump Download & Display

The Memory Dump file must first be downloaded before being displayed. After the download, the bytes are simply displayed in the GUI with their ASCII equivalent. Figure 3-76 and Figure 3-77 respectively present the Flowchart and Communication Diagram of the Memory Dump Download functionality.

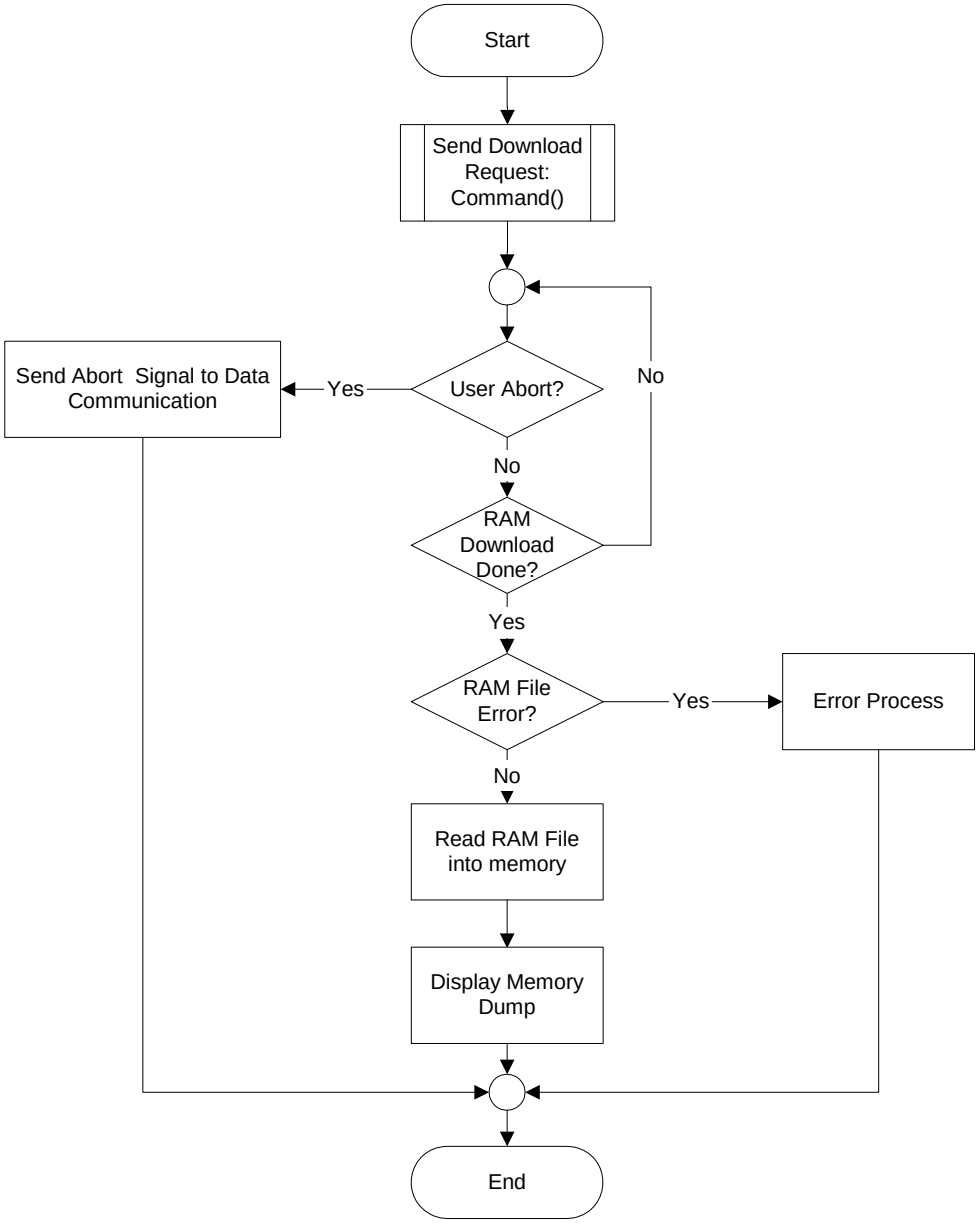


Figure 3-76 Flow Chart - Memory Dump

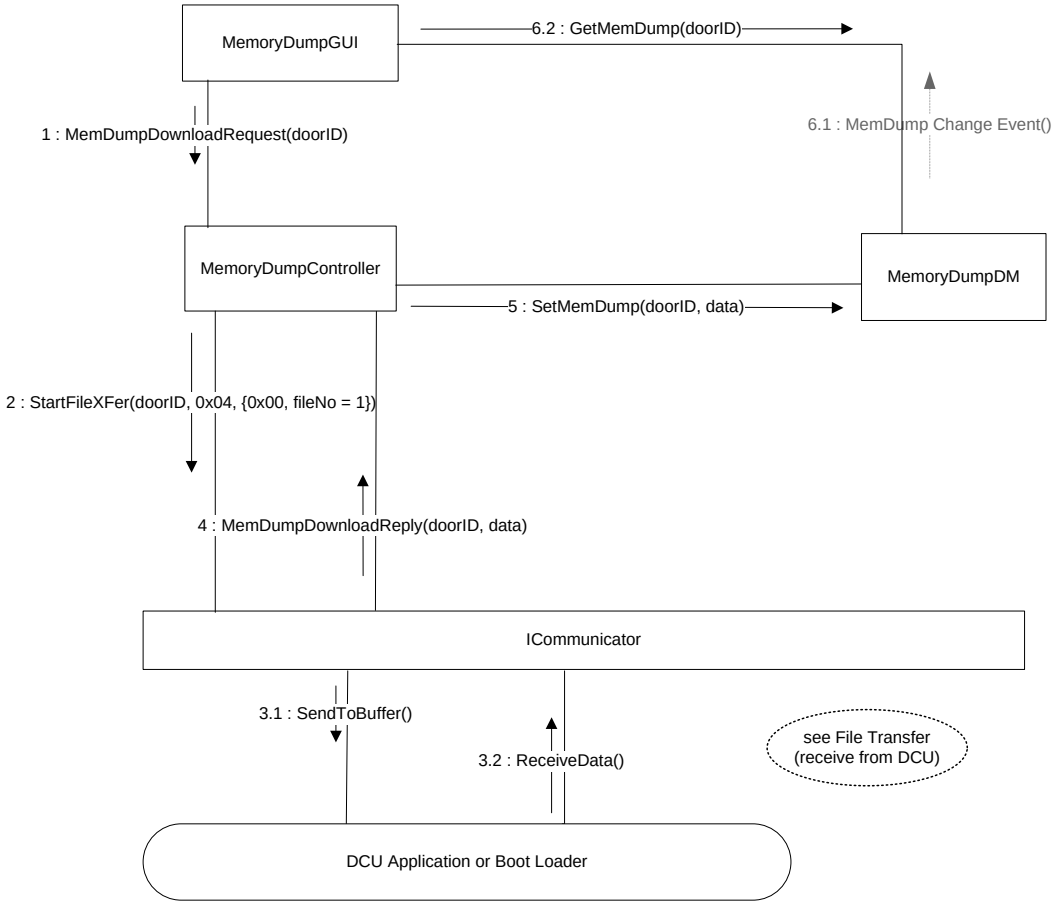


Figure 3-77 Communication Diagram - Memory Dump

3.7.8 Graph Module

The Graph Module is composed of three main entities that operate in an MVC fashion. Figure 3-78 shows the Graph Module Classes.

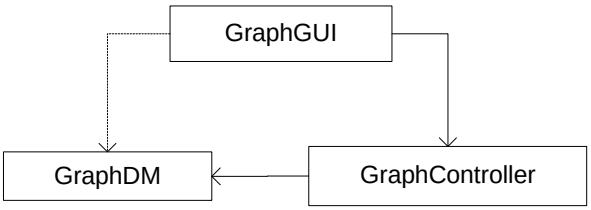


Figure 3-78 Graph Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

3.7.8.1 Interfaces

As per the global architecture described in Section 3.5, the GUI in this module interacts with the PTA Main module by being a child GUI of the MainGUI. The Controller in this module interacts with the Communicator Module.

The Graph GUI also interfaces with the NPlot library, an external library that facilitates the drawing of plots and graphics.

3.7.8.2 Functionalities

3.7.8.2.1 Graph Download & Display

The Graph Module is used to download and display the door movement information graph. Upon reception of the door graph request, the DCU starts to download two files consecutively: the Door Open Graph then the Door Close Graph.

When both files are downloaded, the DCU draws plots for the PWM, Current, Error, Speed and Door Position. Each file has 2 graphs, for a total of 4 graphs. Figure 3-79 and Figure 3-80 respectively present the Flowchart and Communication Diagram of the Graph Download and Display functionality.

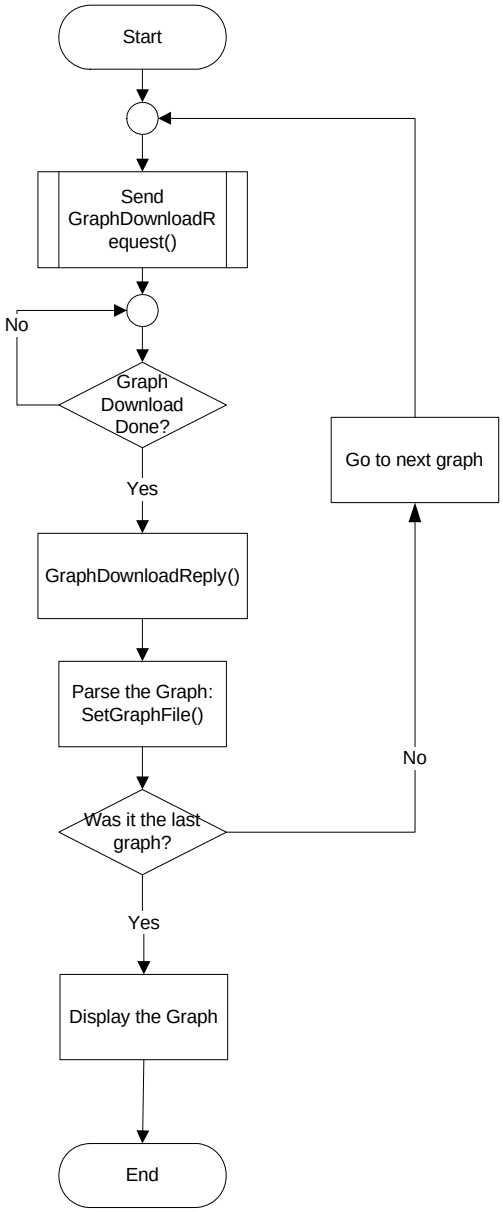


Figure 3-79 Flow Chart - Graph Download

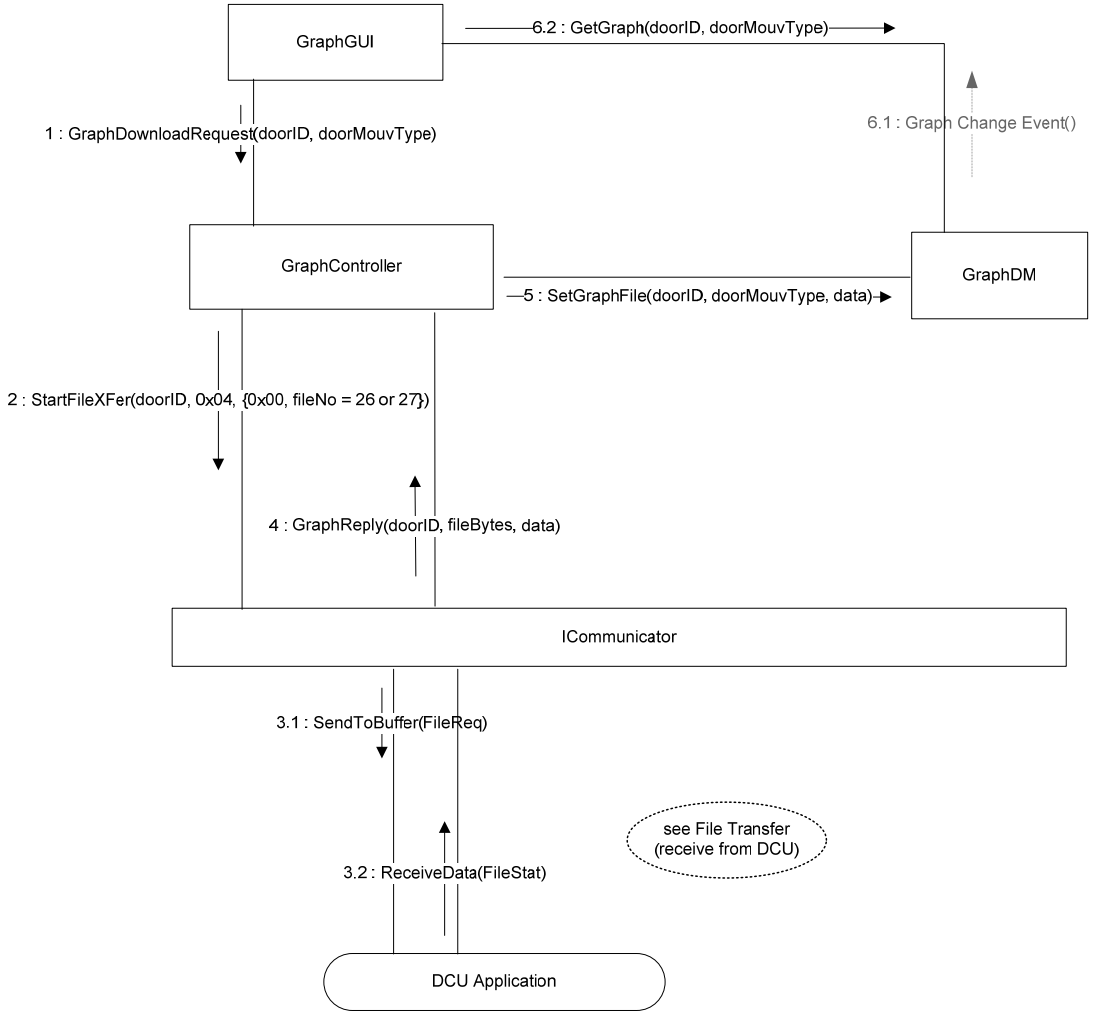


Figure 3-80 Communication Diagram - Graph Download

3.7.9 DCU Configuration Module

The DCU Configuration Module is composed of three main entities that operate in an MVC fashion. Figure 3-81 presents the DCU Configuration Module Classes.

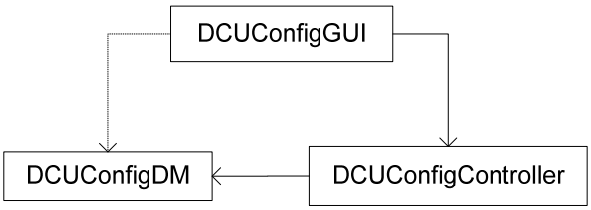


Figure 3-81 DCU Configuration Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

All the configurabe parameters listed hereafter are described in the Appendix A and their values are defined in the SACS project's specific document.

The DCU Configuration is used to retrieve and modify the DCU parameters:

- SACS.Door.DefaultOpeningDelay
- SACS.Door.DefaultOpeningTime
- SACS.Door.OpeningTime.Range
- SACS.Door.DefaultClosingDelay
- SACS.Door.DefaultClosingTime
- SACS.Door.ClosingTime.Range
- SACS.Door.CrewSwitch.Outside.ClosingDelay
- SACS.Door.CrewSwitch.Inside.ClosingDelay
- SACS.Door.CrewSwitch.Outside.ClosingTime
- SACS.Door.CrewSwitch.Inside.ClosingTime
- SACS.Door.CrewSwitch.Outside.OpeningDelay
- SACS.Door.CrewSwitch.Inside.OpeningDelay
- SACS.Door.CrewSwitch.Outside.OpeningTime
- SACS.Door.CrewSwitch.Inside.OpeningTime
- SACS.Door.CrewSwitch.LinkTiming
- SACS.Door.Obstruction.Distance
- SACS.Door.Obstruction.Count
- SACS.Door.Obstruction.EndBehavior
- SACS.Door.Obstruction.Range
- SACS.Door.AudibleAlarmActivation
- SACS.Door.VisualAlarmActivation
- SACS.Door.SecondaryObstruction.Position
- SACS.Door.SecondaryObstruction.Activation
- SACS.Door.SecondaryObstruction.Delay
- SACS.Door.EmergencyHandle.ClosePower
- SACS.Door.Snapshot.Period
- SACS.Door.Snapshot.Capacity
- SACS.Door.Snapshot.NbOfFiles

The function allows DCU parameters modification only if a valid password was entered.

3.7.9.1 Interfaces

As per the global architecture described in Section 3.5, the GUI in this module interacts with the PTA Main module by being a child GUI of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.9.2 Functionalities

3.7.9.2.1 DCU Configuration Retrieval

For each configuration parameter, a command is sent out. When one parameter is received, the next request is sent automatically until all parameters are received. Figure 3-82 and Figure 3-83 respectively present the Flowchart and Communication Diagram of the Dcu Configuration Retrieval functionality.

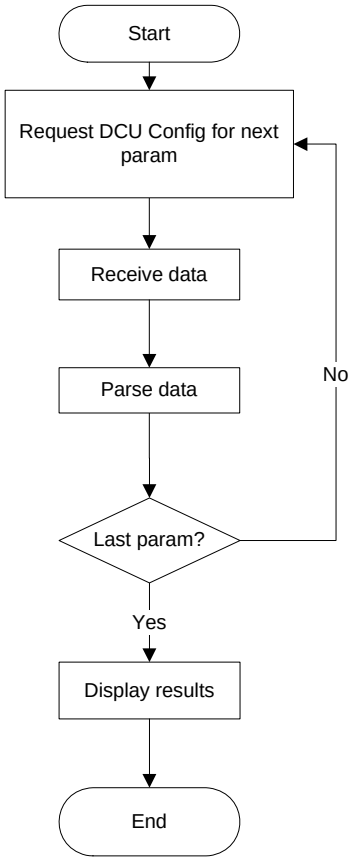


Figure 3-82 Flow Chart - DCU Configuration Retrieval

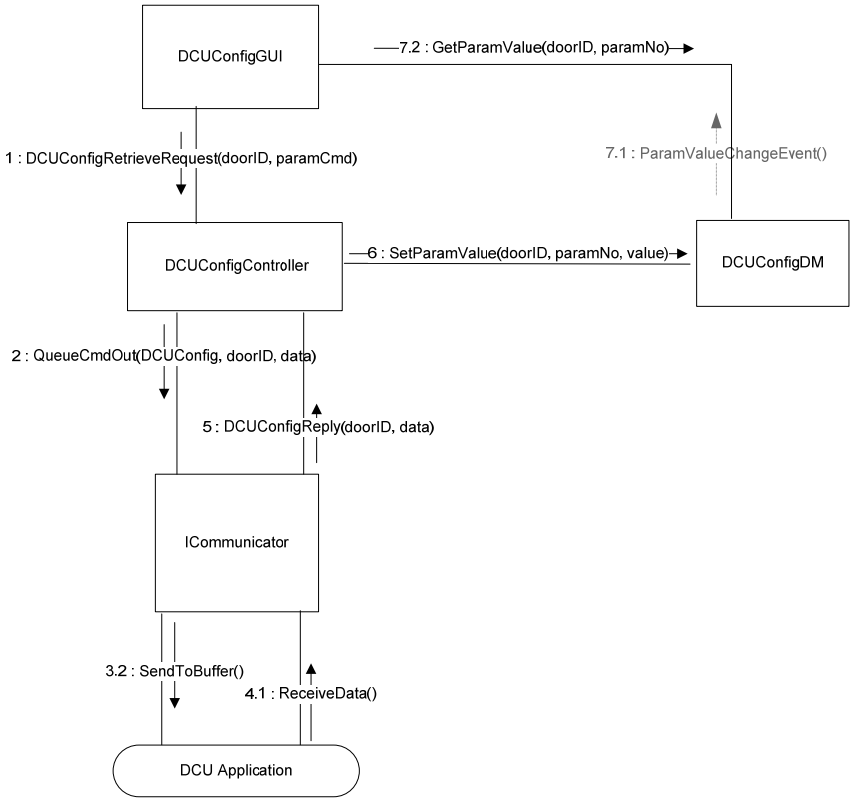


Figure 3-83 Communication Diagram - Configuration Retrieval

3.7.9.2.2 DCU Configuration Change

Upon reception of the user request of the DCU configuration, the DCU reads all the parameter values from the DCU and displays them in the configuration window. After the update of the parameters with the new values, the parameter values are read back from the DCU and displayed on the configuration window to confirm the modifications. Figure 3-84 and Figure 3-85 respectively present the Flowchart and Communication Diagram of the DCU Configuration Change functionality

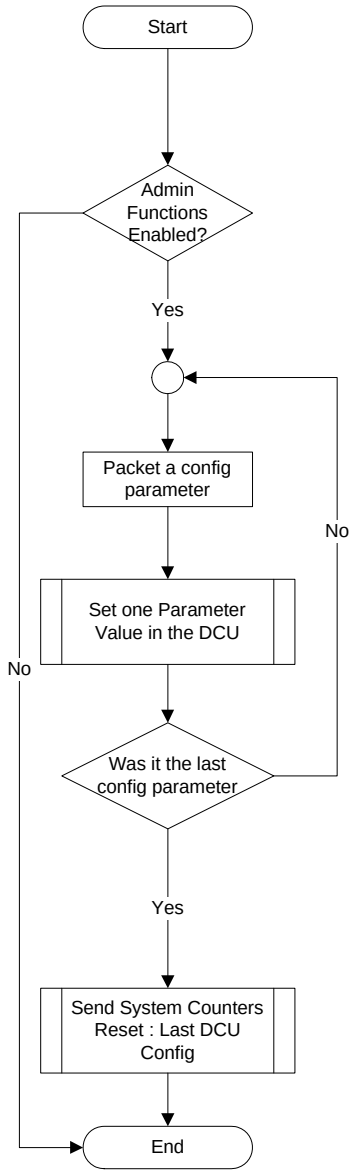


Figure 3-84 Flow Chart – DCU Configuration Change

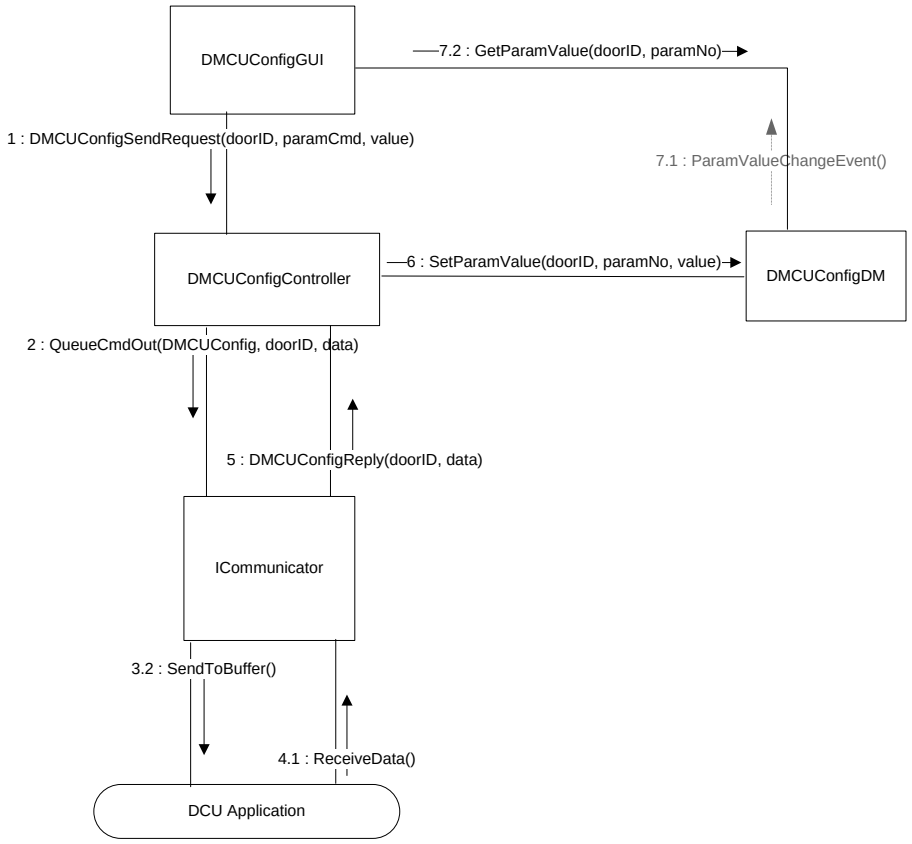


Figure 3-85 Communication Diagram – DCU Configuration Change

3.7.10 DCU Time Module

The DCU Time Module is composed of three main entities that operate in an MVC fashion. The DCU Time Module Classes are presented in Figure 3-86.

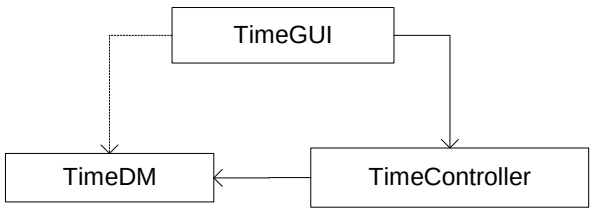


Figure 3-86 DCU Time Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

The DCU Time Module allows the user to retrieve and set the time in the DCU. The time sent to the DCU will be the current time on the Operating System of the computer the PTA is running on. The DCU Time converts time_data to date and time. The time_data received from the DCU is local time in “C” extended (unsigned 32-bit) and it is the number of seconds elapsed since Jan. 1, 1970. The date and time are displayed in DCU Time window in the local format is defined by the Microsoft Windows operating system local configuration.

3.7.10.1 Interfaces

As per the global architecture described in Section 3.5, the GUI in this module interacts with the PTA Main module by being a child GUI of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.10.2 Functionalities

3.7.10.2.1 DCU Time Retrieval

The DCU Time Module starts the retrieval process by opening the TimeGUI or by a request from the user by pressing a button. Figure 3-87 and Figure 3-88 respectively present the Flowchart and Communication Diagram of the DCU Time Retrieval functionality.

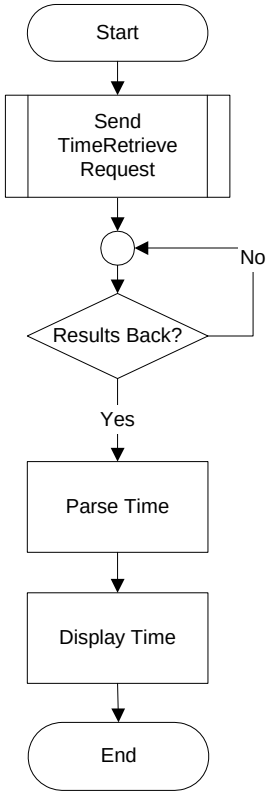


Figure 3-87 Flow Chart - DCU Time Retrieval

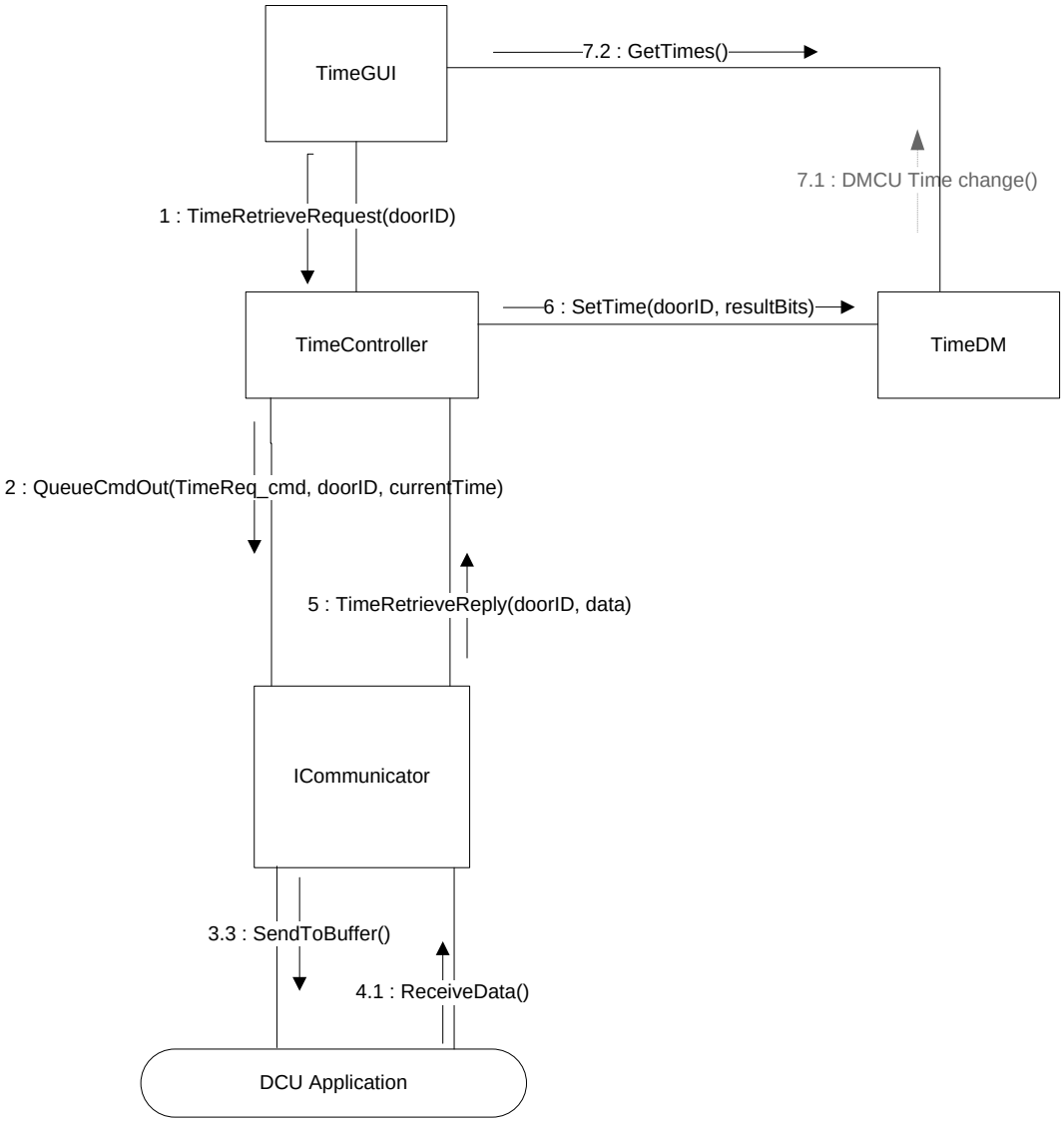


Figure 3-88 Communication Diagram - DCU Time Retrieval

3.7.10.2.2 DCU Time Set

The set time sequence is very similar to the time retrieve sequence. After the time set, a retrieve response is still received from the DCU Application, updating the time displayed in the GUI. Figure 3-89 andFigure 3-90 respectively present the Flowchart and Communication Diagram of the DCU Time Set functionality

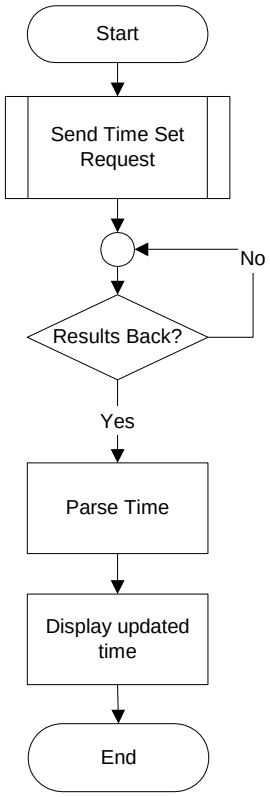


Figure 3-89 Flow Chart - DCU Time Set

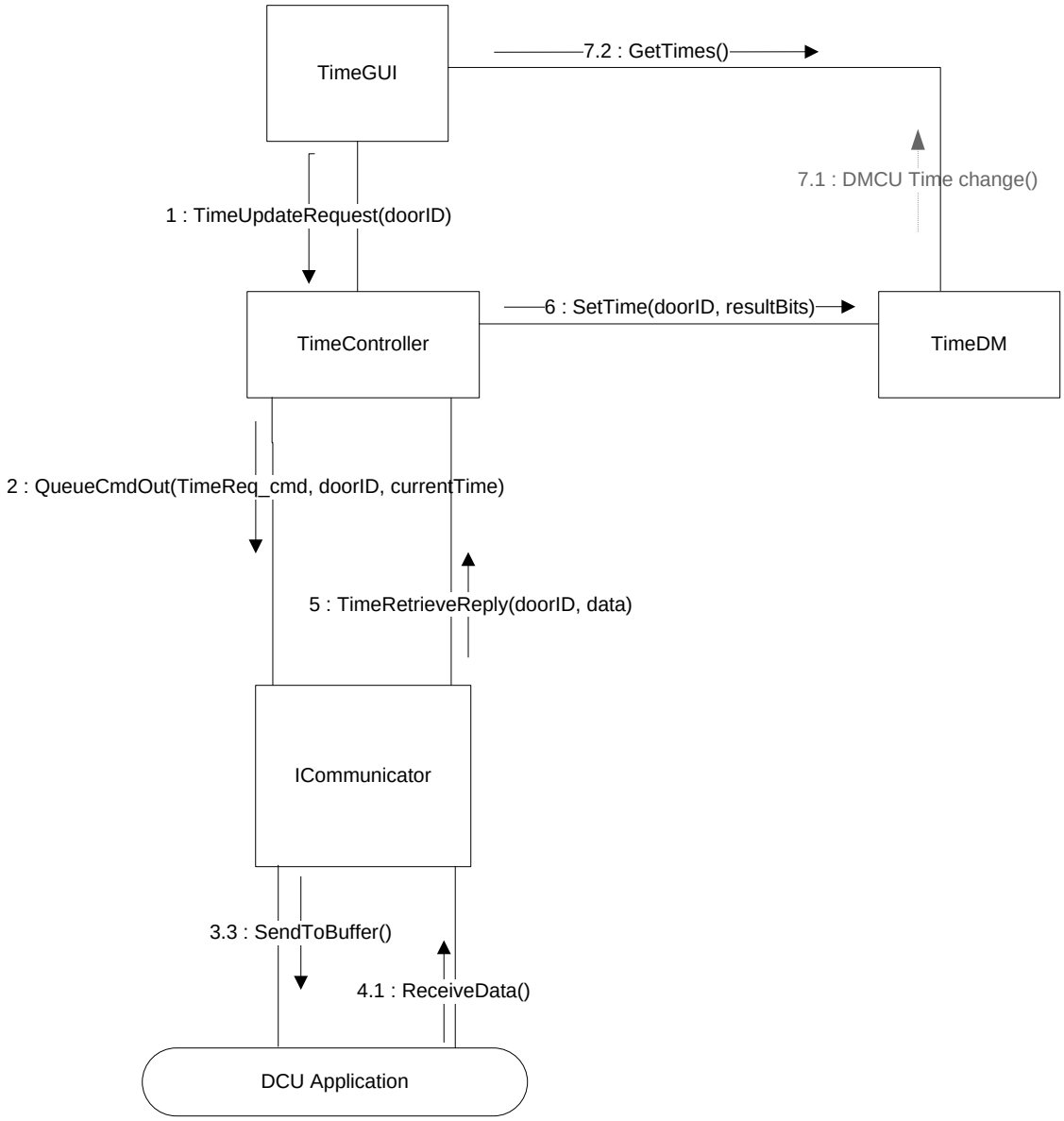


Figure 3-90 Communication Diagram - DCU Time Set

3.7.11 SW Versions Module

The SW Version Module is composed of five main entities that operate in an MVC fashion. In addition, two other GUIs have access to the SW Version Controller to view the Data Model entries. The Self-Test Module Classes are presented in the Figure 3-91.

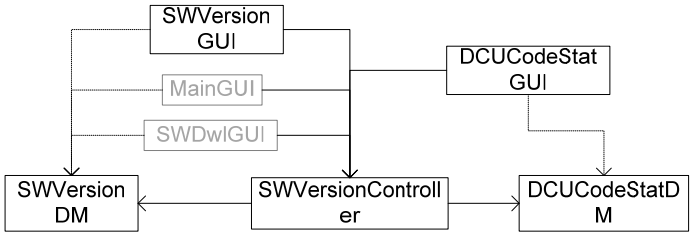


Figure 3-91 Self-Test Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

The SW Version Module is very important in the PTA. It helps validate what is on the DCU and allows the user to verify that file updates are done correctly. The features are accessible when logged into the Boot Loader or the DCU Application. The DCU will only return the SCI versions loaded. Because of this, when connected to the Boot Loader, some SCIs and the EDF or OPDF version may appear to be missing.

3.7.11.1 Interfaces

As per the global architecture described in Section 3.5, the GUIs in this module interact with the PTA Main module by being child GUIs of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.11.2 Functionalities

3.7.11.2.1 SCI Version

This function parses the version_data to get the following versions:

- DCU Boot Loader Application Version
- DCU Application Version
- CPLD version (if applicable to project)
- Configuration File version (described in SACS document)

The versions of DCU Boot Loader Application and DCU Application are displayed in the SWVersionGUI.

Refer to section 3.7.1.2.5 for the sequence in which SW Version is requested. Figure 3-92 and Figure 3-93 respectively present the Flowchart and Communication Diagram of the SCI Version functionality.

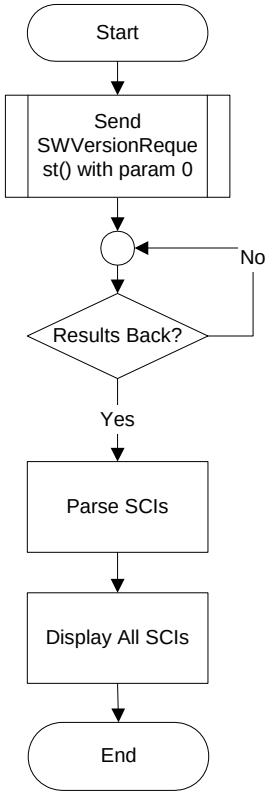


Figure 3-92 Flow Chart - SCI Version

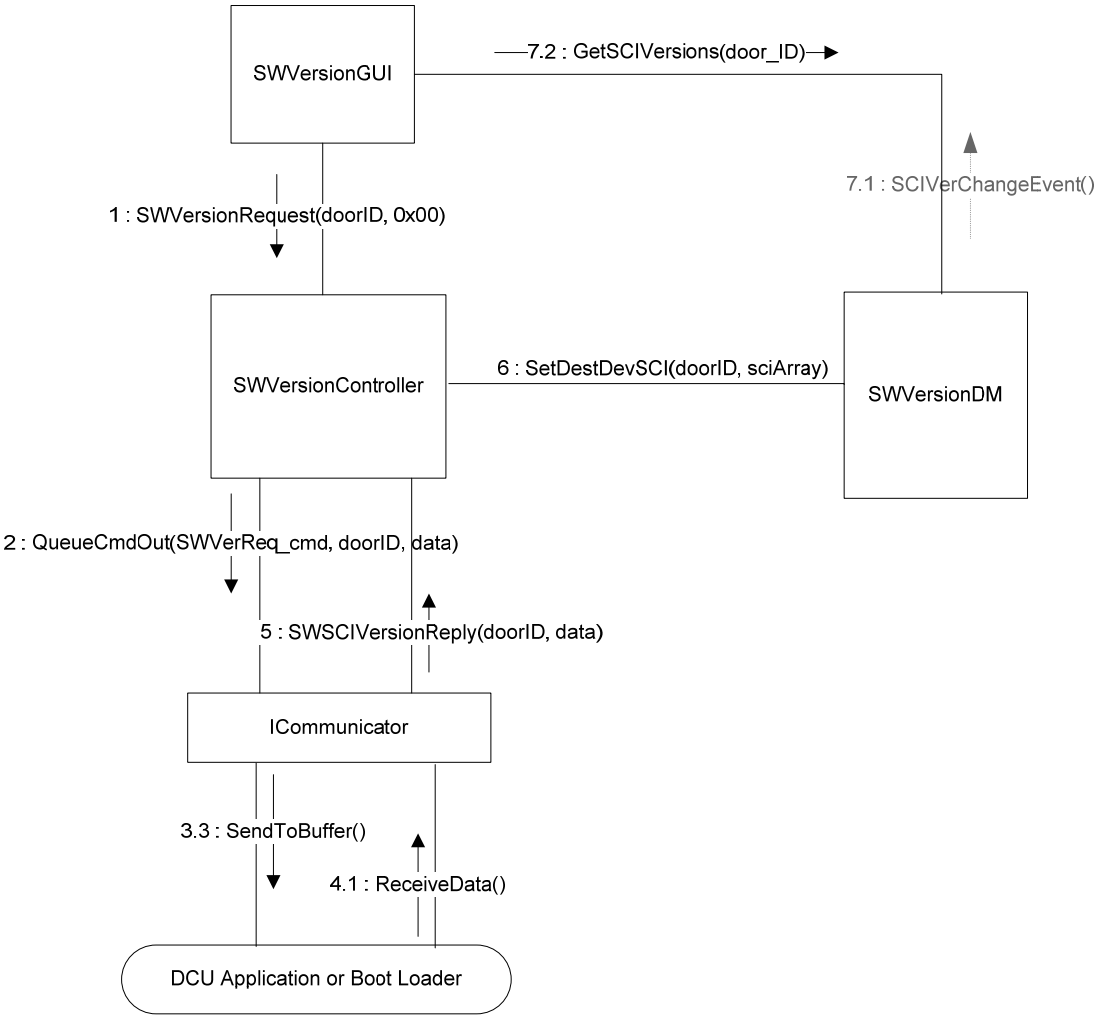


Figure 3-93 Communication Diagram - SCI Version

3.7.11.2.2 SW Version (EDF & OPDF)

This function is used to retrieve the versions of the EDF and OPDF stored on the DCU. These are not to be confused with the version of the EDF and OPDF stored on the PTA as they may be different.

These versions are used primarily to verify if the PTA is out of date regarding these files, and if it must trigger an update.

If the EDF version is different than the one in the DCU Application data folder, it automatically sends EDF request to the Data Communication module. This functionality is addressed in section 3.7.4.2.1.
If OPDF version is different than the one in the DCU Application, it automatically sends an OPDF request to the Data Communication module. This functionality is addressed in section 3.7.5.2.1.

Figure 3-94 and Figure 3-95 respectively present the Flowchart and Communication Diagram of the Software Version (EDF and OPDF) functionality.

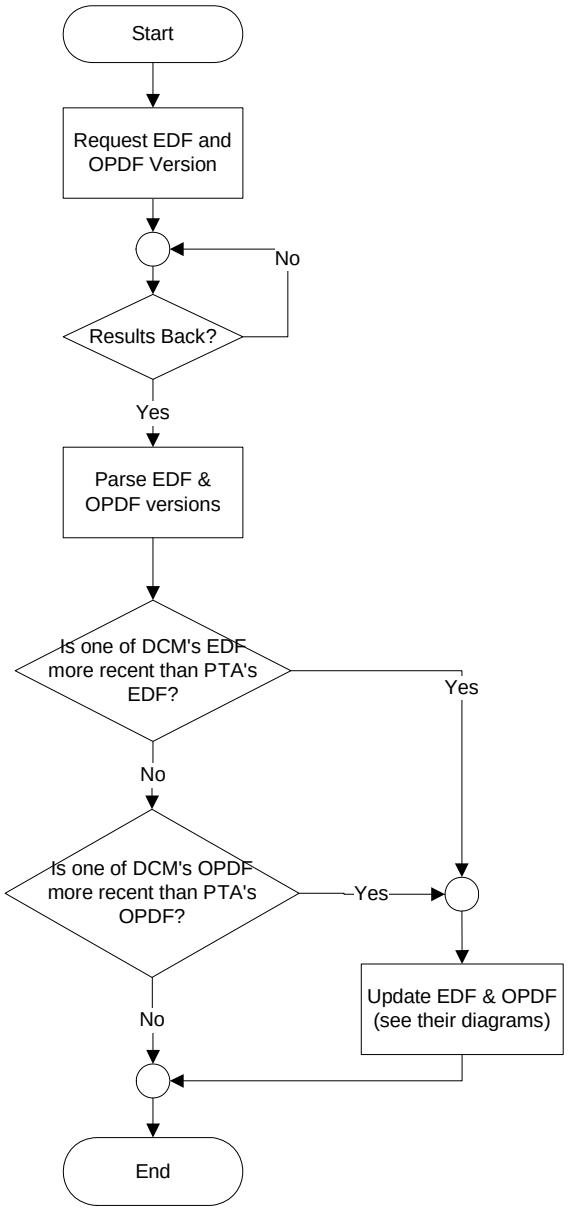


Figure 3-94 Flow Chart - SW Version (EDF & OPDF)

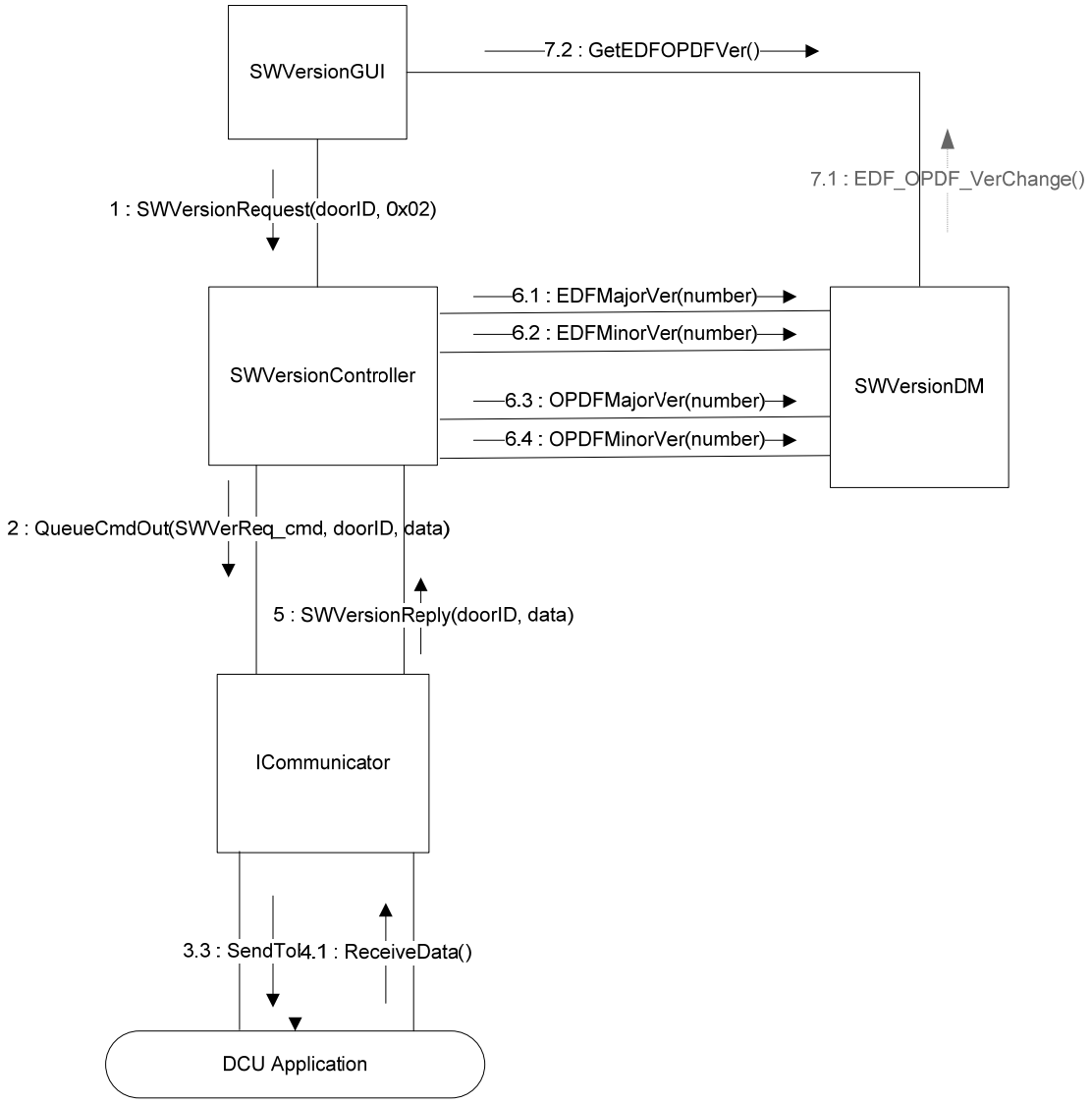


Figure 3-95 Communication Diagram - SW Version (EDF & OPDF)

3.7.11.2.3 DCU Code Status

The DCU Code status is similar to the SCI Version in the sense that it retrieves information on the SCI's loaded in the DCU. The information displayed is the SCI file size and CRC. Figure 3-96 and Figure 3-97 respectively present the Flowchart and Communication Diagram of the DCU Code Status functionality.

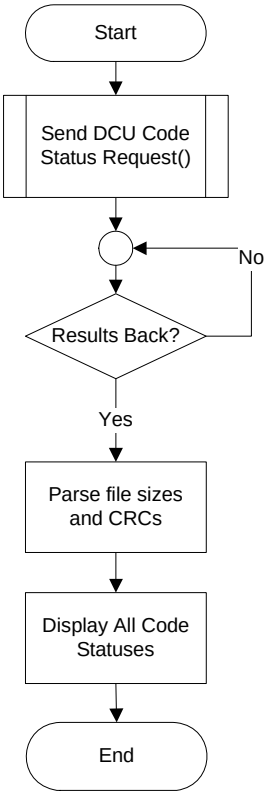


Figure 3-96 Flow Chart - DCU Code Status

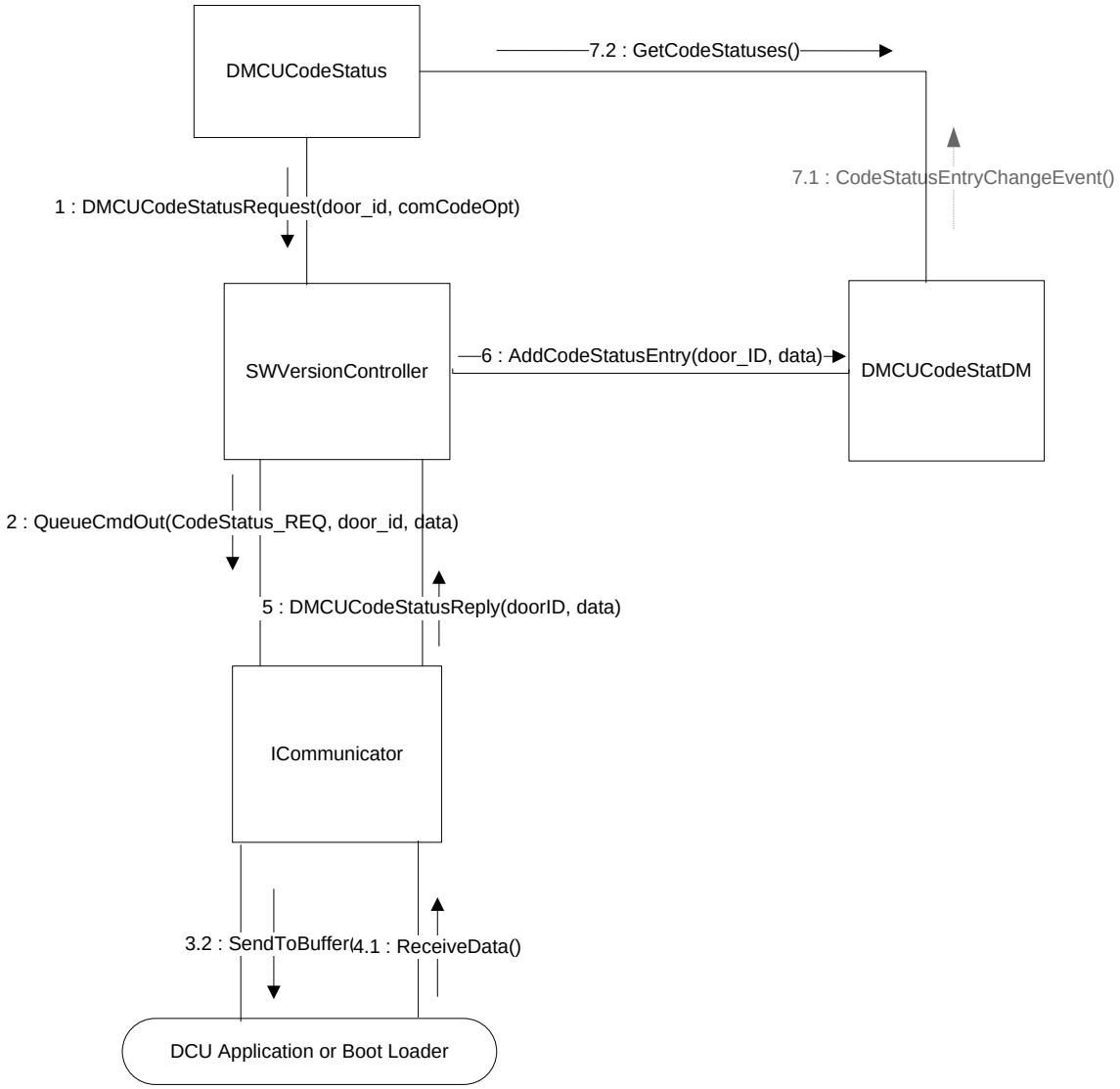


Figure 3-97 Communication Diagram - DCU Code Status

3.7.12 System Counters Module

The System Counters Module is composed of three main entities that operate in an MVC fashion. In addition, three other GUIs have access to the System Counters Controller to send specific resets. System Counters Module Classes are shown in Figure 3-98.

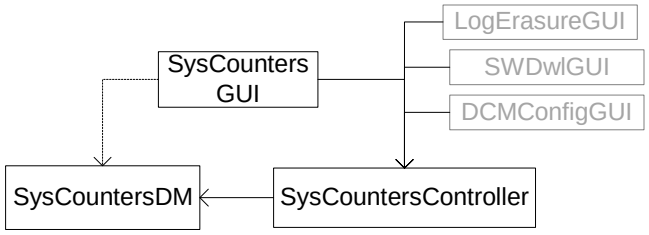


Figure 3-98 System Counters Module Classes

The relation between these classes has already been explained in the MVC pattern and section 3.5.

System Counters are used to keep statistical information on changes in the DCU. They keep information such as a count of an event, time it last occurred, last count and the previous time it occurred.

System Counters is a downloaded file store on the DCU, parsed then displayed. Part of the system counters is the Event Occurrences which are dependent on the EDF. For best results, it is recommended that the EDF stored on the PTA matches the one on the DCU Application.

Through the SysCountersGUI, the user can request specific resets. These resets are:

- Reset Door Cycles: Reset the door cycles counter in the DCU to zero
- Reset Operating Hours: Reset the operating hours counter in the DCU to zero
- Reset Event Occurrences: Reset all the counters for the number of times the various events in the DCU occurred.

In addition, some resets are sent automatically as a secondary effect of functionality triggered in other GUIs. These resets are:

- Reset Last DCU Configuration Occurrence
- Reset Last SW Download Occurrence
- Reset Last log Erasure Occurrence

3.7.12.1 Interfaces

As per the global architecture described in Section 3.5, the GUIs in this module interact with the PTA Main module by being child GUIs of the MainGUI. The Controller in this module interacts with the Communicator Module.

3.7.12.2 Functionalities

3.7.12.2.1 System Counters Download & Display

REF _Ref306610281 \h Figure 3-99 and Figure 3-100 respectively present the Flowchart and Communication Diagram of the System Counters Download functionality.

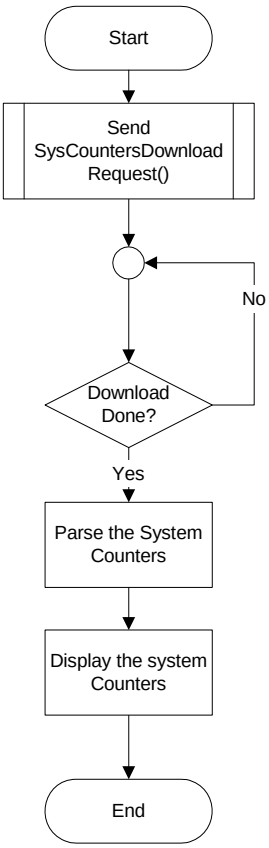


Figure 3-99 Flow Chart - System Counters Download

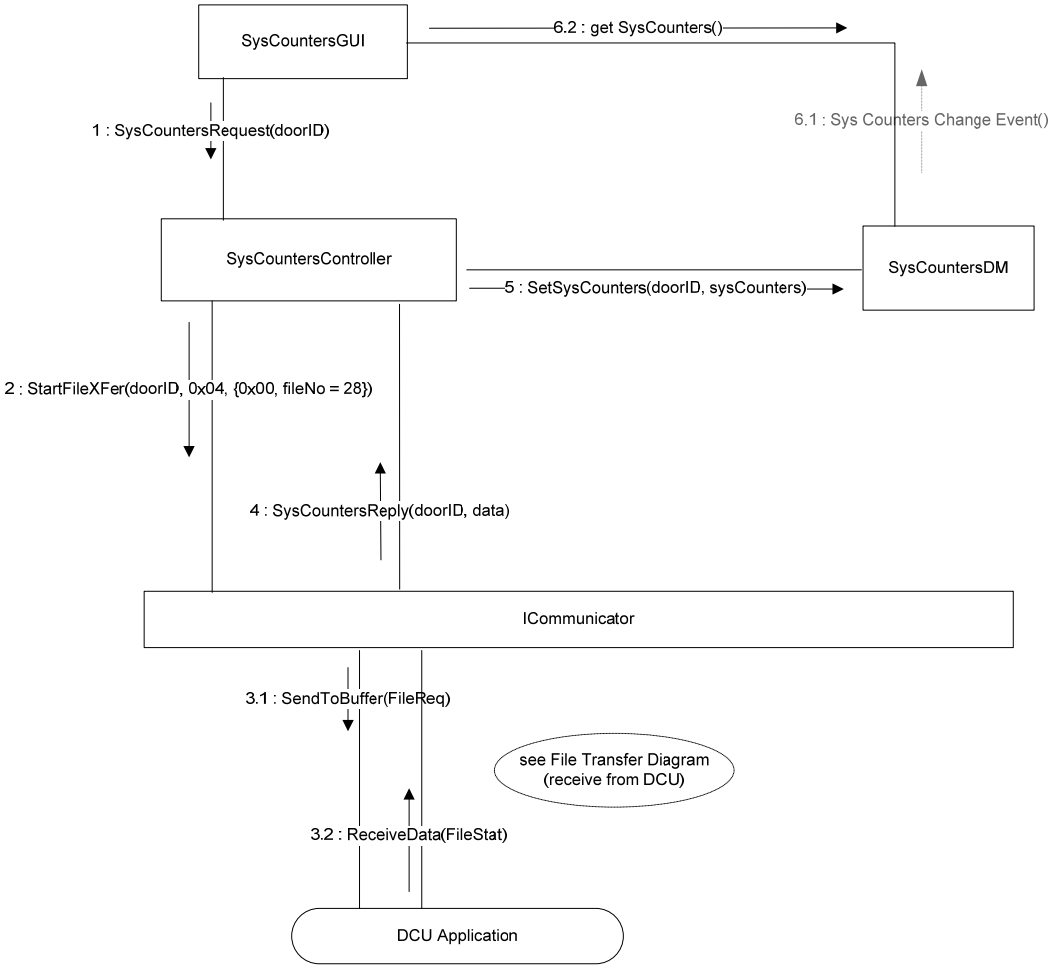


Figure 3-100 Communication Diagram - System Counters

3.7.12.2.2 Reset Door Cycles

The reset of door cycles is initiated by the user through the SysCountersGUI.
Figure 3-101 presents the Communication Diagram of the Reset Door Cycle functionality.

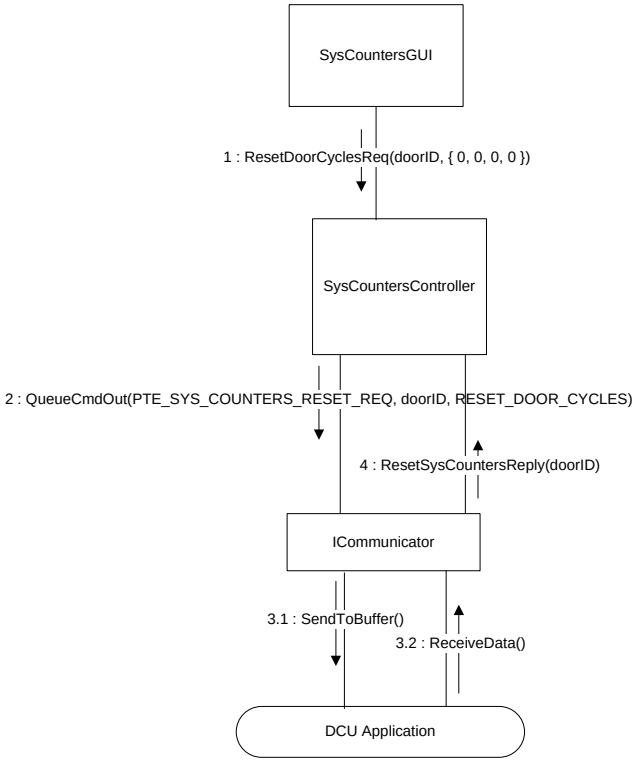


Figure 3-101 Communication Diagram - Reset Door Cycles

3.7.12.2.3 Reset Operating Hours

The reset of operating hours is initiated by the user through the SysCountersGUI. Figure 3-102 presents the Communication Diagram of the Reset operating Hours functionality.

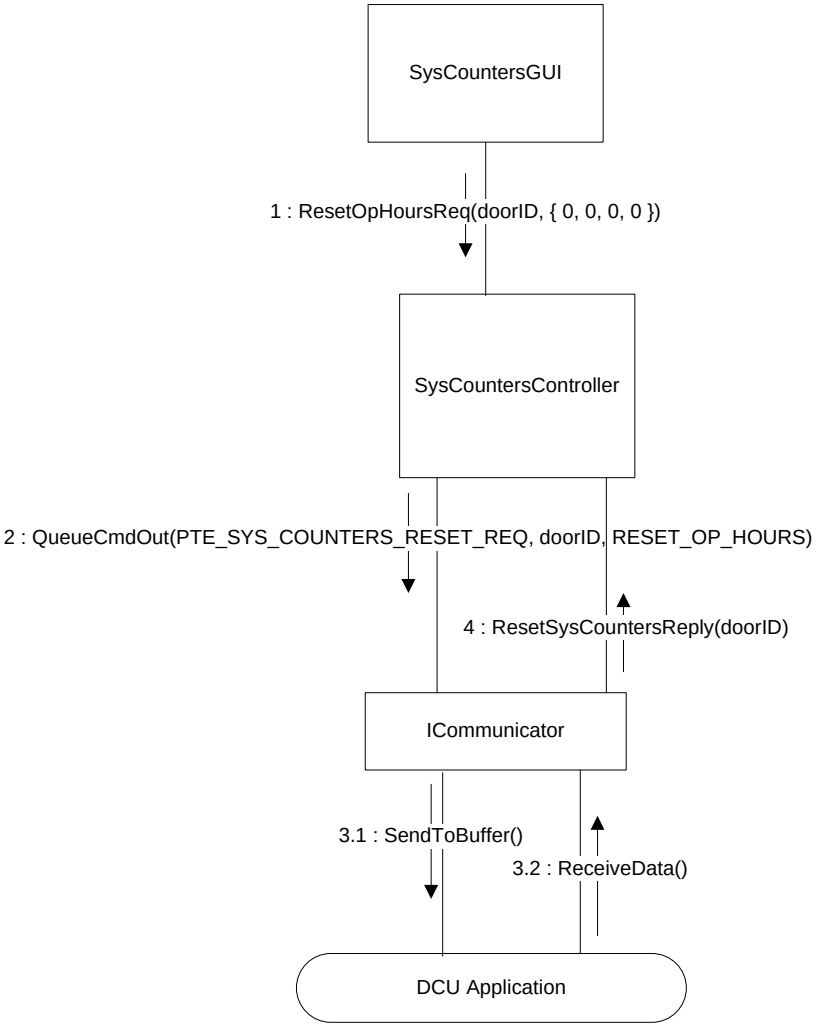


Figure 3-102 Communication Diagram - Reset Operating Hours

3.7.12.2.4 Reset Event Occurrences

The reset of event occurrences is initiated by the user through the SysCountersGUI. Figure 3-103 presents the Communication Diagram of the Reset Event Occurences functionality.

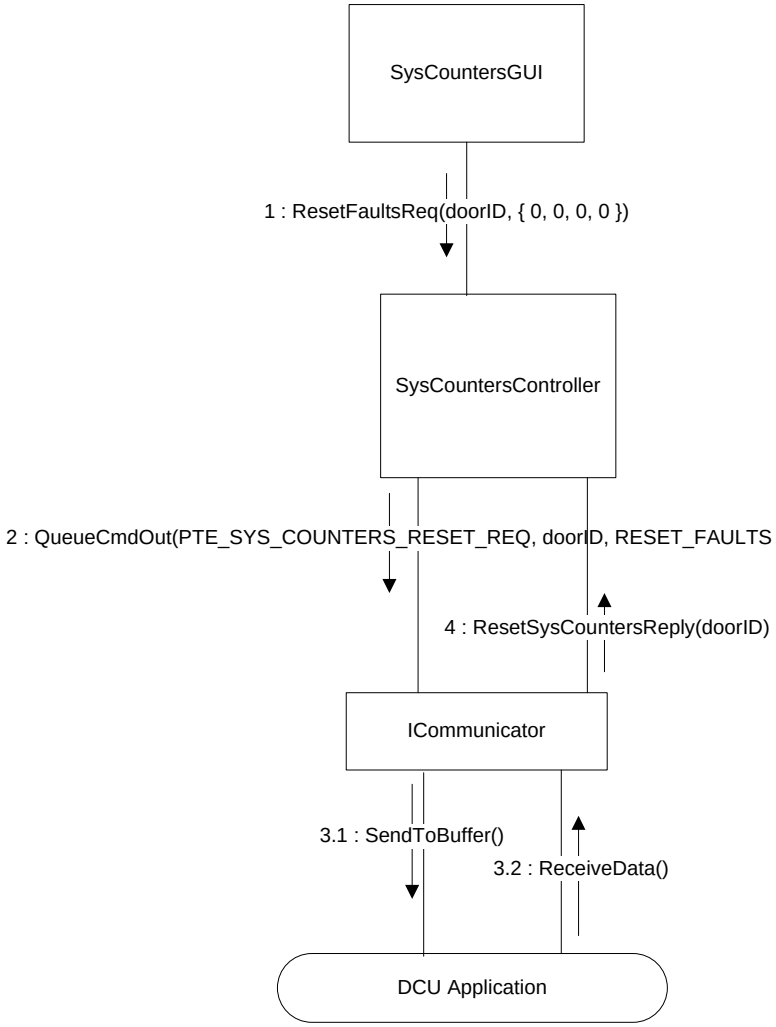


Figure 3-103 Communication Diagram - Reset Event Occurrences

3.7.12.2.5 Reset Last DCU Configuration Occurrence

The reset of the last DCU configuration occurrence is initiated after a successful change of DCU Configuration. The reset is requested by the DCUConfigGUI. Figure 3-104 presents the Communication Diagram of the Reset Last DCU Configuration Occurrence functionality.

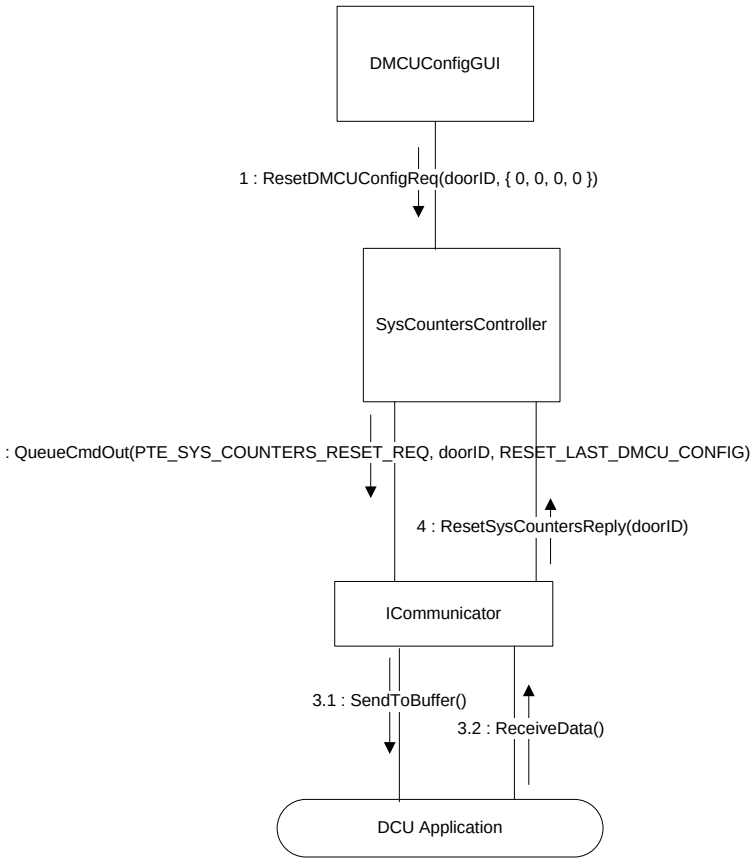


Figure 3-104 Communication Diagram - Reset Last DCU Configuration Occurrence

3.7.12.2.6 Reset Last SW Download Occurrence

The reset of the last SW Download occurrence is initiated after a SW Download has succeeded and SCI versions have been received. This implies that after a successful SWDownload, the PTA will go through the login sequence. The reset is requested by the SWDwlGUI. Figure 3-105 presents the Communication Diagram of the Reset Last Software Download Occurrence functionality.

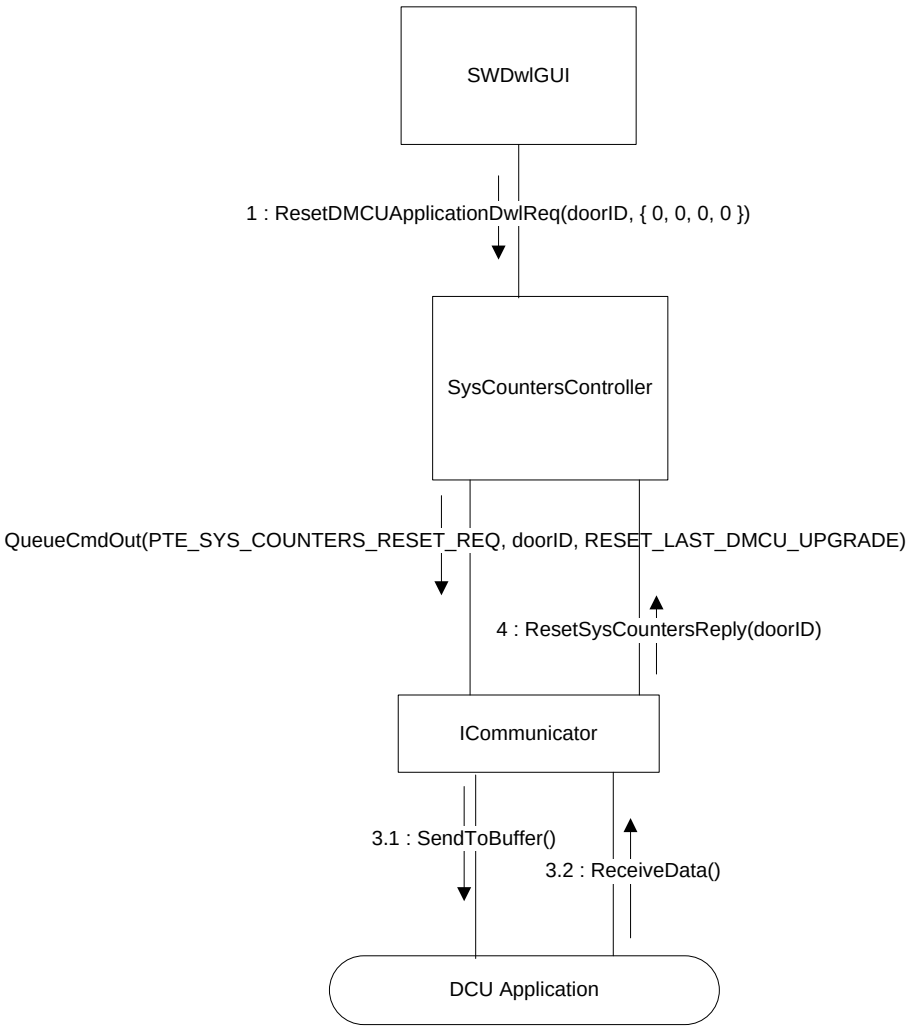


Figure 3-105 Communication Diagram - Reset Last SW Download Occurrence

3.7.12.2.7 Reset Log Erasure Occurrence

The reset of the last log erasure occurrence is initiated after a log erasure has been sent. The reset is requested by the LogErasureGUI. Figure 3-106 presents the Communication Diagram of the Reset Last Log Erasure Occurrence functionality.

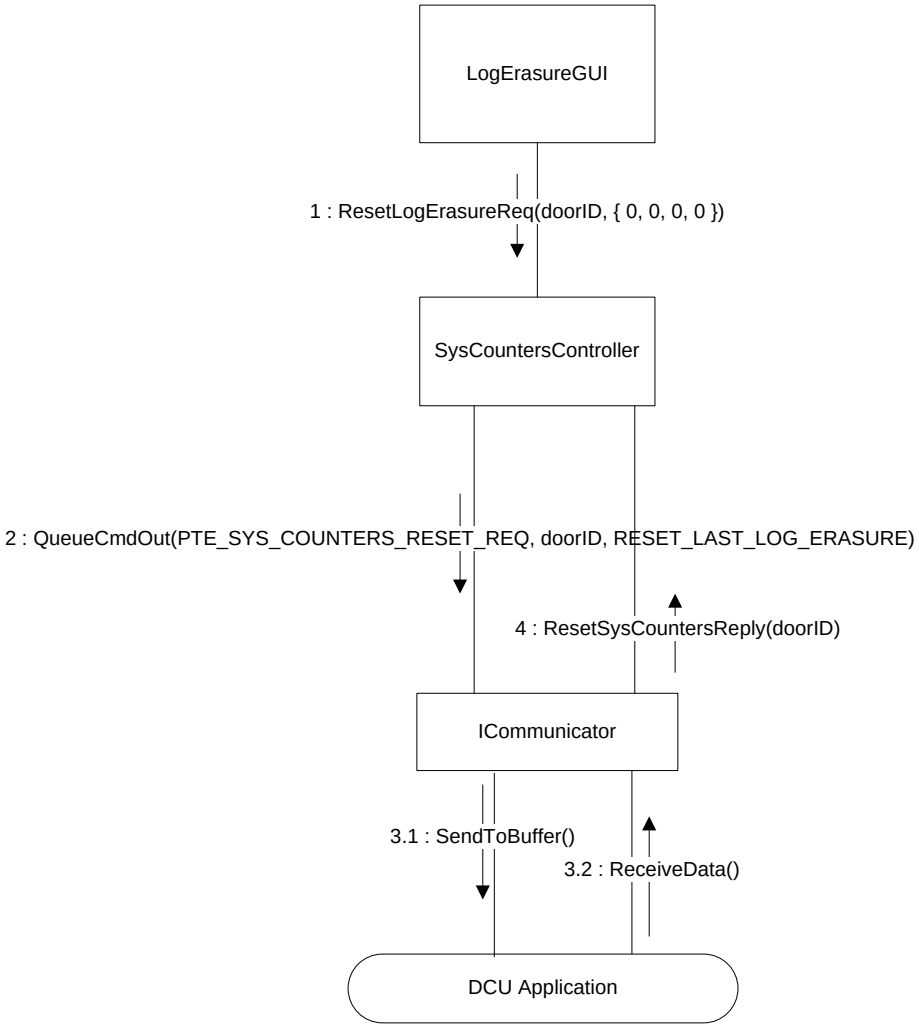


Figure 3-106 Communication Diagram - Reset Last Log Erasure Occurrence

3.7.13 CGM Log Module

The CGM Log Module is composed of two entities. The CGM Log Module Classes are shown in Figure 3-107.

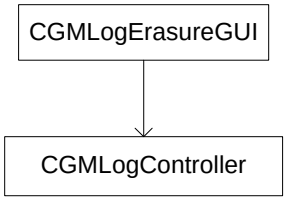


Figure 3-107 CGM Log Module Classes

No data storage is necessary, the GUI provides an interface for commands but does not display any stored information.

3.7.13.1 Interfaces

As per the global architecture described in Section 3.5, the GUI in this module interacts with the PTA Main module by being a child GUI of the MainGUI. The Controller in this module interacts with the Communicator Module. The Controller does not output any feedback regarding the commands, and only accepts input from the GUI.

3.7.13.2 Functionalities

3.7.13.2.1 CGM Log Erasure

The CGM Log Module allows the user to request the erasure of the CGM Network Log file stored on the DCU.

The controller simply sends the erasure command through the usual command transfer protocol. If the command succeeds, the user will get a message to confirm the erasure. If the command is rejected by the DCU, the user will see an error message on the PTA screen.

3.7.14 Project File Module

The Project File Module is responsible for two major features: the project file open and save; the download all. The Project Module Classes are shown in Figure 3-108.

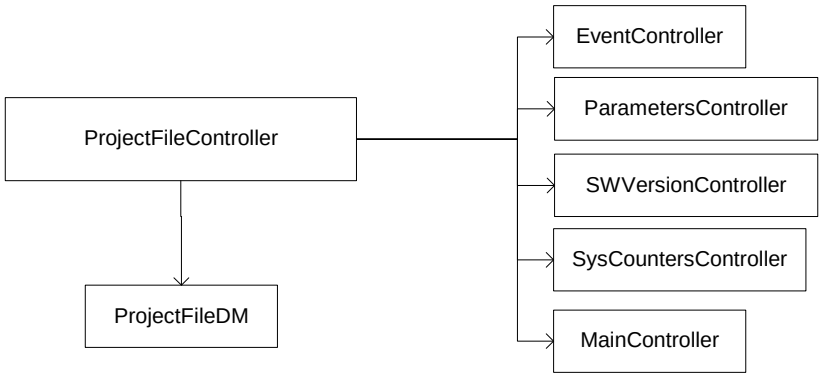


Figure 3-108 Project Module Classes

3.7.14.1 Interfaces

The Project File Module needs information stored by the other modules to save in the file. Because of this, the ProjectFileController class interfaces with the following modules:

- Event & Log Module
- Parameters & Snapshot Module
- Graph Module
- SW Versions Module
- System Counters Module
- Main Module

3.7.14.2 Functionalities

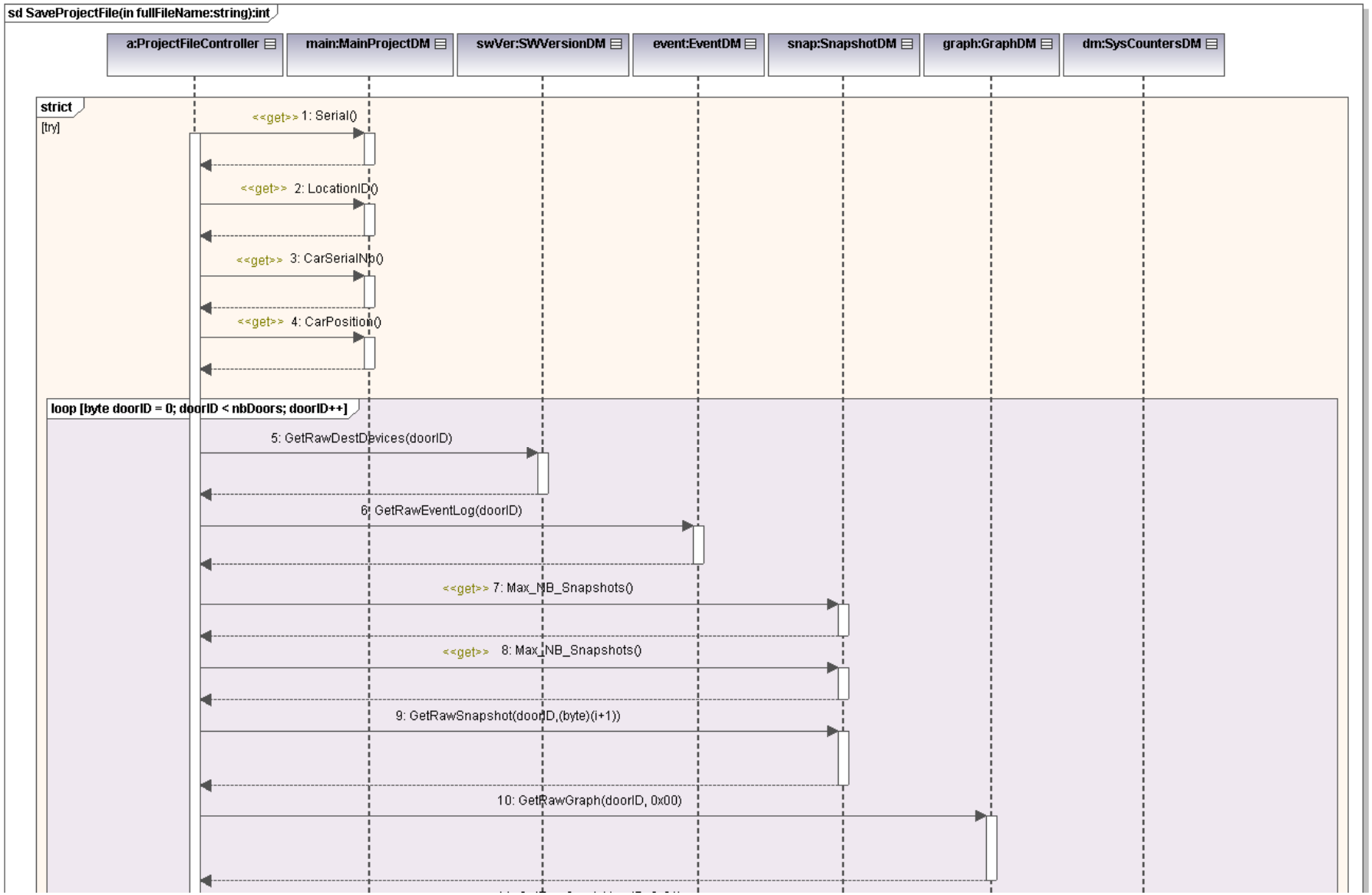
3.7.14.2.1 Save Project File

Saving the Project file saves the following information:

- PTA Version: the version of the PTA
- Serial: A identifier entered by the user in the Control Menu
- Location: The location of the DCU in the car, returned by LocationID request.
- CarSerial: The car number, returned by LocationID request.
- CarPosition: The car's position in a married pair configuration, returned by LocationID request.
- SCI Versions: The software versions loaded into the DCU, returned by the Version request.
- Event Logs: The event logs downloaded from the DCU.
- Snapshots: The Snapshots downloaded from the DCU
- Graph Open: The open graph downloaded from the DCU
- Graph Closes: The Close graph downloaded from the DCU
- System counters: The System Counters downloaded from the DCU.

The information saved is in raw byte format, meaning it is the same untransformed/unparsed byte array received from the DCU Application. The raw bytes are loaded into a data structure (ProjectFileDM) which uses serialization to save as-is on the hard drive. Figure 3-109 presents the Sequence Diagram of the SaveProjectFile() function.

This page has been intentionnaly left blank.


sd SaveProjectFile(in fullFileName:string):int
a:ProjectFileController
main:MainProjectDM
swVer:SWVersionDM
event:EventDM
snap:SnapshotDM
graph:GraphDM
dm:SysCountersDM
strict
[try]
<<get>> 1: Serial()
<<get>> 2: LocationID()
<<get>> 3: CarSerialNb()
<<get>> 4: CarPosition()
loop [byte doorID = 0; doorID < nbDoors; doorID++]
5: GetRawDestDevices(doorID)
6: GetRawEventLog(doorID)
<<get>> 7: Max_NB_Snapshots()
<<get>> 8: Max_NB_Snapshots()
9: GetRawSnapshot(doorID,(byte)(i+1))
10: GetRawGraph(doorID, 0x00)

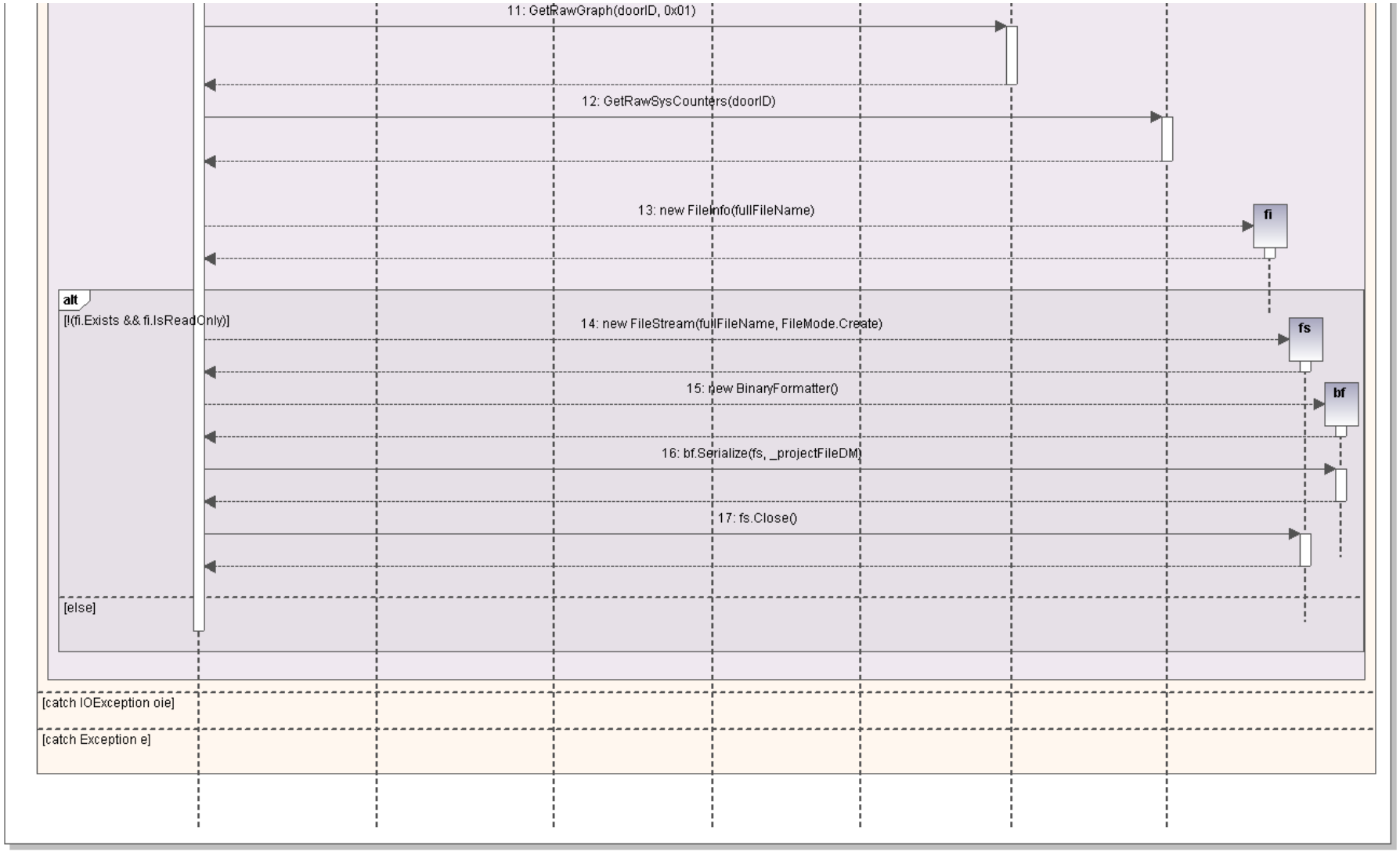


Figure 3-109 Sequence Diagram – SaveProjectFile()

3.7.14.2.2 Open Project File

De-serialization is used to load the data into a structure (the same structure used in the Save feature). The raw bytes for each data mimics the unparsed bytes received from the DCU.

The raw bytes are fed through various controllers via their “Reply” method. The reason they are not placed directly into the data models is that the Reply methods generate events and set flags needed for the overall functioning of the PTA.

3.7.14.2.3 Download All

The Download All feature was integrated to improve usability but is not stated as a requirement.

The Download All functionality downloads files from the DCU in a sequential manner, requiring only one click from the user. The singular downloads behave in the same way as described in their respective modules. The only difference is the automatic call of the next download, when the previous one is completed.

The Download All functionality downloads:

- Event Log
- Snapshots
- Graphs
- System Counters

Figure 3-110 present the Flowchart of the Download All functionality.

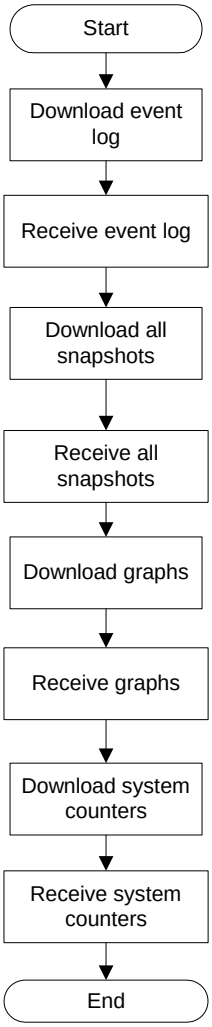


Figure 3-110 Flow Chart - Download All

3.7.15 Reports Module

The Reports Module is composed of GUIs, data sets (not to be confused with data models) and reports. The Reports Module Classes are shown in Figure 3-111.

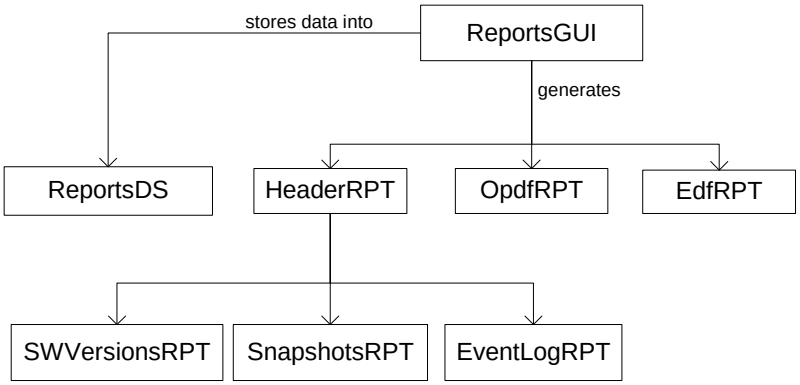


Figure 3-111 Reports Module Classes

The reports are developed with the help of the DevExpress XtraReports 2009 (v9.1) Application. The report's layout and behavior is predefined so that at runtime, the class generating the report only has to pass the data and a few parameters.

Each class of entity has the following purpose:

- Reports GUI: Retrieves and arranges the data into the dataset, then generates the reports.
- HeaderRPT: A master report that generates the Header of the report. It can contain SWVersionRPT, SnapshotRPT and EventsLogRPT
- SWVersionRPT: A sub report that displays the EDF, OPDF and SCI versions.
- SnapshotRPT: A sub report that displays all the downloaded snapshot details, including the value of all the parameters at a given time, in a tabular format.
- EventLogRPT: A sub report that displays the event log.
- OpdfRPT: A stand alone report that displays the OPDF contents.
- EdfRPT: A stand alone report that displays the EDF contents.

3.7.15.1 Interfaces

The Reports Module only interfaces with the Main Module. The Reports GUI is a child of the Main GUI.

The Reports Module interfaces with the external application DevExpress XtraReports 2009 (v9.1).

3.7.15.2 Functionalities

3.7.15.2.1 Print

The Print option allows the user to select the data to be printed. The user can choose the following data entries: File Summary, Event Log and Snapshots. Figure 3-112 shows the process of this function.

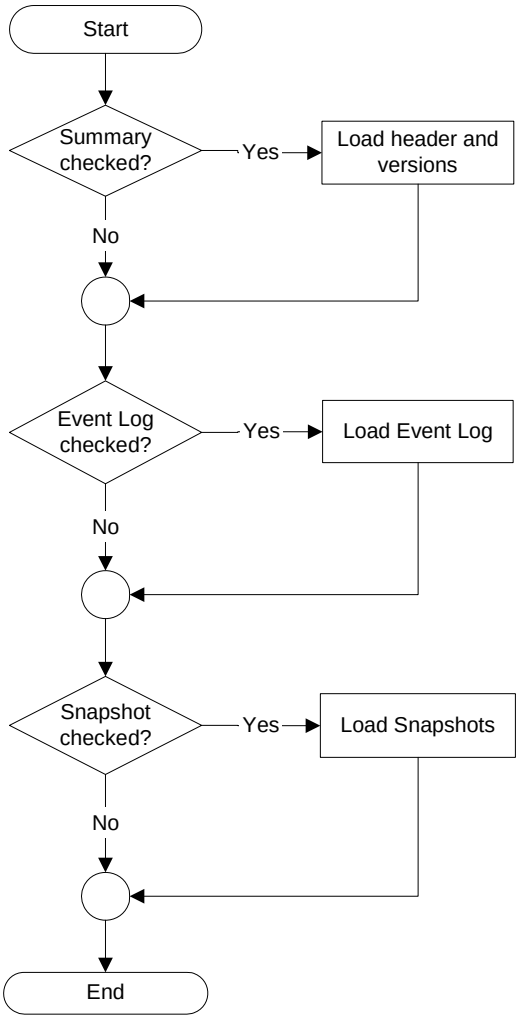


Figure 3-112 Flow Chart - Print

3.7.15.2.2 Print OPDF

The OPDF Print generates a tabular report using the OPDF information loaded into the PTA.

3.7.15.2.3 Print EDF

The EDF Print generates a tabular report using the EDF information loaded into the PTA.

3.7.16 Debug Log Module

The Debug Log Module does not follow the MVC pattern and is not required by the SRS. This module is used by the PTA software developers for debugging purposes.

The Debug Log Module is based partly on the Observer Pattern. Both GUIs update their display when the DataModels notifies them of the change.

The DebugErrorLogDM is based on the Singleton Pattern. This means that the class ensures that only one instance of itself is created by being the sole owner of that instance. It has a property called *Instance*, which is made public and available for retrieval by all other classes.

Upon first retrieval of that instance, the property method checks if there is an instance created, and if not, it creates it. Figure 3-113 shows the Classes of this module.

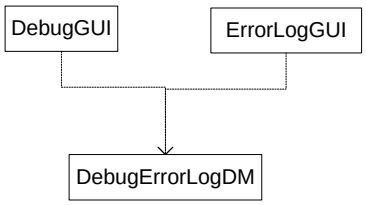


Figure 3-113 Debug Log Module Classes

3.7.16.1 Interfaces

In the same fashion as the other module, the Debug GUIs are child GUIs of the Main GUI.

The DebugErrorLogDM, however, follows the Singleton pattern, meaning the class contains the reference to its own instance thus ensuring that no other instances of the class can be created. This instance is made public to all classes, therefore the class interfaces with all other modules. DebugErrorLogDM ignore all modules that might access it.

3.7.16.2 Functionalities

3.7.16.2.1 Debug Log

When a new debug message is sent to the debug log, the DebugErrorLogDM appends the timestamp to the beginning of the message and a carriage return at the end of the message. This is added to a String Builder that keeps the entire debug log.

When the log is full the function automatically removes the first 1/3 of the log. The maximum log size is 250000 characters (equal to 250000 bytes). Figure 3-114 shows the Sequence Diagram of the AddCommLog() function.

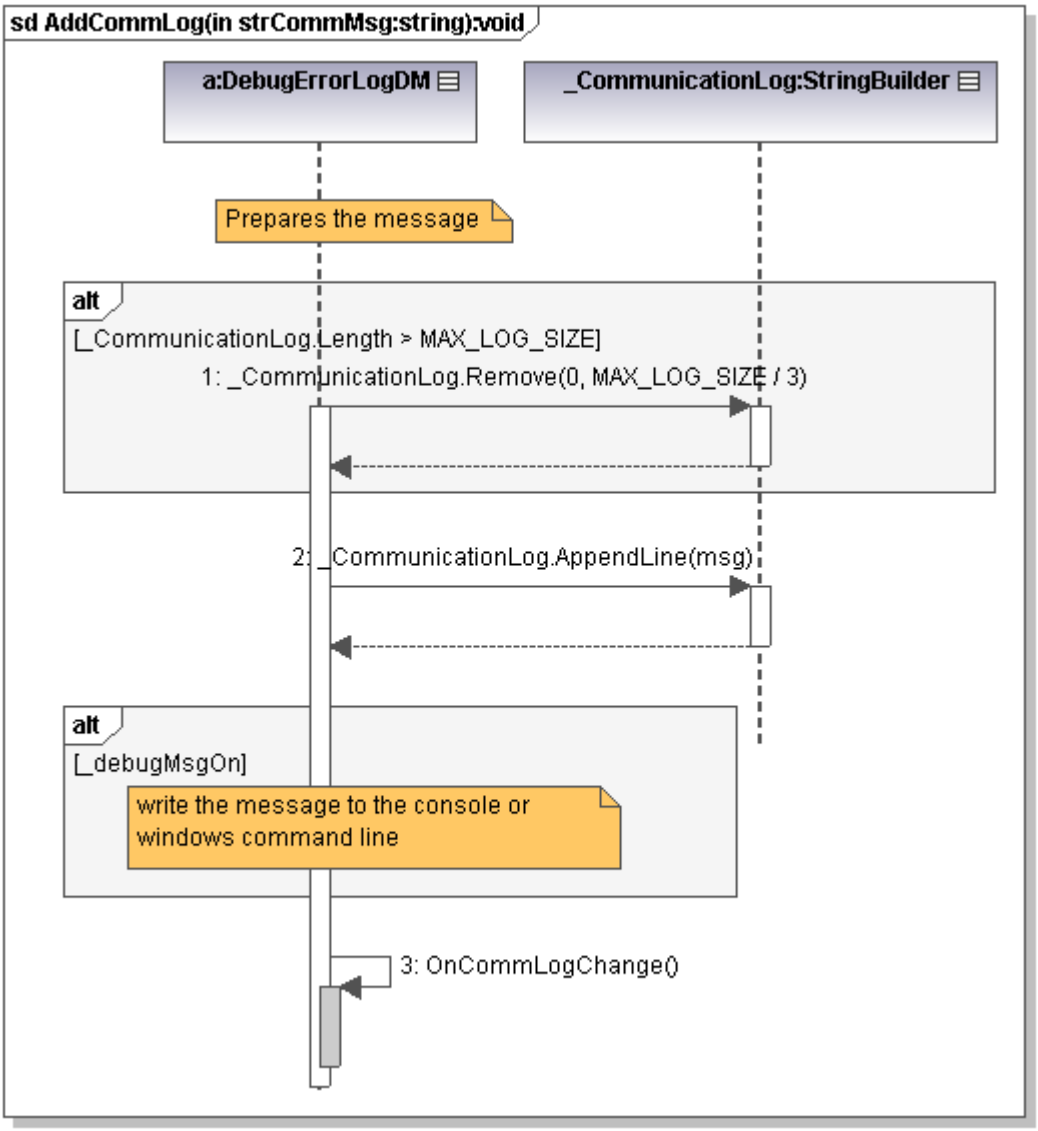


Figure 3-114 Sequence Diagram - AddCommLog()

3.7.16.2.2 Error Log

When a new error message is sent to the error log, the DebugErrorLogDM appends the timestamp to the beginning of the message and a carriage return at the end of the message. This is added to a String Builder that keeps the entire error log.

When the log is full the function automatically removes the first 1/3 of the log. The maximum log size is 250000 characters (equal to 250000 bytes). Figure 3-115 shows the Sequence Diagram of the AddErrLog() function.

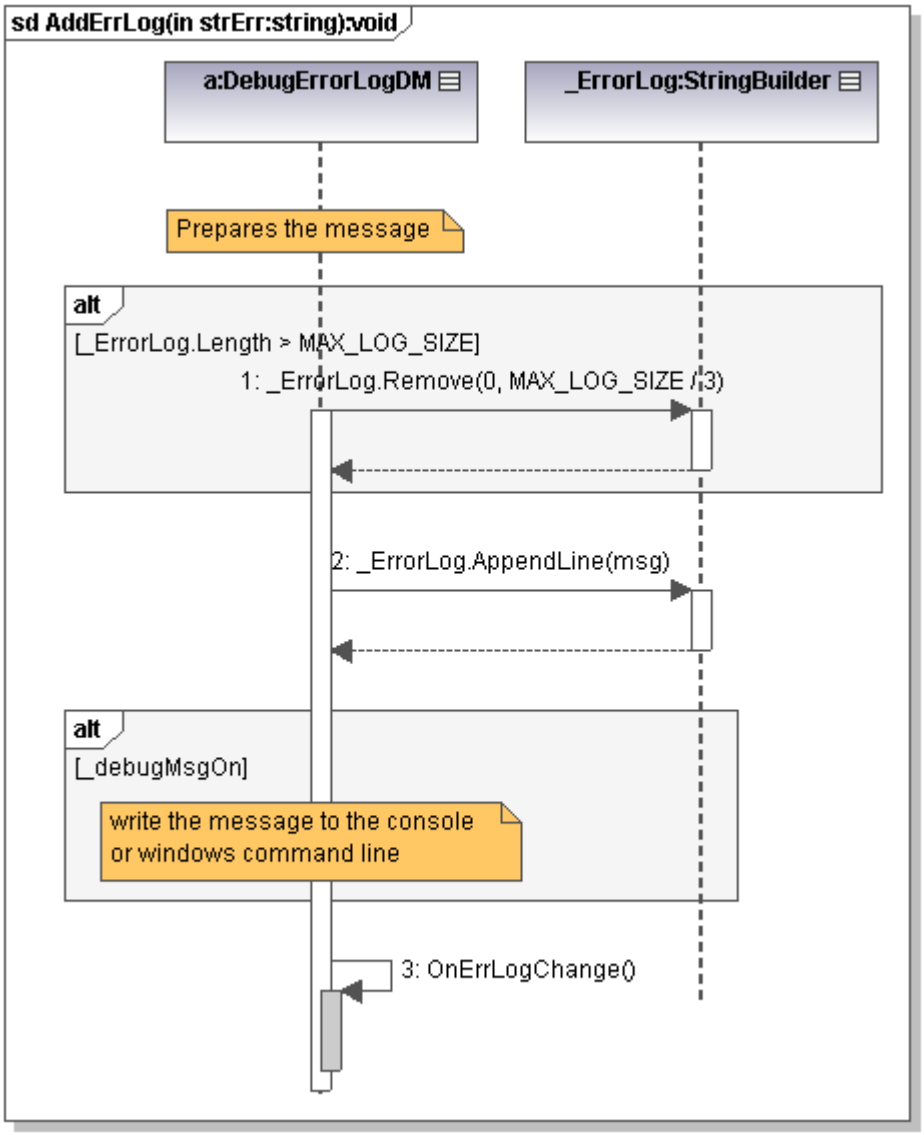


Figure 3-115 Sequence Diagram - AddErrLog()

APPENDIX A: CONFIGURABLE PARAMETERS

The data contained in this appendix are project's specific, the values given hereafter are the default values. The project's specific values are detailled in the SACS document.

- **Internal SRAM start adress :** 0x0000
- **Internal NVRAM start adress :** 0x1200

The PTA can directly change a number of parameters describing the behavior of the DCU applications. All the parameters listed below are described here with their default range and their values are defined in the Software Applications Configuration Specifications document, SACS.

Table 4-1: Generic Parameters

MNEMONIC	Description	Range
SACS.Door open&close		
SACS.Door.DefaultOpeningDelay	After receiving an open command, the door will not move during the opening delay, and start to open only after the end of it. (Seconds)	1.0-2.0
SACS.Door.DefaultOpeningTime	The default time taken by the door to go from totally closed to within 3 inches of the fully open position. (Seconds)	SACS. Door.O pening Time.R ange
SACS.Door.OpeningTime.Range	The value of the opening time can be changed within the opening time range through the PTA. (Seconds)	N/A
SACS.Door.DefaultClosingDelay	After receiving a Close command, the door will not move during the Closing Delay, and start to close after the end of it. (Seconds)	1.0-2.5
SACS.Door.DefaultClosingTime	Default Time taken by the door to go from totally open to totally closed. (Seconds)	SACS. Door.Cl osingTi me.Ra nge
SACS.Door.ClosingTime.Range	The closing time value can be changed within the closing time range. (Seconds)	N/A
SACS.Door.CrewSwitch		
SACS.Door.CrewSwitch.Outside.Closi ngDelay	After receiving a Close command from the Outside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0

MNEMONIC	Description	Range
SACS.Door.CrewSwitch.Inside.ClosingDelay	After receiving a Close command from the Inside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0
SACS.Door.CrewSwitch.Outside.ClosingTime	The time taken by the door to go from totally open to totally close when commanded to close through the outside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.Inside.ClosingTime	The time taken by the door to go from totally open to totally close when commanded to close through the inside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.Outside.OpeningDelay	After receiving a Close command from the Outside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0
SACS.Door.CrewSwitch.Inside.OpeningDelay	After receiving a Close command from the Inside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0
SACS.Door.CrewSwitch.Outside.OpeningTime	The time taken by the door to go from totally close to totally open when commanded to close through the outside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.Inside.OpeningTime	The time taken by the door to go from totally close to totally open when commanded to close through the inside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.LinkTiming	When this parameter is equal to 1, the Crew Switches timing values are linked with the default door values for opening/closing delay & time. If the timing values are independent, the parameter is equal to 0.	0-1
SACS.Door.Obstruction		
SACS.Door.Obstruction.Distance	After detecting an obstruction, the door will move from the obstruction opening distance in the open direction.(Inches)	0-10
SACS.Door.Obstruction.Count	After detecting an obstacle, the door will try to open and re-close for a maximum number of time equals to the obstruction count.	0-4

MNEMONIC	Description	Range
SACS.Door.Obstruction.EndBehavior	After having reached the maximum number of retries of closing the door, the door will adopt the behavior selected by the Obstruction End Behavior value. Value 1 means that the door will fully re-open.	0-2
SACS.Door.Obstruction.Range	The obstruction detection is working inside this obstruction range, measured in inches from the fully open position.(Inches)	0-10
SACS.Door.AudibleAlarmActivation	The behavior of the audible alarm upon opening or closure is selected by this parameter. Value 0 means the alarm will sounds as soon as the command is received, until the door stop moving.	0-1
SACS.Door.VisualAlarmActivation	The behavior of the audible alarm upon opening or closure is selected by this parameter. Value 0 means the alarm will blink as soon as the close command is received, until the door stop moving.	0-1
SACS.Door.SecondaryObstruction		
SACS.Door.SecondaryObstruction.Position	Position of the door, closed but not locked, stopped to detect a secondary obstruction. The position is measured in 10^{-2} inches from the locked position.	TBD
SACS.Door.SecondaryObstruction.Activation	If equal to 1, the secondary obstruction function is activated, if equal to 0, desactivated.	Boolean
SACS.Door.SecondaryObstruction.Delay	Delay during which the door stay stopped in closed position to detect an obstruction.(Milliseconds)	TBD
SACS.Door.EmergencyHandle		
SACS.Door.EmergencyHandle.ClosePower	If the parameter is equal to 1, Close power will be activated if motion trainline is active and the Emergency handle pulled. If the value is 0, close power will be inactive.	Boolean
SACS.Door.Snapshot		
SACS.Door.Snapshot.Period	The number of snapshot samples taken per second.	0 to 100
SACS.Door.Snapshot.Capacity	The maximum number of Snapshot samples which can be kept simultaneously in memory in a single Snapshot file.	0 to 41
SACS.Door.Snapshot.NbOfFiles	Maximum number of Snapshot Files saved in memory simultaneously.	0 to 20